

The image shows a well-lit industrial networking lab. In the foreground, there is a long wooden workbench on a black frame with multiple drawers. A black ergonomic chair is positioned in front of it. The background is filled with rows of black equipment racks containing various electronic components and wiring. A projector is mounted on the ceiling, and the room has a drop ceiling with fluorescent lights.

INDUSTRIAL NETWORKING

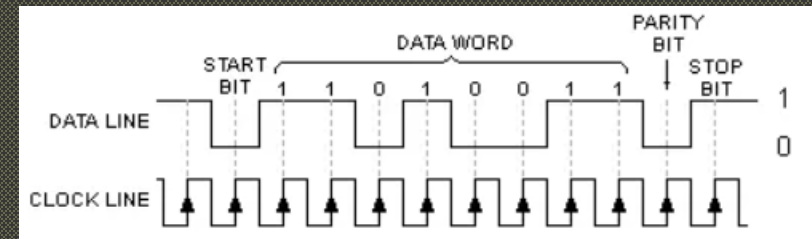
MENG 4010

CONTENTS

- 3. Serial Data Communications and Fieldbuses
 - 3.1 Serial data communications standards
 - 3.2 EIA/TIA – 232 standard
 - 3.3 RS-423 standard
 - 3.4 RS-422 standard
 - 3.5 RS-485 standard
 - 3.6 Comparison of standards
 - 3.7 The universal serial bus (USB)
 - 3.8 Introduction to CAN
 - 3.9 CAN & CANOpen
 - 3.10 CANopen

3.1 SERIAL DATA COMMUNICATIONS STANDARDS

- Serial communication sends data one bit at a time over a communication link.
- Interface standards define the electrical and physical rules that allow devices from different manufacturers to communicate.
- Common serial interfaces covered in this course:
 - RS-232
 - RS-423
 - RS-422
 - RS-485
 - USB
- The RS interfaces can be grouped by signaling method:
 - Unbalanced / single-ended
 - RS-232
 - RS-423
 - Balanced / differential
 - RS-422
 - RS-485
- USB is also serial communication, but it is a different interface standard and will be discussed separately



<https://ipc2u.com/articles/knowledge-base/the-main-differences-between-rs-232-rs-422-and-rs-485/>

3.1 SERIAL DATA COMMUNICATIONS STANDARDS

TRANSMISSION LINES: UNBALANCED VS. BALANCED

- Serial communication can use:
 - Unbalanced transmission
 - Balanced transmission

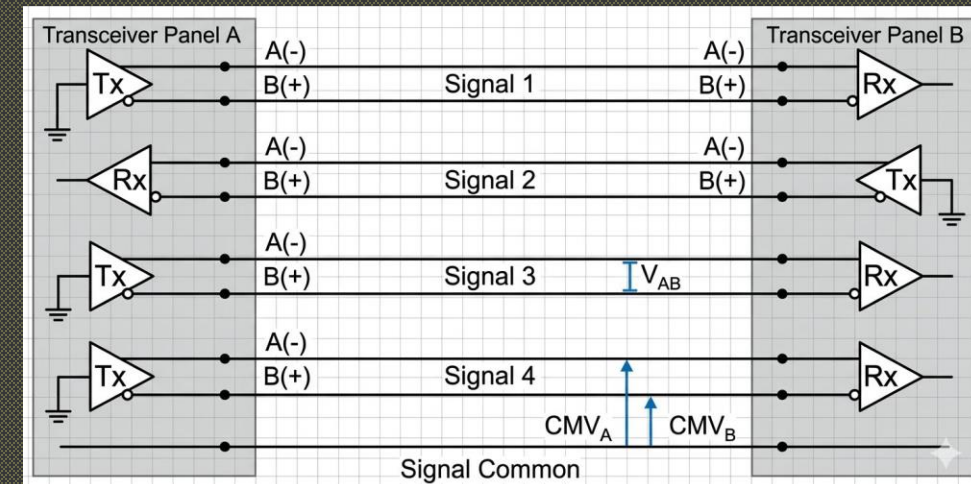
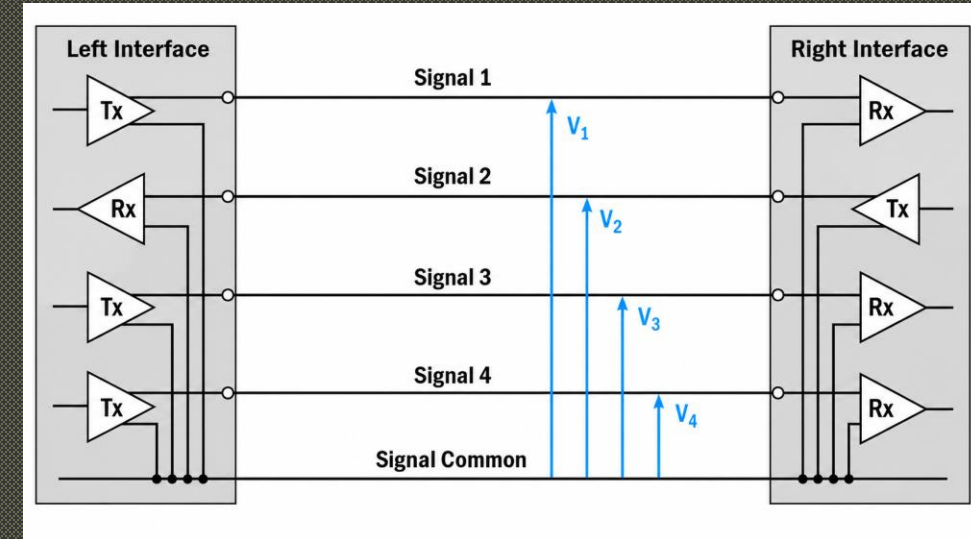
Unbalanced

- Uses a signal conductor referenced to a common ground.
- Receiver measures the signal voltage relative to ground.
- Simpler wiring.
- More sensitive to:
 - Electrical noise
 - Ground-potential differences

Balanced

- Uses two conductors for each signal.
- Receiver measures the voltage difference between the two conductors.
- Provides better rejection of electrical noise.
- Better suited to electrically noisy industrial environments.

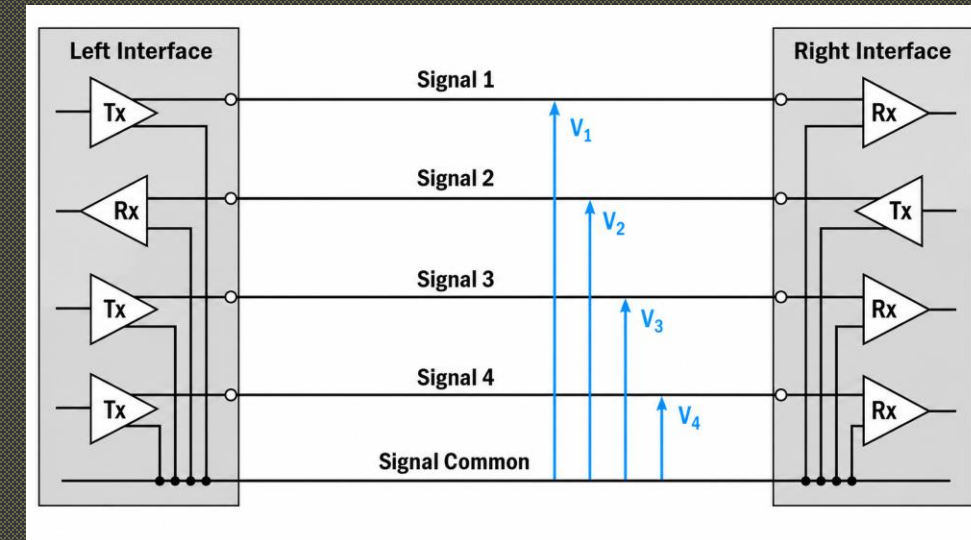
- Unbalanced = Signal – Ground
- Balanced = Wire A – Wire B



3.1 SERIAL DATA COMMUNICATIONS STANDARDS

UNBALANCED TRANSMISSION

- Uses:
 - Signal conductor
 - Common reference/ground
- The receiver measures signal voltage relative to the common reference.
- Several signals may share the same reference conductor.
- Performance can be affected by:
 - Resistance in the ground conductor
 - Ground-potential differences
 - EMI
 - Cable length
- Generally better suited to shorter and less electrically noisy connections.
- Examples:
 - RS-232
 - RS-423



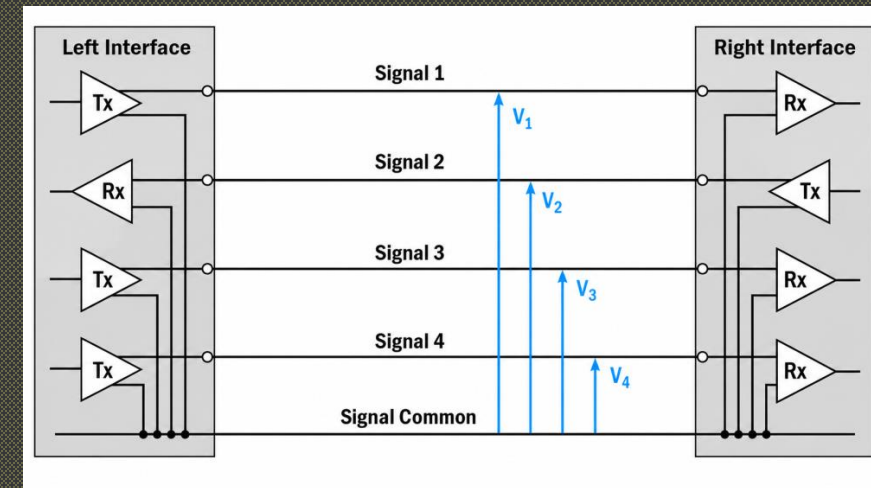
$$V_{\text{signal}} = V_{\text{signal conductor}} - V_{\text{ground}}$$

Unbalanced systems have a large loop area between the signal conductor and the ground conductor.

3.1 SERIAL DATA COMMUNICATIONS STANDARDS

WHY UNBALANCED COMMUNICATION IS MORE NOISE-SENSITIVE

- Real ground conductors are not perfect zero-voltage references.
- Ground voltage may differ between the transmitter and receiver.
- Electrical noise can be induced into:
 - Signal conductor
 - Ground/reference conductor
- If the reference voltage changes, the voltage seen by the receiver also changes.
- Longer wiring can increase these effects.
- Shielding can reduce interference when installed correctly.



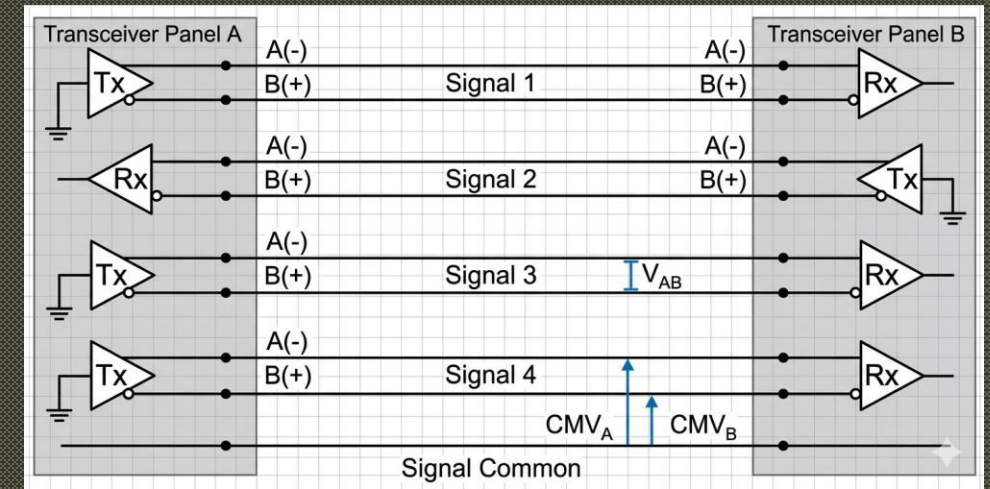
Real ground wires have some resistance and can carry current, so their voltage can shift slightly instead of staying at exactly 0 V. If the transmitter and receiver are connected to different ground points, those points may be at slightly different voltages, creating a ground potential difference.

Unbalanced communication depends heavily on a stable reference voltage.

3.1 SERIAL DATA COMMUNICATIONS STANDARDS

BALANCED/DIFFERENTIAL TRANSMISSION

- Uses two conductors for a signal.
- Receiver measures:
 $V_{diff} = V_A - V_B$
- External noise often affects both wires in a similar way.
- The receiver subtracts the two voltages, so much of the common noise can be rejected.
- Advantages:
 - Better noise immunity
 - Better performance over longer cable runs
 - Well suited to industrial environments
- Examples:
 - RS-422
 - RS-485



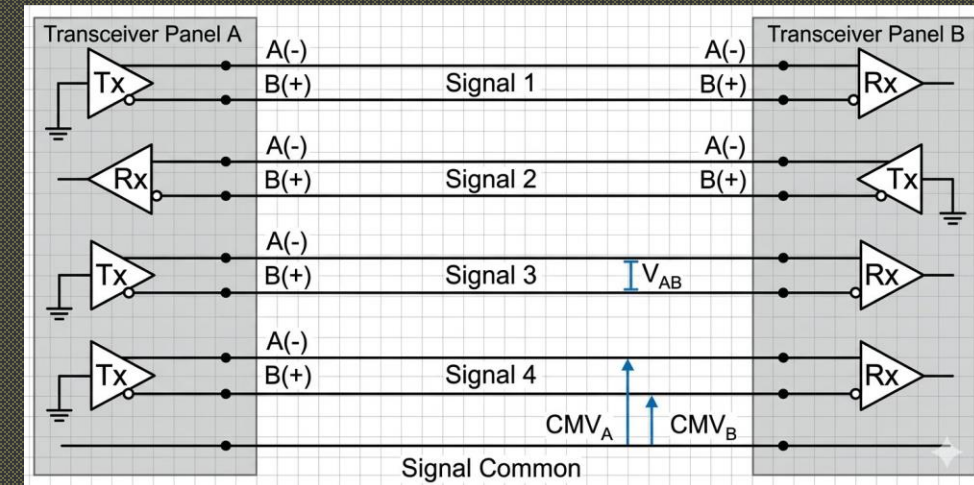
The receiver responds mainly to the difference between the wires, not their voltage relative to ground.

3.1 SERIAL DATA COMMUNICATIONS STANDARDS

COMMON-MODE VOLTAGE AND CMRR

- Common-Mode Voltage (CMV)
 - Voltage appearing approximately equally on both conductors of a differential pair.
- Sources can include:
 - EMI
 - Ground-potential differences
 - Motors
 - Drives
 - High-current equipment
- CMRR – Common-Mode Rejection Ratio
 - Indicates how well a differential receiver rejects common-mode signals.
- Higher CMRR means:
 - Better noise rejection
 - Cleaner received signal

- CMV = voltage common to both signal wires.
- Differential receivers attempt to reject CMV.
- CMRR measures how effectively the receiver rejects it.
- High CMRR is desirable in electrically noisy environments.
- Excessive CMV can still exceed the receiver's allowed input range.



3.1 SERIAL DATA COMMUNICATIONS STANDARDS

DIFFERENTIAL NOISE-REJECTION EXAMPLE

Example:

- Original:

- $V_A = +2.5 \text{ V}$
- $V_B = -2.5 \text{ V}$

Therefore:

$$V_{\text{diff}} = 2.5 - (-2.5) = 5 \text{ V}$$

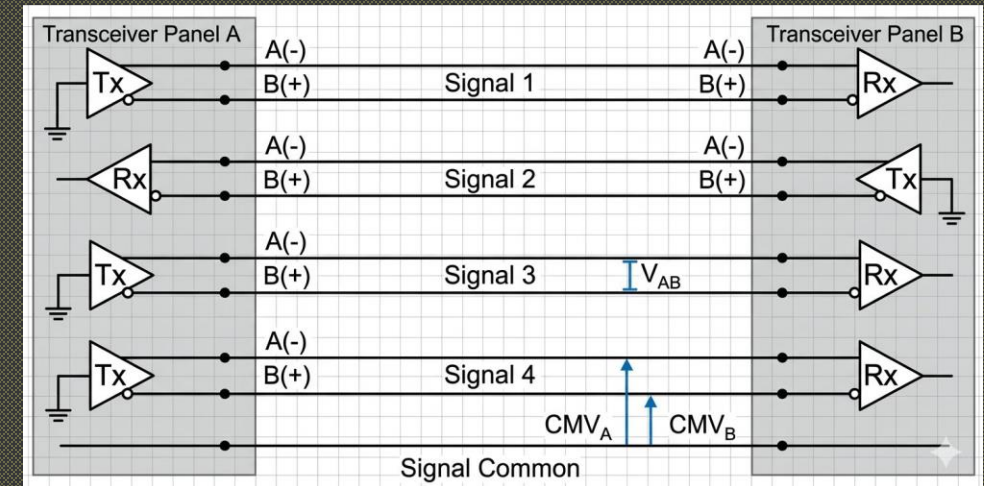
If +15 V of common noise appears on both wires:

- $V_A = 17.5 \text{ V}$
- $V_B = 12.5 \text{ V}$

Then:

$$V_{\text{diff}} = 17.5 - 12.5 = 5 \text{ V}$$

The differential signal is unchanged, so the receiver still interprets the correct data.



Common Mode Voltage, CMV: 15 V. It is a shared unwanted signal.

- If it exceeds the receiver's input limits, it may cause data corruption or even damage
- It does not affect the receiver as long as:
 - The CMV stays within the receiver's common-mode input range (usually around -7 V to +12 V for RS-485).
 - The receiver has a high common-mode rejection ratio (CMRR).

3.1 SERIAL DATA COMMUNICATIONS STANDARDS

COMPARING UNBALANCED AND BALANCED INTERFACES

Unbalanced

- Uses signal + common reference.
- Simpler.
- More sensitive to noise.
- Examples:
 - RS-232
 - RS-423

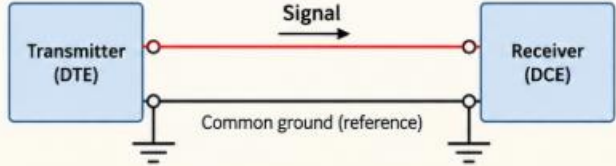
Balanced

- Uses a differential pair.
- Better noise immunity.
- Better suited to longer and noisier industrial links.
- Examples:
 - RS-422
 - RS-485

Comparing Unbalanced and Balanced Serial Communication Interfaces

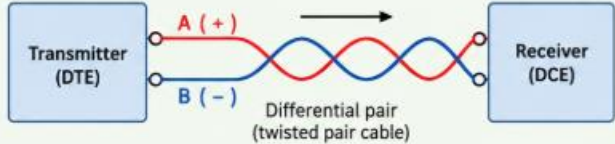
Unbalanced (RS-232 / RS-423)

- Uses one signal conductor referenced to a common ground.
- Simpler wiring.
- More sensitive to noise.
- Examples: RS-232, RS-423.



Balanced (RS-422 / RS-485)

- Uses two conductors (differential pair).
- Receiver measures the voltage difference between the two wires.
- Better noise immunity.
- Better suited to longer and noisier industrial links.
- Examples: RS-422, RS-485.



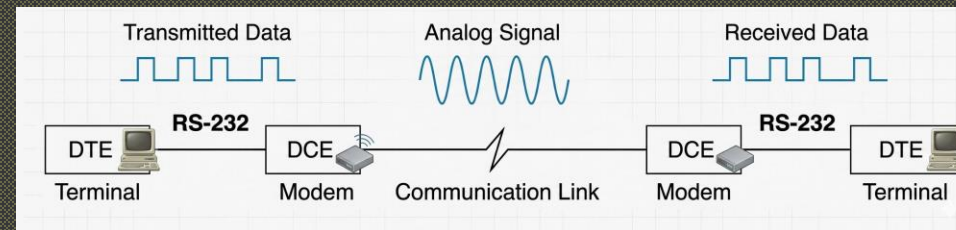
Feature	Unbalanced (RS-232 / RS-423)	Balanced (RS-422 / RS-485)
Wiring	1 signal + ground	2 twisted wires (differential pair)
Noise Immunity	Poor (more sensitive to noise)	Excellent (common-mode rejection)
Typical Distance	Short (≈ 15 m)	Long (up to ≈ 1,200 m, depending on standard)
Multi-device Support	No (typically point-to-point)	Yes (especially RS-485)
Common Standards	RS-232, RS-423	RS-422, RS-485

Key Point: Unbalanced interfaces are simpler but more sensitive to noise, while balanced interfaces provide better noise immunity and longer communication distances, making them well suited for industrial environments.

3.2 TIA/EIA-232 STANDARD INTRODUCTION

- RS-232 is a serial communication interface mainly designed for communication between:
 - DTE – Data Terminal Equipment
 - DCE – Data Communication Equipment
- Examples:
 - DTE → computer, PLC programming terminal
 - DCE → modem or communication device
- RS-232 defines:
 - Electrical characteristics
 - Signal functions
 - Interface connections
- RS-232 defines the interface, not the application protocol or meaning of the transmitted data.
- Uses unbalanced / single-ended signaling.
- Commonly found today in:
 - Legacy industrial equipment
 - PLC programming/configuration ports
 - Drives and instruments
 - Serial consoles and diagnostic interfaces

ITU - International Telecommunication Union
ISO - International Organization for Standardization
EIA - Electronic Industries Alliance
TIA - Telecommunications Industry Association



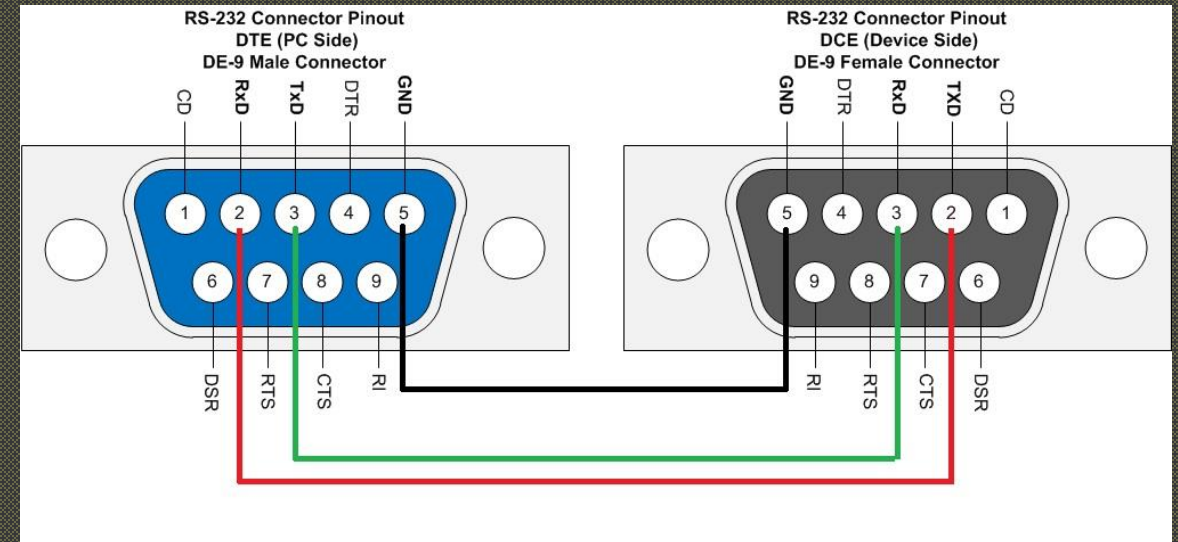
RS-232 = short-distance, point-to-point, single-ended serial communication.

3.2 TIA/EIA-232 STANDARD

DTE and DCE Data Connections

- DTE and DCE use separate lines for transmitting and receiving data.
 - On the common DE-9 DTE connection:
 - Pin 2 = RxD – Receive Data
 - Pin 3 = TxD – Transmit Data
 - Pin 5 = Signal Ground
 - DCE transmit and receive functions are complementary to those of the DTE.
 - A normal DTE-to-DCE connection therefore connects:
 - DTE TxD → DCE RxD
 - DCE TxD → DTE RxD
 - Ground → Ground
-
- TxD sends data
 - RxD receives data
 - DTE and DCE are designed so these signals normally connect correctly with a straight-through connection.

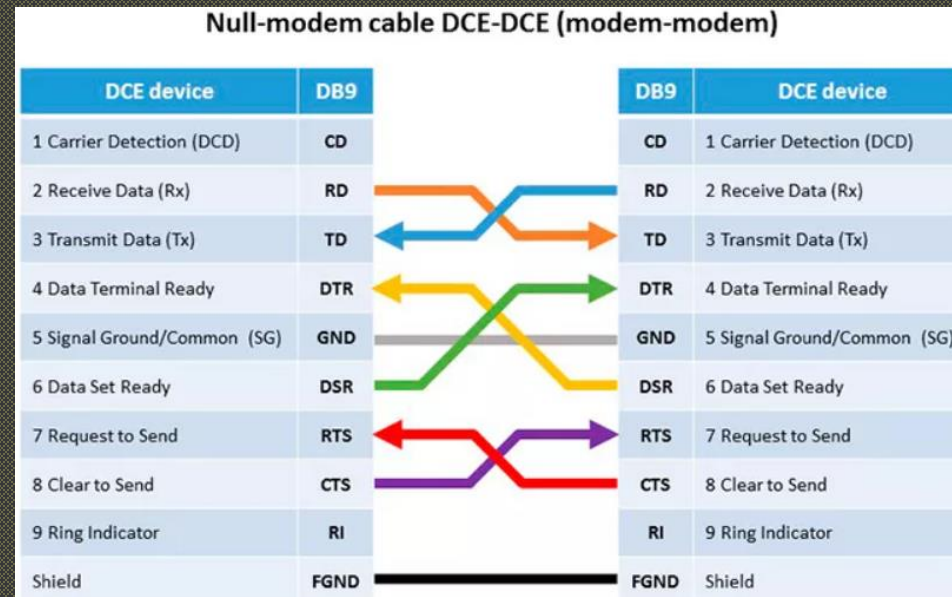
A straight-through or "one to one" cable is used to connect a DTE device to a DCE device. The transmit and receive lines are not cross-connected in this case, hence the name.



3.2 TIA/EIA-232 STANDARD

Null Modem and Handshaking Connections

- A null modem cable is used when connecting two DTE devices directly.
- Since both devices use the same transmit and receive pins:
 - TxD on one device must connect to RxD on the other.
 - RxD must connect to TxD.
- This is called a crossover connection.
- If hardware handshaking is used, control signals may also need to be crossed.
- Common flow-control signals:
 - RTS – Request to Send
 - CTS – Clear to Send



- DTE ↔ DCE → normally straight-through
- DTE ↔ DTE → normally null modem / crossover

3.2 TIA/EIA-232 STANDARD

Hardware vs. Software Handshaking

- Handshaking controls when devices are ready to send or receive data.

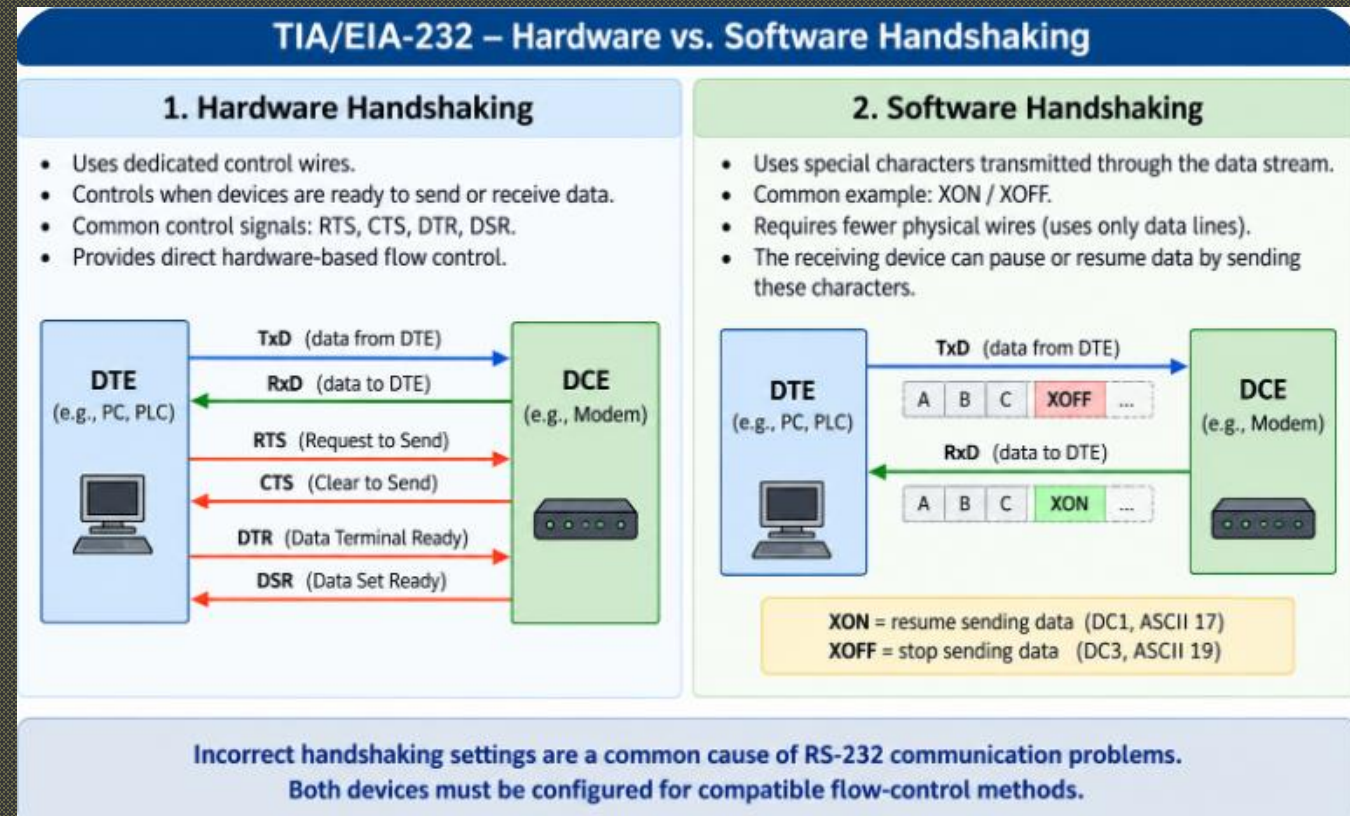
- Two common methods:

1. Hardware Handshaking

- Uses dedicated control wires.
- Common signals:
 - RTS
 - CTS
 - DTR
 - DSR
- Provides direct hardware-based flow control.

2. Software Handshaking

- Uses characters transmitted through the data stream.
- Common example:
 - XON / XOFF
- Requires fewer physical wires.



- Incorrect handshaking settings are a common cause of RS-232 communication problems.
- Both devices must be configured for compatible flow-control methods.

3.2 TIA/EIA-232 STANDARD – CONNECTION TYPES

Three common connection situations are:

- DTE ↔ DCE
 - Normally uses a straight-through connection.
- DTE ↔ DTE
 - Normally requires a null-modem / crossover connection.
- DCE ↔ DCE
 - May also require crossover wiring depending on the equipment.

Important signals

- TxD → transmitted data
- RxD → received data
- Signal Ground → common electrical reference
- Additional control signals may be used when handshaking is required.

Before connecting two RS-232 devices, determine: Is each device DTE or DCE?

1. Data Lines:

- Pin 2 (TxD): The computer transmits data to the modem.
- Pin 3 (RxD): The computer receives data from the modem.

2. Control/Handshaking Signals: (Used for flow control and status)

- Ready Signals: Pin 4 (DTR) and Pin 6 (DSR) indicate if the computer and modem, respectively, are powered on and ready.
- Send/Receive Signals: Pin 7 (RTS) is a request from the computer to send data, and Pin 8 (CTS) is the modem's approval to send.
- Connection Signals: Pin 1 (DCD) indicates a live connection, and Pin 9 (RI) signals an incoming call.

3. Ground:

- Pin 5 (SG): Provides a common ground reference voltage essential for all signals.

Direct DTE cable - DCE (computer modem)

DTE computer	DB9		DB9	DCE device
1 Carrier Detection (DCD)	CD	←	CD	1 Carrier Detection (DCD)
2 Receive Data (Rx)	RD	←	RD	2 Receive Data (Rx)
3 Transmit Data (Tx)	TD	→	TD	3 Transmit Data (Tx)
4 Data Terminal Ready	DTR	→	DTR	4 Data Terminal Ready
5 Signal Ground/Common (SG)	GND	→	GND	5 Signal Ground/Common (SG)
6 Data Set Ready	DSR	←	DSR	6 Data Set Ready
7 Request to Send	RTS	→	RTS	7 Request to Send
8 Clear to Send	CTS	←	CTS	8 Clear to Send
9 Ring Indicator	RI	←	RI	9 Ring Indicator
Shield	FGND	→	FGND	Shield

Null-modem cable DCE-DCE (modem-modem)

DCE device	DB9		DB9	DCE device
1 Carrier Detection (DCD)	CD	→	CD	1 Carrier Detection (DCD)
2 Receive Data (Rx)	RD	→	RD	2 Receive Data (Rx)
3 Transmit Data (Tx)	TD	←	TD	3 Transmit Data (Tx)
4 Data Terminal Ready	DTR	←	DTR	4 Data Terminal Ready
5 Signal Ground/Common (SG)	GND	→	GND	5 Signal Ground/Common (SG)
6 Data Set Ready	DSR	→	DSR	6 Data Set Ready
7 Request to Send	RTS	←	RTS	7 Request to Send
8 Clear to Send	CTS	→	CTS	8 Clear to Send
9 Ring Indicator	RI	→	RI	9 Ring Indicator
Shield	FGND	→	FGND	Shield

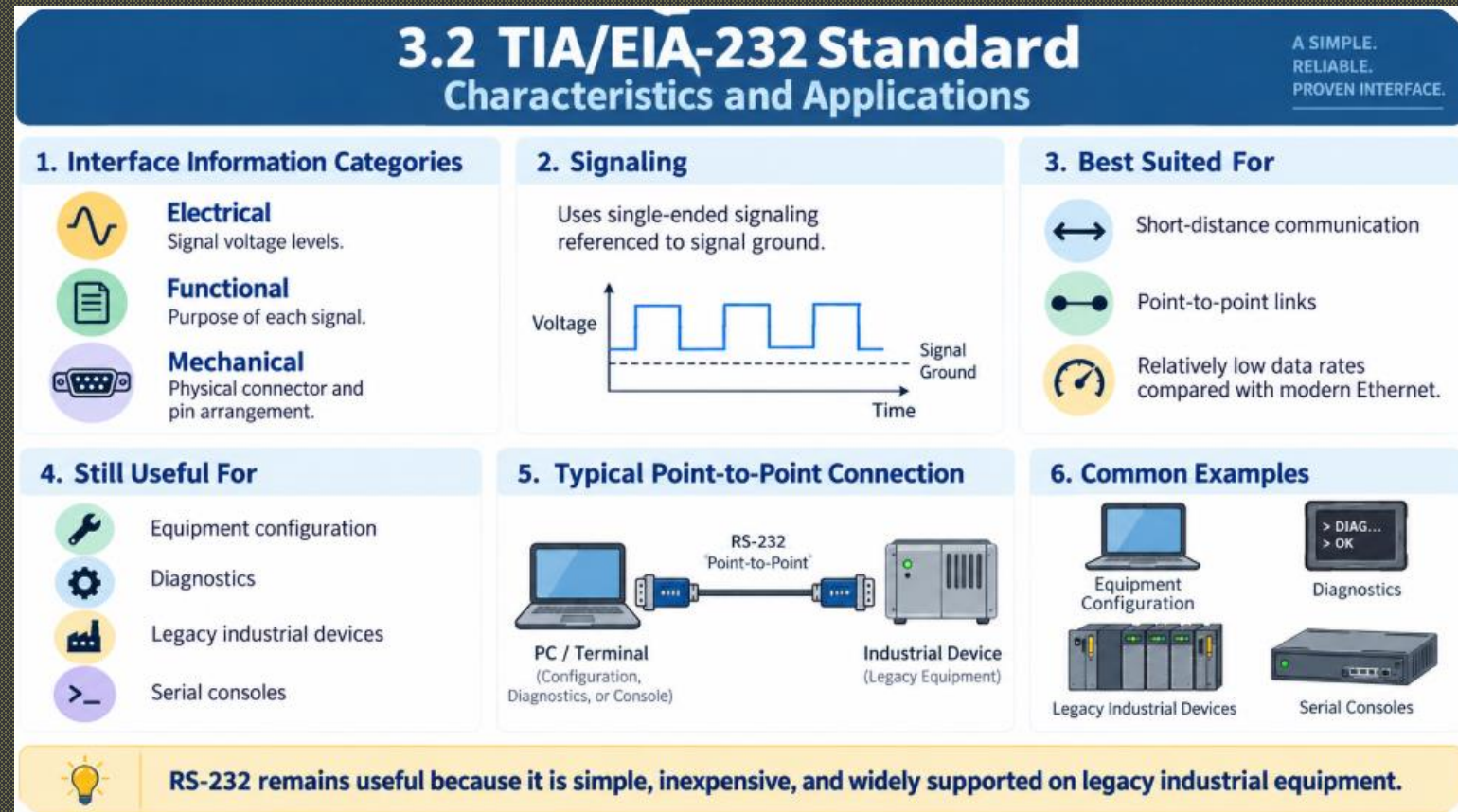
Null-modem cable DTE - DTE (computer-to-computer)

DTE device	DB9		DB9	DTE device
1 Carrier Detection (DCD)	CD	→	CD	1 Carrier Detection (DCD)
2 Receive Data (Rx)	RD	→	RD	2 Receive Data (Rx)
3 Transmit Data (Tx)	TD	←	TD	3 Transmit Data (Tx)
4 Data Terminal Ready	DTR	←	DTR	4 Data Terminal Ready
5 Signal Ground/Common (SG)	GND	→	GND	5 Signal Ground/Common (SG)
6 Data Set Ready	DSR	→	DSR	6 Data Set Ready
7 Request to Send	RTS	←	RTS	7 Request to Send
8 Clear to Send	CTS	→	CTS	8 Clear to Send
9 Ring Indicator	RI	→	RI	9 Ring Indicator
Shield	FGND	→	FGND	Shield

3.2 TIA/EIA-232 STANDARD

Characteristics and Applications

- RS-232 provides three important categories of interface information:
 - Electrical → signal voltage levels
 - Functional → purpose of each signal
 - Mechanical → physical connector and pin arrangement
- Uses single-ended signaling referenced to signal ground.
- Best suited to:
 - Short-distance communication
 - Point-to-point links
 - Relatively low data rates compared with modern Ethernet
- Still useful for:
 - Equipment configuration
 - Diagnostics
 - Legacy industrial devices
 - Serial consoles



3.2 TIA/EIA-232 STANDARD ELECTRICAL CHARACTERISTICS

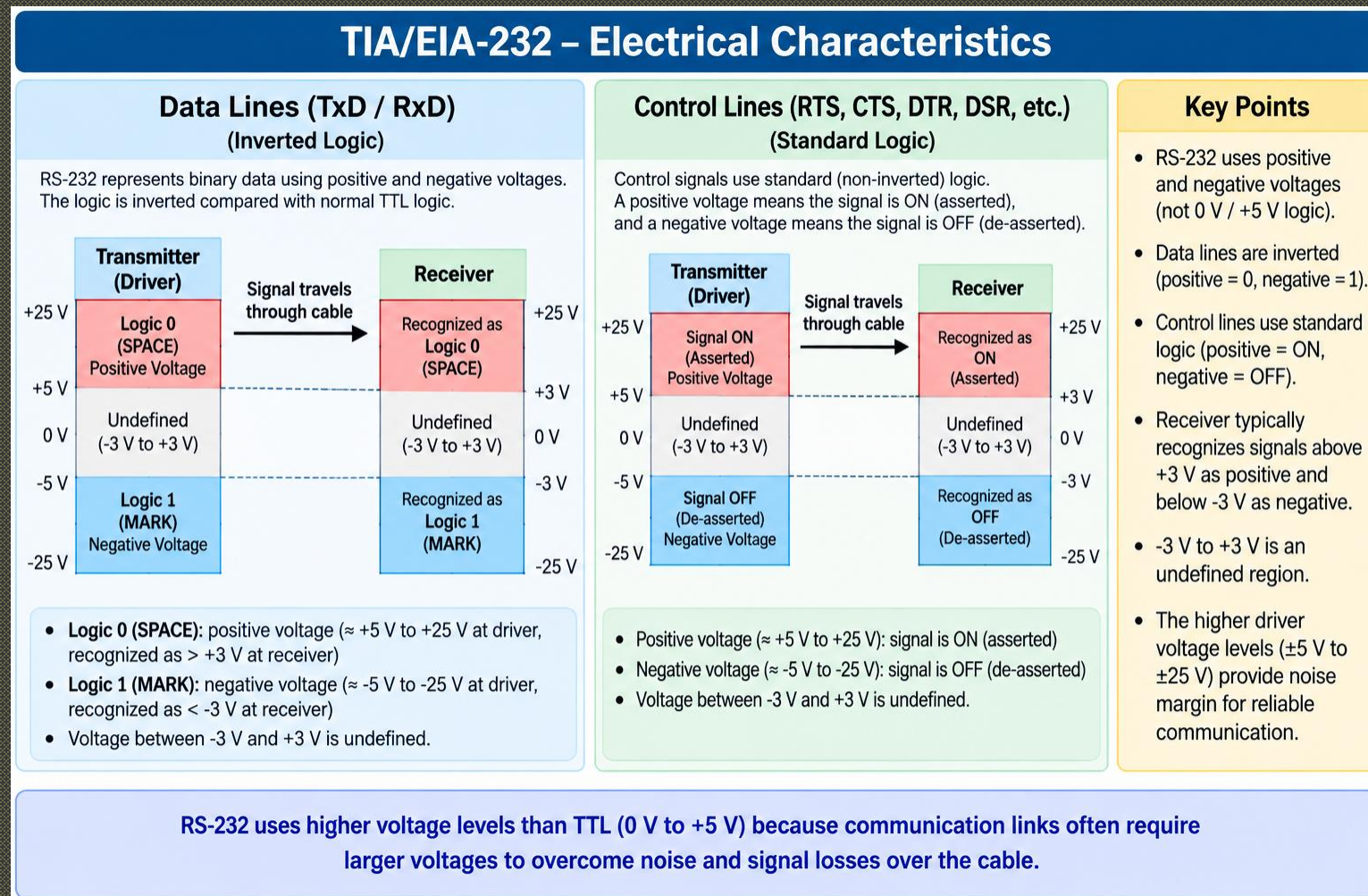
- RS-232 represents binary data using positive and negative voltages.
- The logic is inverted compared with normal TTL logic.

At the transmitter

- Logic 1 / MARK
 - Negative voltage
- Logic 0 / SPACE
 - Positive voltage

At the receiver

- Negative voltage is interpreted as Logic 1.
- Positive voltage is interpreted as Logic 0.
- A small voltage region around 0 V is considered undefined.



3.2 TIA/EIA-232 STANDARD

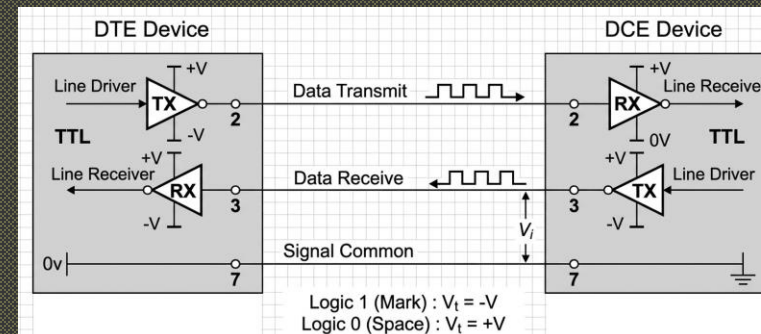
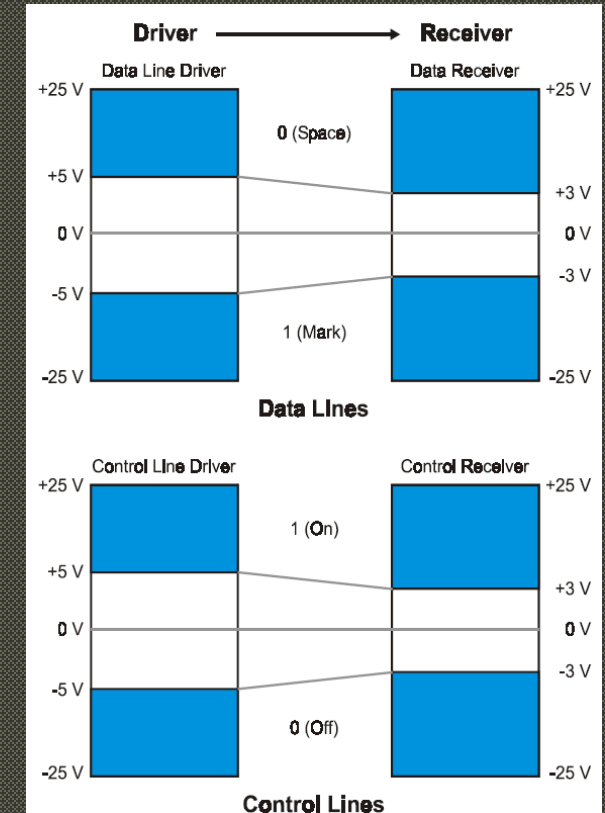
ELECTRICAL CHARACTERISTICS

Hardware Interface & Conversion

- Logic Differences: RS-232 positive and negative voltages differ from a microprocessor's standard 0V to +5V logic.
- Required Components: A line driver and a line receiver are required at the sending and receiving end of every wire.
- Voltage Conversion: These components convert RS-232 signals back into 0V and +5V levels that internal electronics can understand.

Flow Control Protocols

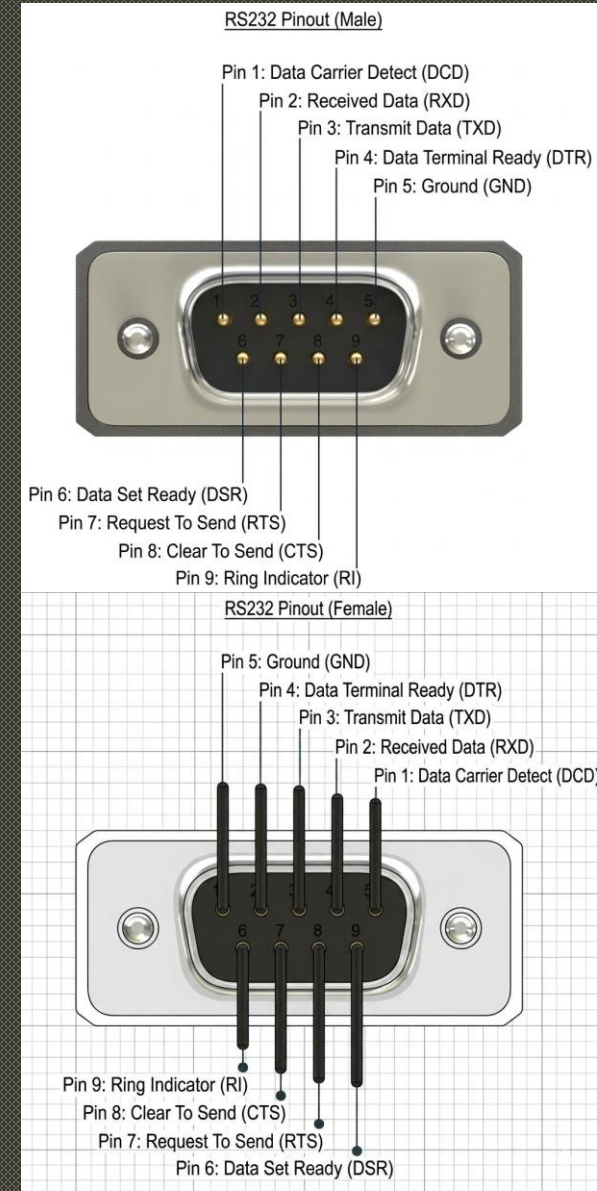
- Hardware Lines: Physical control lines are often left unused.
- Preferred Protocol: Software-based protocols, such as Xon/Xoff, are preferred for data management.
- Control Characters: The protocol uses specific ASCII characters to manage data flow:
 - Xon: Represented by ASCII character 17.
 - Xoff: Represented by ASCII character 19.



3.2 TIA/EIA-232 STANDARD MECHANICAL CHARACTERISTICS

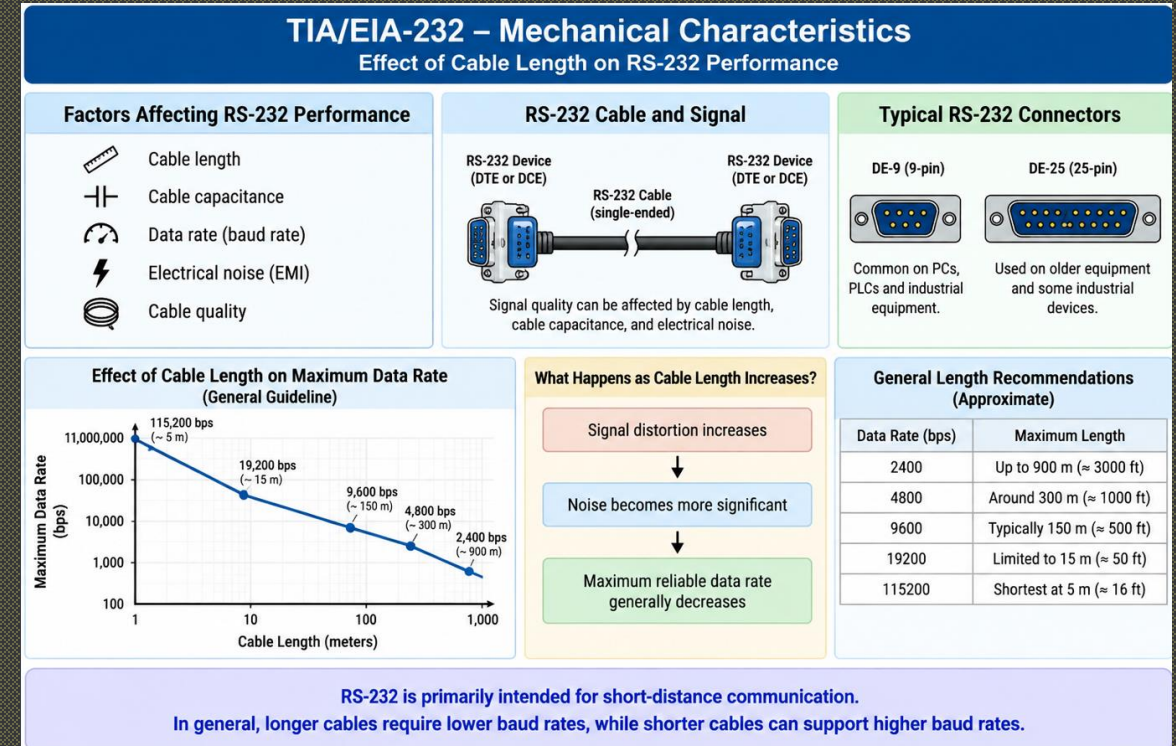
- RS-232 equipment may use different connector styles.
- A very common PC-style connector is the 9-pin D-subminiature connector, usually called DE-9.
- Older equipment may use a 25-pin connector.
- Common DE-9 signals include:
 - Pin 2 → RxD
 - Pin 3 → TxD
 - Pin 5 → Signal Ground
 - Pin 7 → RTS
 - Pin 8 → CTS
- Connector gender and pin assignments depend on whether the equipment is acting as DTE or DCE.

Do not assume two connectors with the same physical appearance have identical wiring - always verify the manufacturer's pinout



3.2 TIA/EIA-232 STANDARD MECHANICAL CHARACTERISTICS

- RS-232 performance depends on:
 - Cable length
 - Cable capacitance
 - Data rate
 - Electrical noise
 - Cable quality
- As cable length increases:
 - Signal distortion increases.
 - Noise becomes more significant.
 - Maximum reliable data rate generally decreases.
- Short cables can normally support higher data rates than long cables.
- RS-232 is therefore primarily intended for short-distance communication.
- Longer cable → generally lower reliable baud rate
- Shorter cable → higher baud rates are easier to achieve



General Length Recommendations:

- 2400 bps: Up to 900 meters (approximately 3000 feet)
- 4800 bps: Around 300 meters (about 1000 feet)
- 9600 bps: Typically 150 meters (roughly 500 feet)
- 19200 bps: Limited to 15 meters (around 50 feet)
- 115200 bps: Shortest at 5 meters (approximately 16 feet)

3.2 TIA/EIA-232 STANDARD

FUNCTIONAL CHARACTERISTICS

RS-232 signals can be grouped into several categories.

Data Signals

- TxD – Transmit Data
 - Carries outgoing serial data.
- RxD – Receive Data
 - Carries incoming serial data.

Control / Handshaking Signals

- RTS – Request to Send
- CTS – Clear to Send
- DTR – Data Terminal Ready
- DSR – Data Set Ready
- DCD – Data Carrier Detect
- RI – Ring Indicator

TIA/EIA-232 – Functional Characteristics

RS-232 Signals and Their Functions

RS-232 Signals can be grouped into several categories: data signals and control (handshaking/status) signals.

Data Signals

Carry serial data between devices.

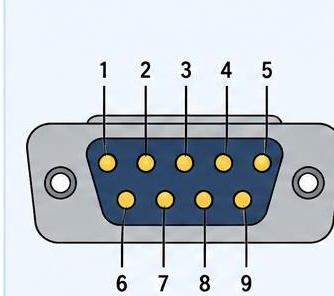
Signal	Direction (DTE)	Function	Typical Use
TxD (Transmit Data)	Output →	Carries outgoing serial data from the DTE to the DCE.	Sends data
RxD (Receive Data)	Input ←	Carries incoming serial data from the DCE to the DTE.	Receives data

Control / Handshaking Signals

Used to control the flow of data and provide status information.

Signal	Direction (DTE)	Function	Typical Use
RTS (Request to Send)	Output →	Indicates the DTE is ready to send data.	Flow control
CTS (Clear to Send)	Input ←	Indicates the DCE is ready to receive data.	Flow control
DTR (Data Terminal Ready)	Output →	Indicates the DTE is on and ready.	Status
DSR (Data Set Ready)	Input ←	Indicates the DCE is on and ready.	Status
DCD (Data Carrier Detect)	Input ←	Indicates a carrier signal is present (e.g., modem).	Status
RI (Ring Indicator)	Input ←	Indicates an incoming call (e.g., modem).	Status

Typical RS-232 Connector (DE-9) – DTE (PC/PLC)



View of female connector
(on device, DTE)

Pin	Signal (Typical DTE)
1	DCD (Data Carrier Detect)
2	RxD (Receive Data)
3	TxD (Transmit Data)
4	DTR (Data Terminal Ready)
5	Signal Ground (GND)
6	DSR (Data Set Ready)
7	RTS (Request to Send)
8	CTS (Clear to Send)
9	RI (Ring Indicator)

Key Points

- RS-232 has two data signals: TxD and RxD.
- Control signals (RTS, CTS, DTR, DSR, DCD, RI) are used for handshaking and status.
- Signal directions are defined from the DTE perspective.
- Not all devices use every control signal.
- Typical connector for PCs/PLCs is a 9-pin (DE-9), but some equipment uses a 25-pin connector.

Data signals carry information. Control signals coordinate communication.
Together, they allow RS-232 devices to exchange data reliably.

3.2 TIA/EIA-232 STANDARD

FUNCTIONAL CHARACTERISTICS

Hardware Handshaking

- Used to control the flow of data between devices.
- RTS/CTS is a common method.
- A device can indicate that it is ready or temporarily unable to accept additional data.

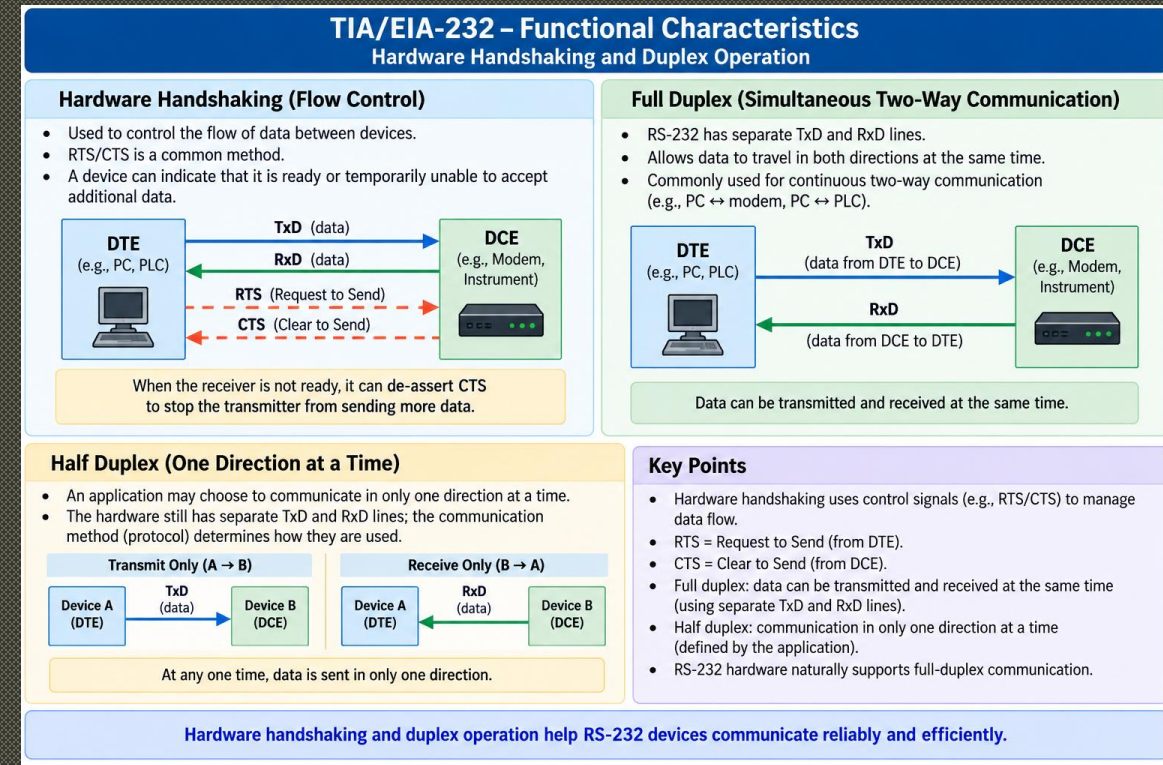
Full Duplex

- RS-232 has separate:
 - TxD line
 - RxD line
- This allows data to travel in both directions at the same time.

Half Duplex

- An application may choose to communicate in only one direction at a time.
- The hardware still has separate TxD and RxD lines; the communication method determines how they are used.

RS-232 hardware naturally supports full-duplex communication.

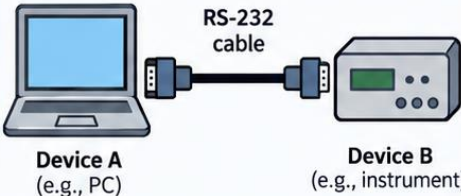


3.2 TIA/EIA-232 STANDARD

LIMITATIONS

RS-232 – Limitations and Practical Performance

1. Point-to-Point



- Normally connects only two devices
- Not designed as a multidrop industrial network

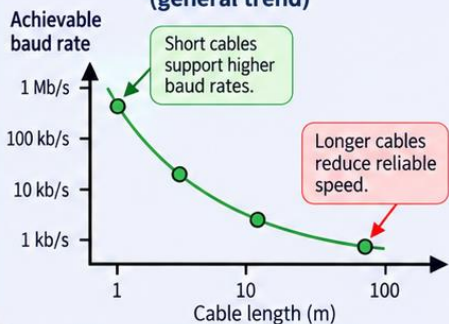
2. Key Limitations

- Limited distance**
Single-ended signaling becomes more sensitive to noise over long cables.
- Noise sensitivity**
Signal voltage is referenced to ground; less suitable for noisy environments than RS-422 or RS-485.
- Ground-potential differences**
Different ground voltages between devices can affect reliable communication.
- Lower performance**
Designed for much lower communication rates than modern network technologies.
- Legacy electrical levels**
Requires interface circuitry when connecting to modern low-voltage electronics.

3. Historical specification vs. practical systems

	Classic RS-232	Modern practical systems
Maximum distance (recommended)	15 m / 50 ft	–
Typical maximum baud rate	20 kb/s	115.2 kb/s or more may be possible when cable runs are short and conditions are suitable.

Cable length vs. achievable baud rate (general trend)



Cable length (m)	Achievable baud rate
1	~1 Mb/s
10	~10 kb/s
100	~1 kb/s



KEY TAKEAWAY

RS-232 is simple and useful, but best suited to short-distance, point-to-point communication.

3.3 RS-423 STANDARD

The Single-Ended Enhancement to RS-232

Key Features

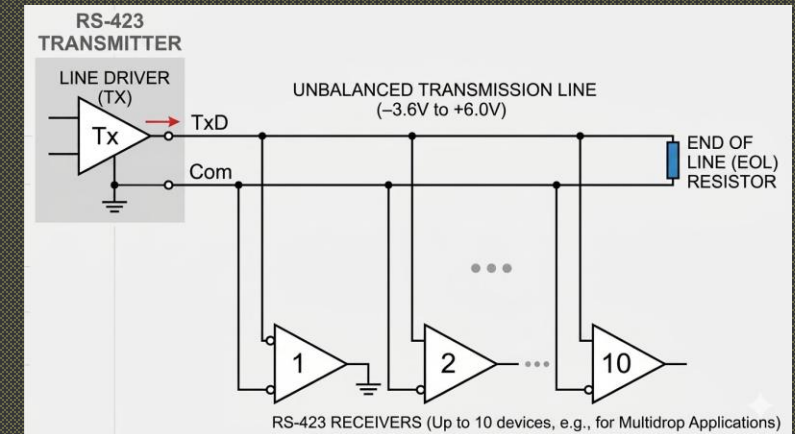
- Purpose: Acts as a higher-speed, longer-range successor to RS-232.
- Distance: Extends up to 1200 meters (4000 ft).
- Speed: Supports data rates up to 100 kbps.
- Topology: Strictly point-to-point (one driver, one receiver).
 - While it can support up to 10 receivers in a specialized "multi-drop" configuration.

Electrical & Innovation

- Signaling: Uses single-ended signaling referenced to a common ground.
- Voltages: Logic 1 (-3.6V to -6V) | Logic 0 (+3.6V to +6V).
- Innovation: Shrinks the voltage swing to ~12V (vs. RS-232's ~50V), slashing cable capacitance to boost speed and range.

Limitations

- Susceptibility: High risk from electrical noise (EMI) because the wire acts like an antenna.
- Ground Loops: Vulnerable to voltage differences between device grounds, which corrupts data.



Today it is mainly useful for:

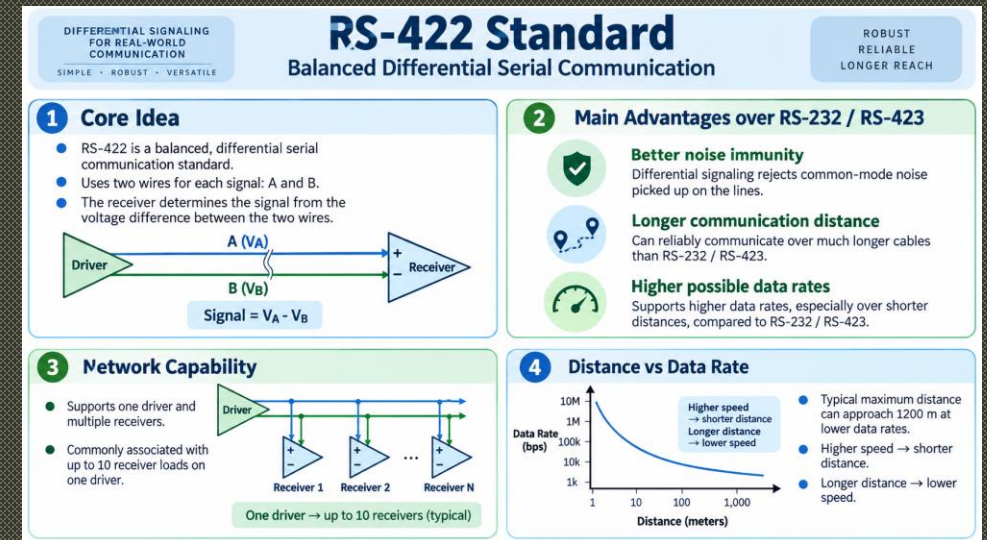
- Understanding the evolution of serial communication standards
- Maintaining some legacy equipment
- Comparing unbalanced and balanced signaling methods

3.4 RS-422 STANDARD

High-Speed, Noise-Immune Serial Communication

- RS-422 is a balanced, differential serial communication standard.
- Uses two wires for each signal:
 - A
 - B
- The receiver determines the signal from the voltage difference between the two wires.
- Main advantages compared with RS-232 and RS-423:
 - Better noise immunity
 - Longer communication distance
 - Higher possible data rates
- Supports:
 - One driver
 - Multiple receivers
- Commonly associated with up to 10 receiver loads on one driver.
- Typical maximum distance can approach 1200 m at lower data rates.
- Data rate and distance are related:
 - Higher speed → shorter distance
 - Longer distance → lower speed

- RS-422 is suitable for:
 - Industrial equipment
 - Machine-to-machine communication
 - Long cable runs
 - Electrically noisy environments



3.4 RS-422 STANDARD

High-Speed, Noise-Immune Serial Communication

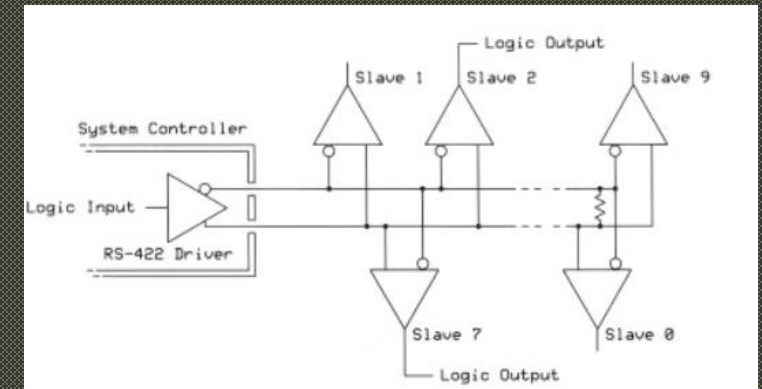
- One polarity of differential voltage represents one logic state.
- Reversing the polarity represents the opposite logic state.
- The key concept for students is:
 - $V_A > V_B \rightarrow$ one state
 - $V_A < V_B \rightarrow$ opposite state

Twisted-Pair Cable

- A and B are normally carried as a twisted pair.
- Twisting helps both wires experience similar external electrical noise.
- This improves common-mode noise cancellation.

Termination

- A termination resistor is placed at the receiving end of a sufficiently long/high-speed transmission line.
- The resistor should approximately match the cable's characteristic impedance.
- A value around $120\ \Omega$ is common for many twisted-pair cables.
- Termination helps prevent:
 - Signal reflections
 - Waveform distortion
 - Communication errors



<https://www.sealevel.com/basics-of-rs-422-and-rs-485-communications?srltid=AfmBOopiNn12d72KM4fX8r9DccrTL-JMrBu3p5TZsml8fTwRhevjjiGk>

3.4 RS-422 STANDARD

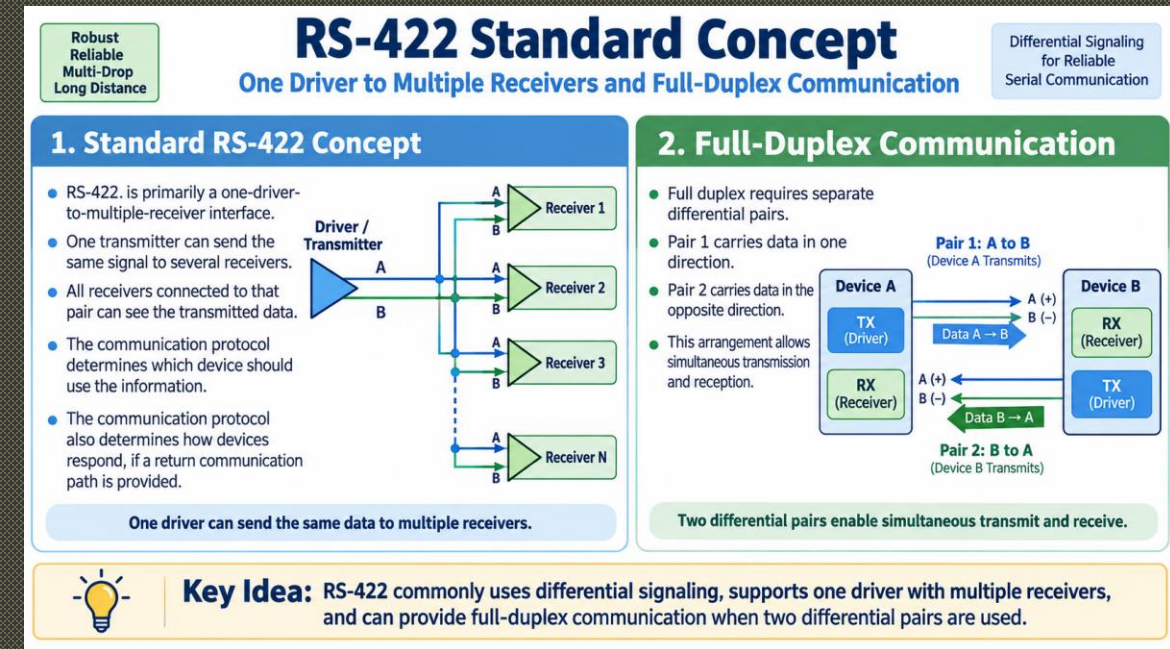
High-Speed, Noise-Immune Serial Communication

Standard RS-422 Concept

- RS-422 is primarily a one-driver-to-multiple-receiver interface.
- One transmitter can send the same signal to several receivers.
- All receivers connected to that pair can see the transmitted data.
- The communication protocol determines:
 - Which device should use the information
 - How devices respond, if a return communication path is provided

Full-Duplex Communication

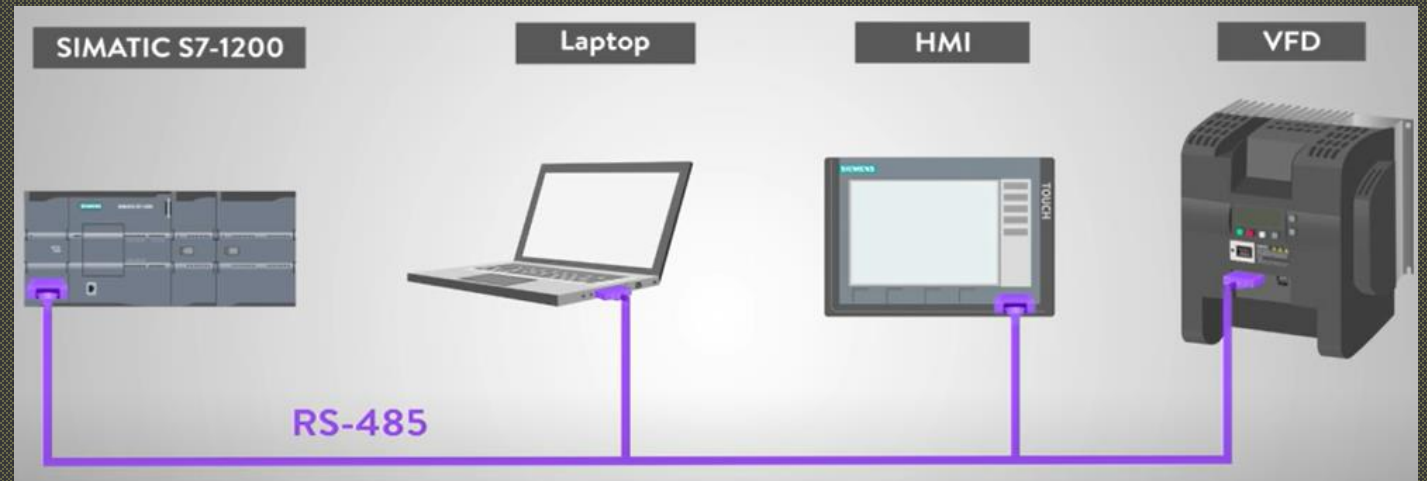
- Full duplex requires separate differential pairs:
 - Pair 1 → data in one direction
 - Pair 2 → data in the opposite direction
- This allows simultaneous transmission and reception when the connection arrangement supports it.



- Single-ended: one wire carries the signal and the other is a ground/reference. The receiver measures: $V_{\text{signal}} - V_{\text{ground}}$
- Differential: both wires are signal conductors, usually called A and B, and the receiver measures: $V_A - V_B$

3.5 RS-485 STANDARD - INTRODUCTION

- RS-485 is a balanced, differential serial communication standard.
- Designed for reliable communication between multiple devices on the same network.
- Particularly useful in:
 - Industrial automation
 - Instrumentation
 - PLC networks
 - Drives
 - Sensors and remote devices
- Uses a shared bus topology.
- Devices can be added or removed without rewiring every other device.
- Main advantages:
 - Good noise immunity
 - Long communication distance
 - Multiple-device capability
 - Relatively simple and inexpensive wiring

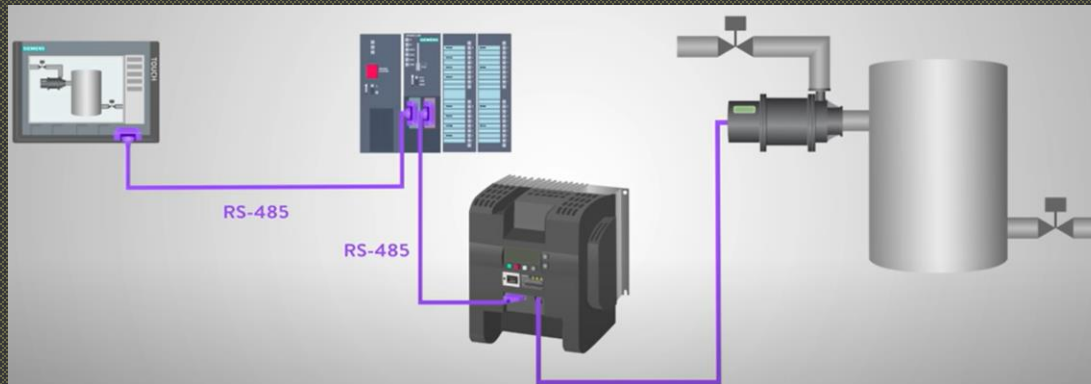


<https://www.youtube.com/watch?v=3wgKcUDiHuM>

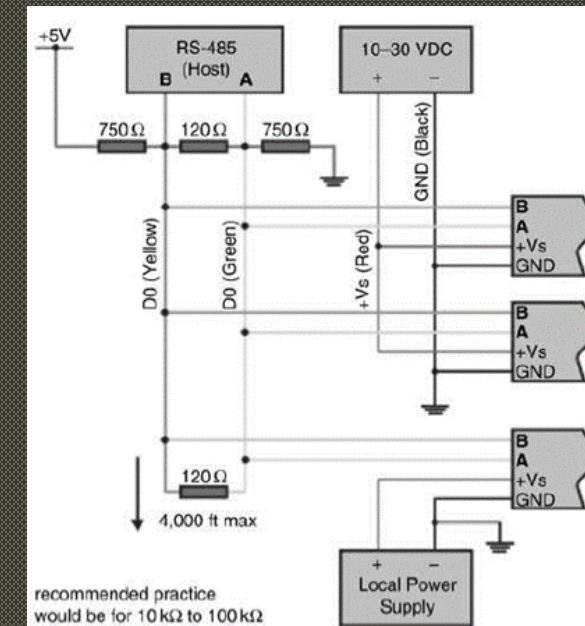
3.5 RS-485 - PERFORMANCE AND NETWORK CAPACITY

- Uses balanced differential signaling.
- Supports long-distance communication:
 - Up to approximately 1200 m under suitable conditions.
- Can support high data rates over shorter distances.
- Distance and data rate trade off against each other:
 - Longer distance → lower reliable data rate
 - Higher data rate → shorter cable length
- Traditional RS-485 systems are commonly described as supporting:
 - Up to 32 unit-load transmitters
 - Up to 32 unit-load receivers
- Modern reduced-unit-load transceivers can support more devices.

The maximum speed and maximum length cannot be achieved at the same time. For 24 AWG twisted pair cable the maximum speed, at 1200 m, is ~100 kbps. The maximum cable length, at 10 Mbps, is less than ~10 m.

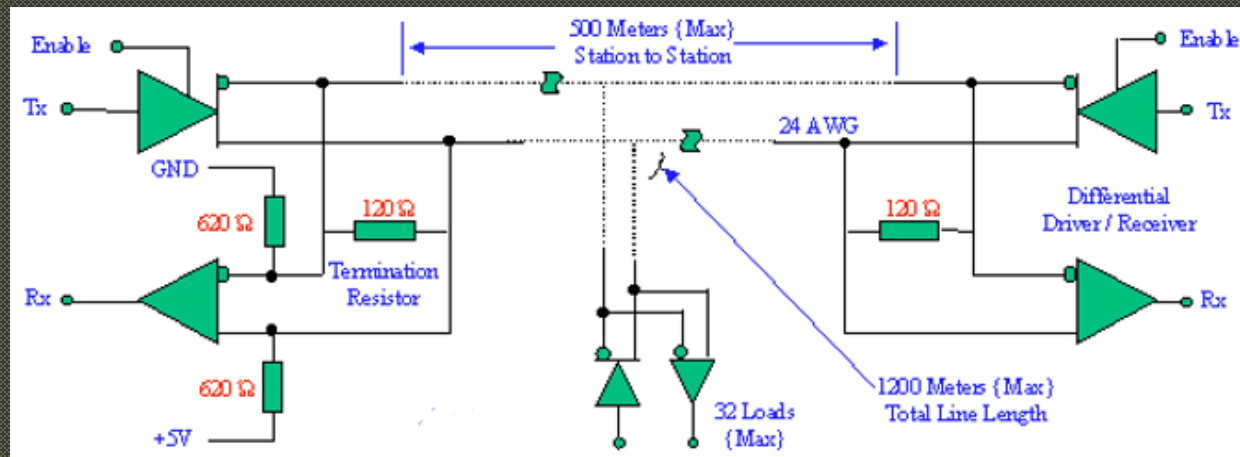


<https://www.youtube.com/watch?v=3wgKcUDIHuM>



3.5 RS-485 - TRI-STATE DRIVERS

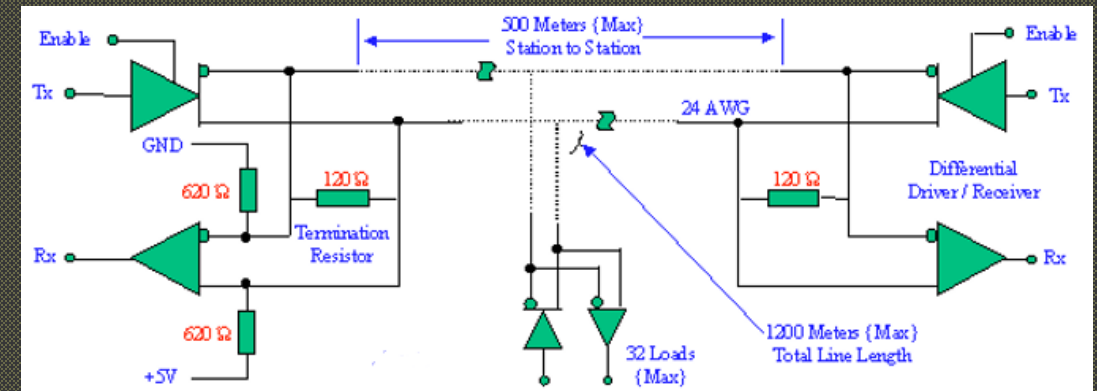
- An RS-485 transmitter has three important output conditions:
 - Driving one differential state
 - Driving the opposite differential state
 - High impedance (Hi-Z)
- In Hi-Z:
 - The transmitter electrically disconnects itself from the bus.
 - It does not actively drive A or B.
- Tri-state operation allows multiple transmitters to share the same bus.
- Only the device currently permitted to transmit should actively drive the bus.



- Transmit $\rightarrow$ driver enabled
- Not transmitting $\rightarrow$ driver Hi-Z

3.5 RS-485 - MULTIPLE DEVICES SHARING THE BUS

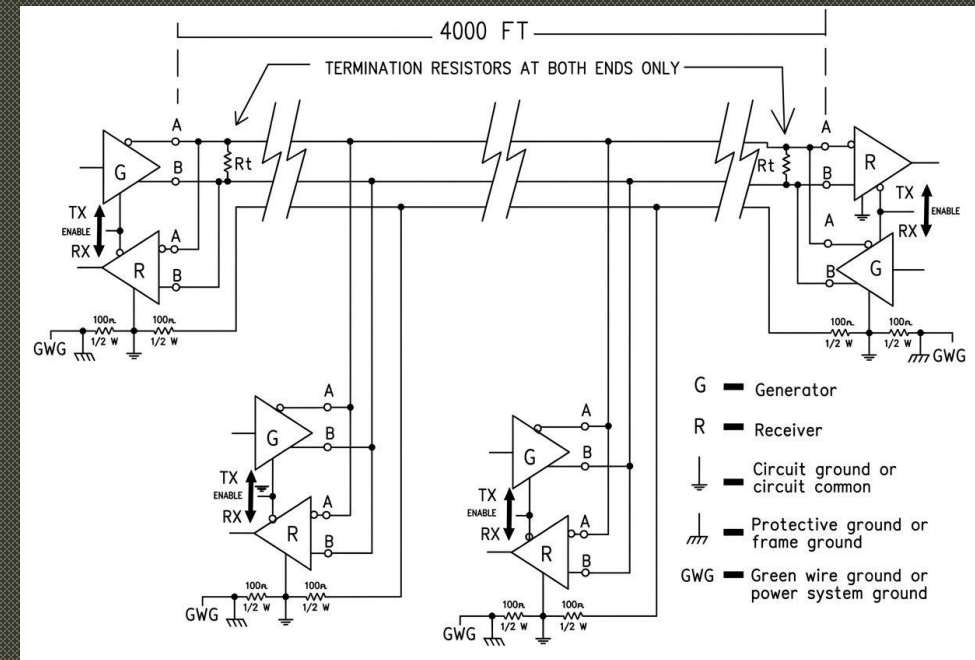
- Multiple devices can connect to the same RS-485 bus.
- Only one transmitter should normally drive a 2-wire bus at a time.
- Devices take turns transmitting.
- A transmitter's driver-enable control switches it between:
 - Active
 - High impedance
- Device addressing and communication control are handled by the protocol, not RS-485 itself.
- Many industrial systems use:
 - Controller/device
 - Master/slave
 - Client/server communication arrangements.
- The protocol determines which device is allowed to respond.



<https://www.ti.com/video/series/precision-labs/ti-precision-labs-rs-485.html>

3.5 RS-485 - TWO WIRE CONFIGURATION

- Uses one differential pair shared by all devices.
- Each device connects to the same pair its:
 - Transmitter
 - Receiver
- Normally operates half-duplex:
 - Communication can travel in both directions.
 - But only one direction is active at a time.
- Devices take turns transmitting.
- Requires transmitters to enter Hi-Z when they are not sending.
- Main advantages:
 - Minimum wiring
 - Low cost
 - Good noise immunity
 - Excellent for multidrop industrial networks

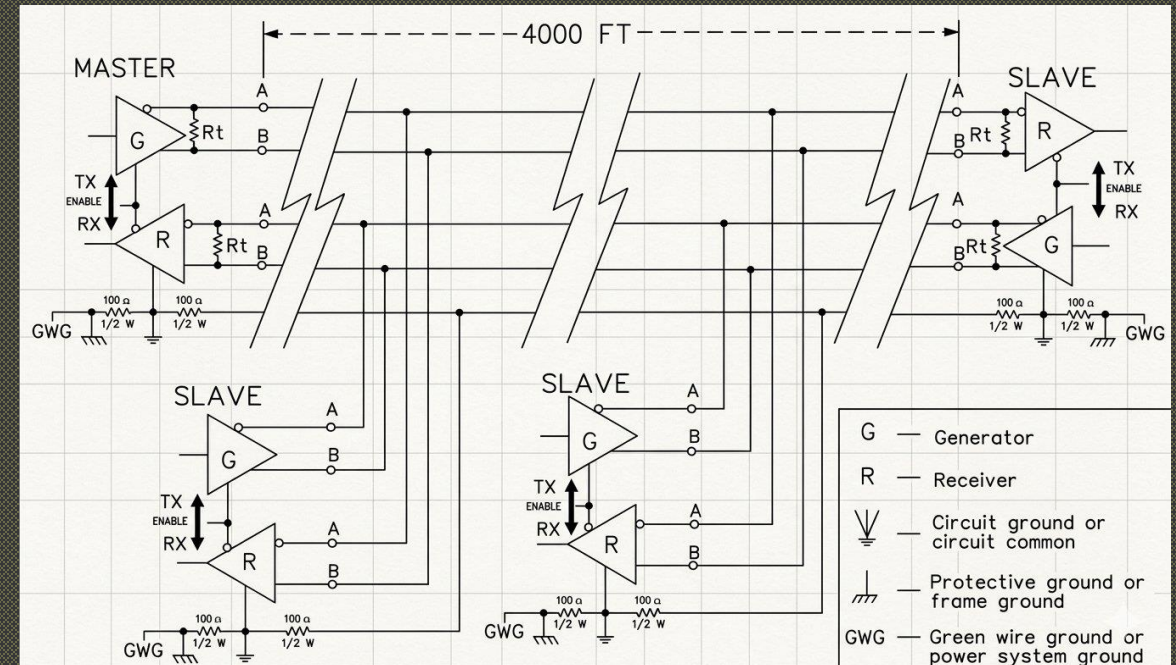


Typical Applications

- Modbus RTU
- Industrial instruments
- Drives
- Remote I/O

3.5 RS-485 - FOUR WIRE CONFIGURATION

- Uses two differential pairs.
- One pair carries data in one direction.
- Second pair carries data in the opposite direction.
- Can support full-duplex communication.
- A common arrangement is:
 - Controller Tx → device receivers
 - Device transmitters → controller Rx
- Allows transmission and reception simultaneously.
- Requires more wiring than 2-wire RS-485.
- Whether slaves/devices can communicate directly with each other is determined by the protocol and network design, not by RS-485 itself.



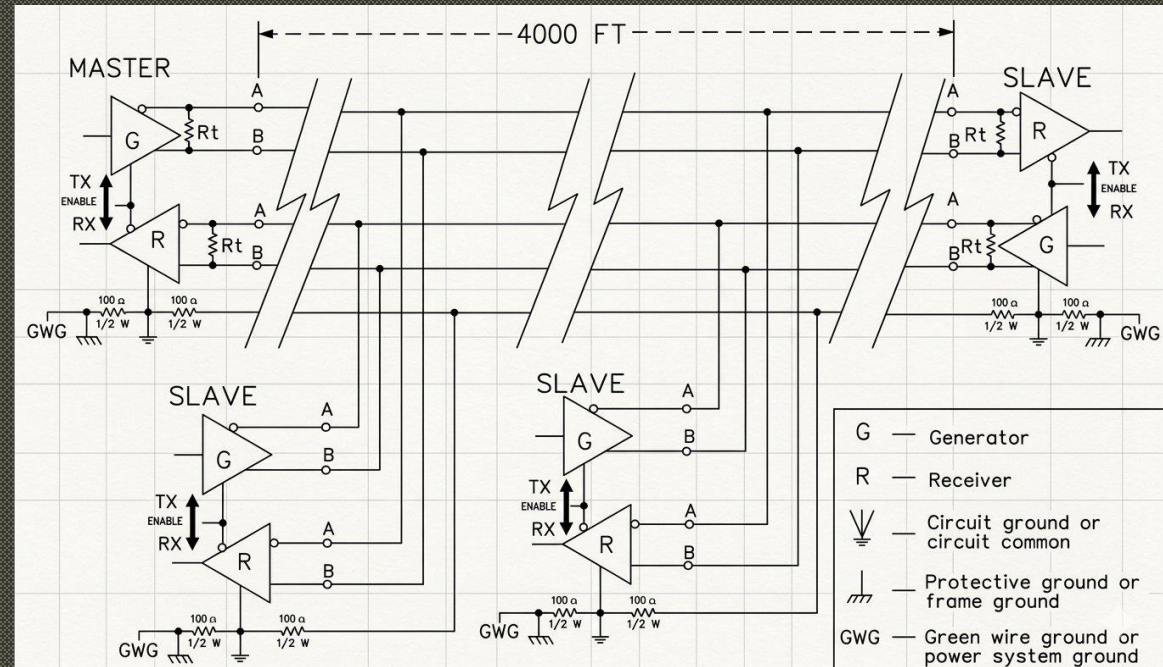
- 2-wire → usually half-duplex
- 4-wire → can provide full-duplex

3.5 RS-485 - SIGNAL REFERENCE AND GROUNDING

- RS-485 receivers determine logic from the voltage difference between A and B.
- However, the receiver also has a permitted common-mode voltage range.
- Large ground-potential differences between devices can exceed that range.
- A signal-reference/common conductor may therefore be used where required.
- The signal reference is not the same thing as the differential signal pair.
- The cable shield should not normally be used as the signal conductor.

- Good grounding practices help reduce:
 - Common-mode problems
 - Electrical noise
 - Communication errors
- For large ground-potential differences:
 - Galvanic isolation may be preferable.

Galvanic isolation in RS-485 safely blocks direct electrical connections between devices using optical or magnetic barriers, which stops ground voltage differences from destroying equipment or disrupting data.



3.5 RS-485 - WHY TERMINATION IS NEEDED

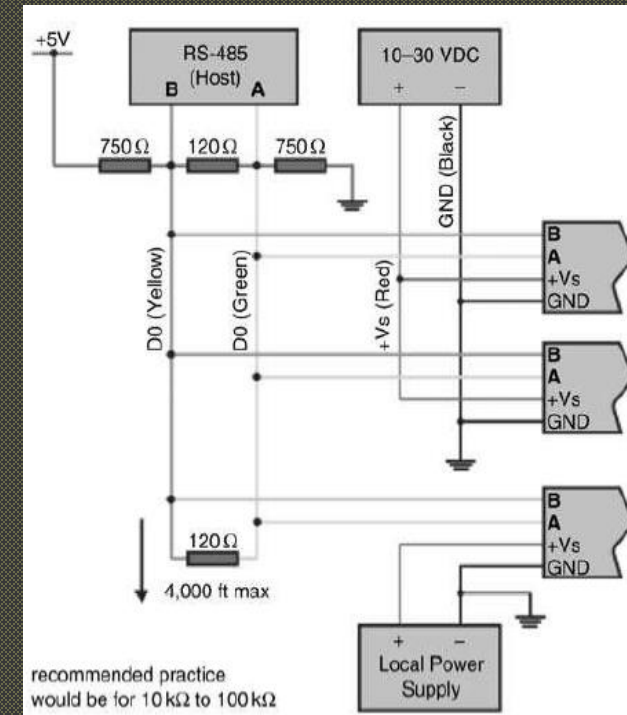
- Long cables behave as transmission lines.
- Fast signal transitions can reflect from the end of an improperly terminated cable.
- Reflections may cause:
 - Signal distortion
 - False transitions
 - Communication errors
- A termination resistor absorbs signal energy and reduces reflections.
- Termination becomes increasingly important with:
 - Long cables
 - High data rates
 - Fast transceiver rise/fall times

Without termination:

Signal → end of cable → reflection → distorted signal

With proper termination:

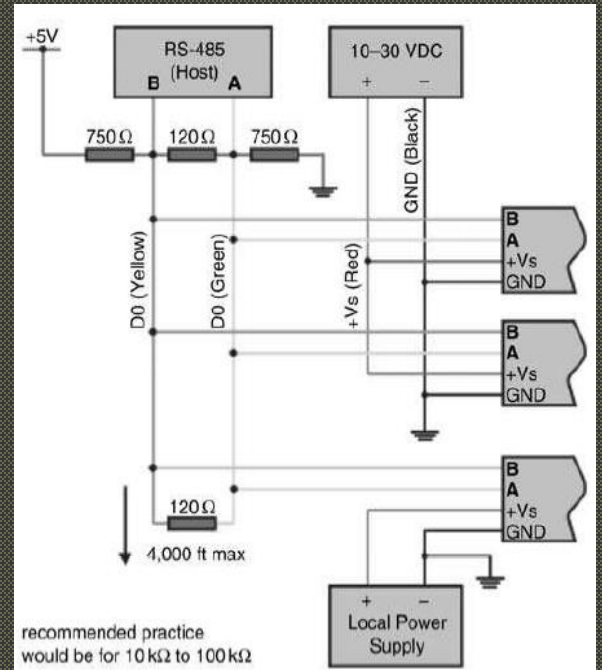
Signal → termination → energy absorbed



Terminator Resistors

- "characteristic impedance" (Z_0)
- $Z_0 = [(R + j\omega L) / (G + j\omega C)]^{1/2}$
- $Z_0 = (L/C)^{1/2}$ for an ideal cable (no losses)
 - R – the series resistance per unit length (Ω/m);
 - L - the series inductance (H/m);
 - G - the shunt conductance (S/m);
 - C - the shunt capacitance (F/m)

- Termination value depends on the cable characteristic impedance, not simply because the interface is RS-485.



3.5 RS-485 – WHEN TERMINATION IS NEEDED

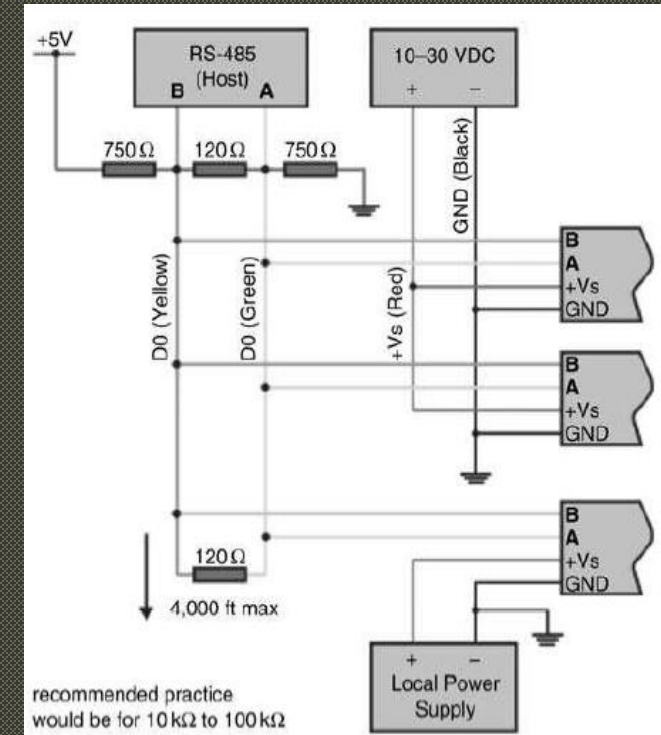
- Termination is most important when cable propagation delay becomes significant compared with signal transition time.
- More likely needed when:
 - Cable is long
 - Data rate is high
 - Driver edges are fast
- Short, low-speed links may sometimes operate reliably without termination.
- Manufacturer recommendations should be followed.

Example to prove when termination is needed.

- Cable: 1200 meters long.
- Signal Speed: $0.66 * c$ (c = speed of light). This slowdown is normal for cables.
- Calculate Round Trip Time:
 - Time one way = Distance / Speed = $1200 \text{ m} / (0.66 * 3 \times 10^8 \text{ m/s}) \approx 6.1 \mu\text{s}$
 - This is the time for the signal to go from one end to the other. A round trip (there and back) is $12.2 \mu\text{s}$.
 - Damping Out: They assume reflections take about 3 round trips to fade away to a negligible level.
 - Total settling time = $3 * 12.2 \mu\text{s} = 36.6 \mu\text{s}$.
- Bit Width: Let us use a common baud rate of 9600 bps.
- Time for one bit = $1 / 9600 \text{ Hz} \approx 104 \mu\text{s}$.

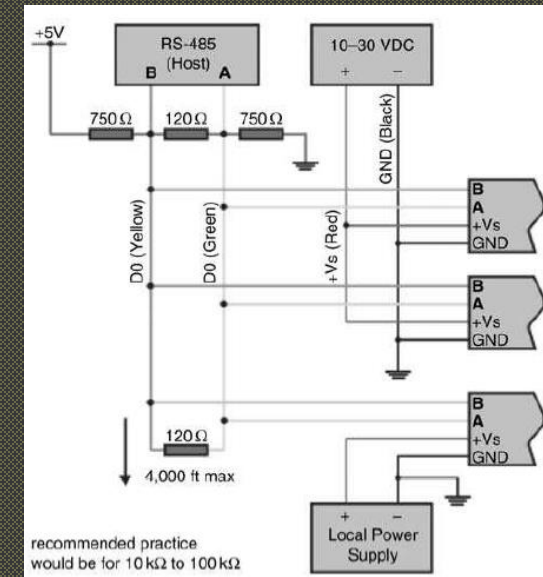
Rationale: The line needs $\sim 37 \mu\text{s}$ to settle from the echo of a signal change. But a single bit is $104 \mu\text{s}$ long. The echo will have completely faded away well before the receiver even tries to read the value of that bit. Therefore, termination is not required.

Conversely, if the bit time was shorter than the settling time (e.g., at a baud rate of 100,000 bps, where a bit is only $10 \mu\text{s}$ long), you would absolutely need terminating resistors to prevent data errors.



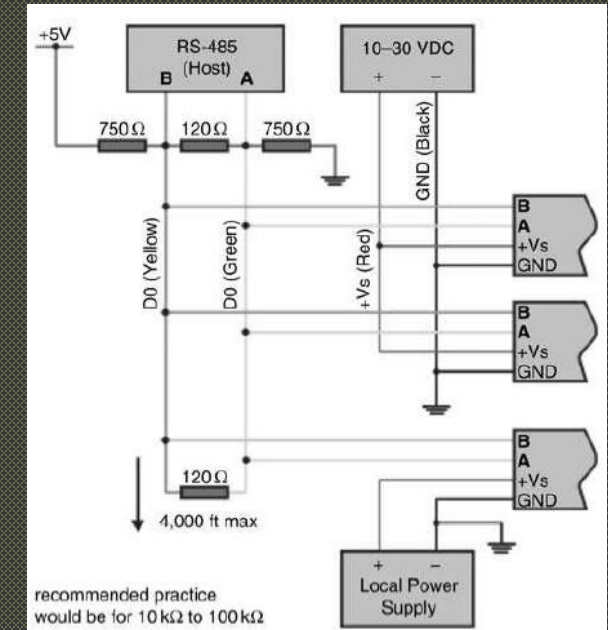
3.5 RS-485 - BIAS / FAIL-SAFE RESISTORS

- When no device is transmitting:
 - All drivers may be in Hi-Z.
 - The bus can otherwise be left electrically undefined.
- Electrical noise could cause the receiver to interpret false data.
- Bias resistors establish a known idle voltage.
- A typical bias network uses:
 - Pull-up resistor on one line
 - Pull-down resistor on the other
- The resulting differential voltage keeps receivers in a predictable idle state.
- Many modern RS-485 transceivers provide internal fail-safe receiver circuitry.
- External bias resistors may therefore not always be required.
- Check the transceiver manufacturer's recommendations.



3.5 RS-485 - SELECTING BIAS RESISTORS

- Bias resistor values must be chosen so that:
 - The idle differential voltage is large enough for a stable receiver state.
 - Excessive DC current is not drawn.
 - Termination resistors are taken into account.
 - The bias network and termination resistors form a combined electrical load.
-
- Bias resistors are applied to force the data lines to the idle condition. They are a pullup resistor on the data B line (+5V) and a pulldown (to ground) of the data A line, installed only at one node.
 - The value of the bias resistors depends on termination and number of nodes in the system.
 - The goal is to generate enough DC bias current to maintain a minimum of 200 mV between B and A data line.



- If each RS-485 node has a load impedance of 12 KΩ and there are 10 nodes in parallel, the total impedance is 1200 Ω.
- If the terminal resistors are 120 Ω, the total impedance is 60 Ω.
- The two impedances in parallel (1200 Ω and 60 Ω), make 57 Ω.
- To maintain a ± 200 mV between B and A line, a current of 3.5 mA is needed (for 57 Ω).
- To create this bias from a 5 V supply, a total of 1428 Ω resistor is needed.
- Subtracting 57 Ω that is already a part of the load, 1371 Ω is left.
- Placing halves for pulldown and pullup gives a bias resistor of 685 Ω.

3.6 COMPARISON OF STANDARDS: SIGNALING METHODS

- The main electrical difference is the signaling method.

RS-232

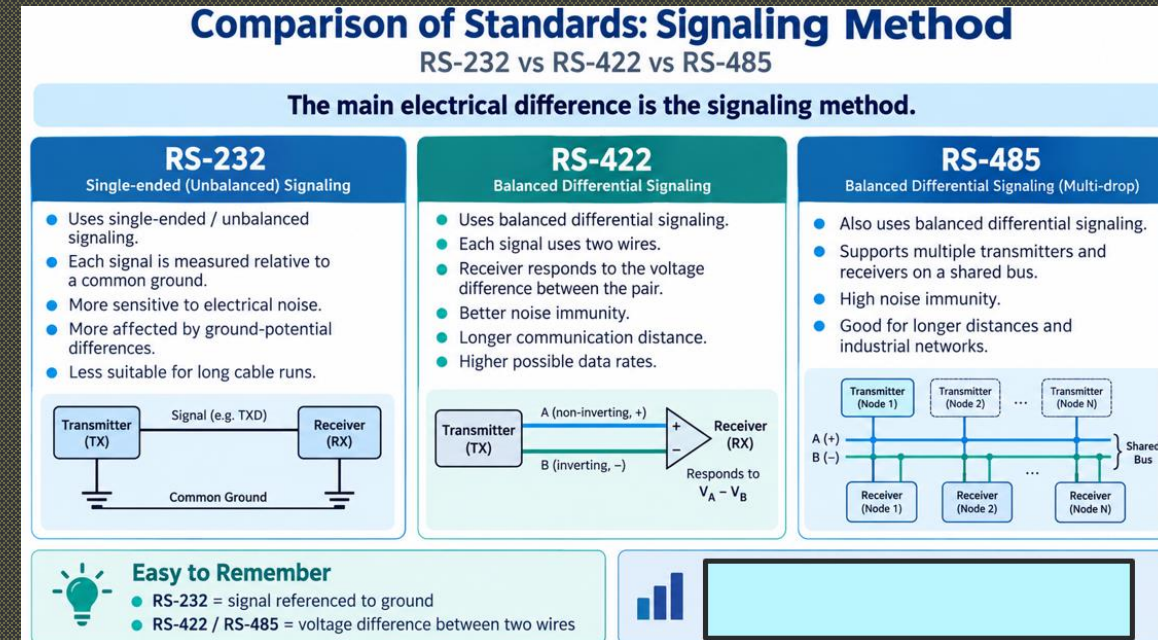
- Uses single-ended / unbalanced signaling.
- Each signal is measured relative to a common ground.
- More sensitive to:
 - Electrical noise
 - Ground-potential differences
 - Long cable runs

RS-422

- Uses balanced differential signaling.
- Each signal uses two wires.
- Receiver responds to the voltage difference between the pair.
- Provides:
 - Better noise immunity
 - Longer communication distance
 - Higher possible data rates

RS-485

- Also uses balanced differential signaling.
- Adds strong support for multiple transmitters and receivers sharing a bus.



- RS-232 = signal referenced to ground
- RS-422 / RS-485 = voltage difference between two wires

3.6 COMPARISON OF STANDARDS

Topology and Number of Devices

RS-232

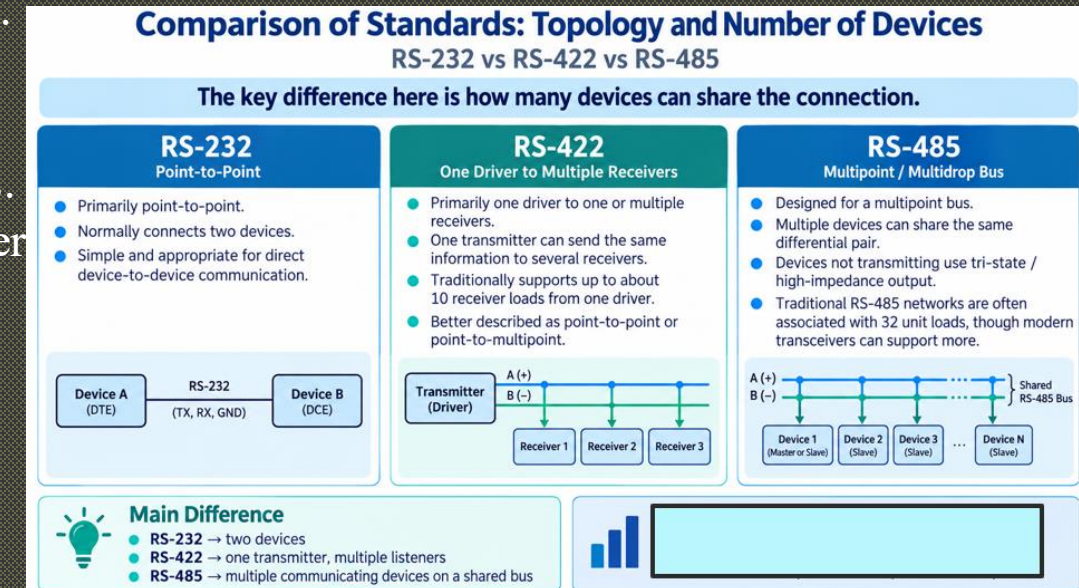
- Primarily point-to-point.
- Normally connects two devices.
- Simple and appropriate for direct device-to-device communication.

RS-422

- Primarily one driver to one or multiple receivers.
- One transmitter can send the same information to several receivers.
- Traditionally supports up to about 10 receiver loads from one driver
- Better described as:
 - Point-to-point, or
 - Point-to-multipoint

RS-485

- Designed for a multipoint / multidrop bus.
- Multiple devices can share the same differential pair.
- Drivers use tri-state / high-impedance operation so devices that are not transmitting do not interfere with the bus.
- Traditional RS-485 networks are often associated with 32 unit loads, although modern reduced-unit-load transceivers can support more devices.



- RS-232 → two devices
- RS-422 → one transmitter, multiple listeners
- RS-485 → multiple communicating devices on a shared bus

3.6 COMPARISON OF STANDARDS – DUPLEX CAPABILITY

RS-232

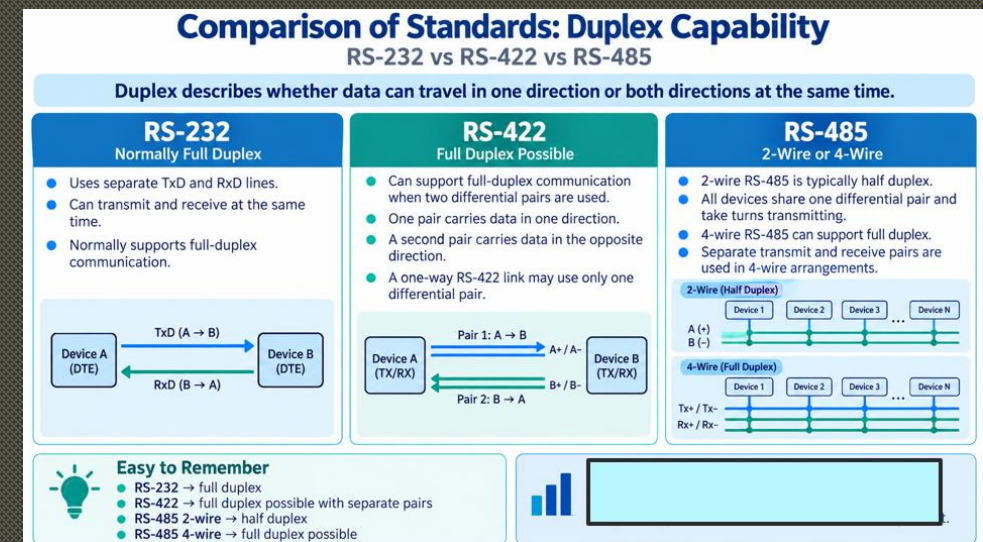
- Normally supports full-duplex communication.
- Uses separate:
 - Tx/D
 - Rx/D
- Data can be transmitted and received at the same time.

RS-422

- Can support full-duplex communication when two differential pairs are used:
 - One pair for transmit
 - One pair for receive
- A one-way RS-422 link may use only one differential pair.

RS-485

- 2-wire RS-485
 - Typically half-duplex.
 - All devices share one differential pair.
 - Devices take turns transmitting.
- 4-wire RS-485
 - Can support full-duplex operation.
 - Uses separate transmit and receive pairs.



- RS-232 → full duplex
- RS-422 → full duplex possible with separate pairs
- RS-485 2-wire → half duplex
- RS-485 4-wire → full duplex possible

3.6 COMPARISON OF STANDARDS - APPLICATIONS

RS-232

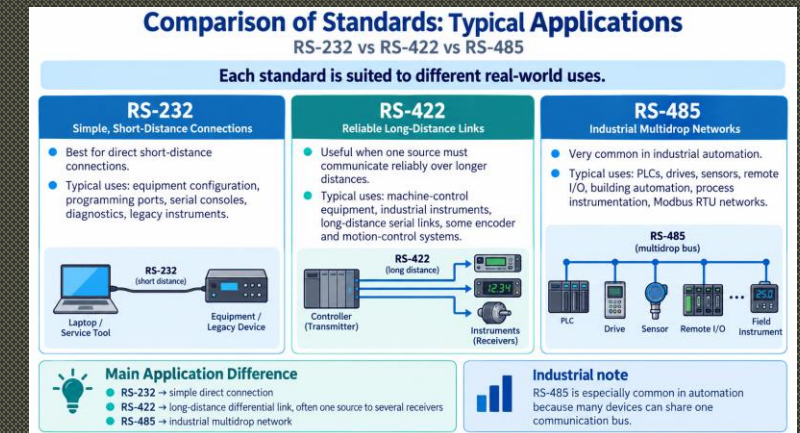
- Best suited to simple, short-distance connections.
- Typical applications:
 - Equipment configuration
 - Programming ports
 - Serial consoles
 - Diagnostics
 - Legacy instruments

RS-422

- Useful when one source must communicate reliably over longer distances.
- Typical applications:
 - Machine-control equipment
 - Industrial instruments
 - Long-distance serial links
 - Some encoder and motion-control systems

RS-485

- Very common in industrial automation.
- Well suited to multiple devices on one network.
- Typical applications:
 - PLCs
 - Drives
 - Sensors
 - Remote I/O
 - Building automation
 - Process instrumentation



- Simple Analogy
- RS-232 is a private conversation between two people.
- RS-422 is a lecture hall where one professor speaks to many students.
- RS-485 is a structured debate with many participants who must raise their hand and be called on before speaking, all using the same microphone.

3.6 COMPARISON OF STANDARDS

Final Comparison of Serial Standards RS-232 vs RS-422 vs RS-485

Feature	RS-232	RS-422	RS-485
Signaling	Single-ended	Differential	Differential
Typical topology	Point-to-point	One driver → multiple receivers	Multipoint bus
Duplex	Full duplex	Full duplex possible	2-wire half / 4-wire full
Noise immunity	Lower	High	High
Distance	Short	Long	Long
Multiple devices	No	Multiple receivers	Multiple transmitters/receivers
Typical industrial role	Configuration / diagnostics	Long-distance link	Industrial multidrop network



One additional point worth adding

These standards mainly define the **electrical interface**, not the application protocol.



Example

- RS-485 tells us how electrical signals are transmitted.
- Modbus RTU tells us how messages, addresses, and data are organized.

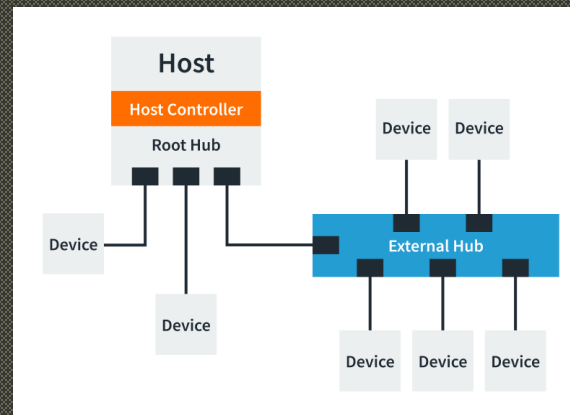
This distinction helps students avoid confusing RS-485 with Modbus.

3.7 THE UNIVERSAL SERIAL BUS (USB)

- Developed to provide a standardized connection for computers and peripheral devices.
- Designed to simplify:
 - Device connection
 - Automatic device detection/configuration
 - Data transfer
 - Power delivery
- USB uses a host-controlled architecture:
 - Host initiates and schedules communication.
 - Devices respond to the host.
- Supports Plug-and-Play.
- Common applications:
 - Keyboards and mice
 - Printers
 - Storage devices
 - Cameras
 - Measurement and DAQ devices

- Major data-rate generations:
 - Low Speed → 1.5 Mb/s
 - Full Speed → 12 Mb/s
 - High Speed → 480 Mb/s
 - SuperSpeed → 5 Gb/s and higher
 - USB4 → 20, 40, and 80 Gb/s classes

USB4 Version 2 supports up to 80 Gb/s symmetric operation, with optional asymmetric operation up to 120 Gb/s in one direction.



- Universal Serial Bus (USB)
- Standardized interface for PCs and peripherals
- Host-controlled, packet-based bus
- Plug-and-Play (automatic detection and configuration)
- Provides both data and power
- Common devices: keyboards, mice, printers, storage, cameras, DAQ

A small diagram showing a laptop connected via a USB cable to a group of peripheral devices including a printer, a hard drive, and a camera.

Main Idea:
USB simplifies device connection and communication for a wide range of peripherals.

3.7 USB HOST CONTROLLER AND ROOT HUB

Host Controller

- Controls communication on the USB bus.
- Typically integrated into the computer's chipset or processor platform.
- Responsible for:
 - Scheduling USB transfers
 - Communicating with connected devices
 - Managing available bus bandwidth
 - Coordinating data transfers

Root Hub

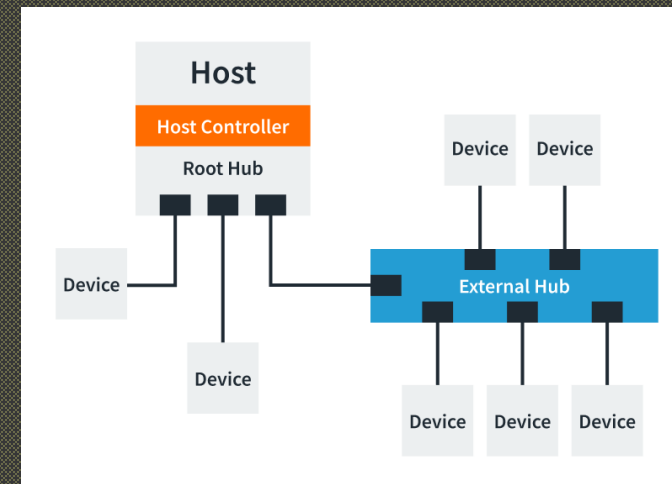
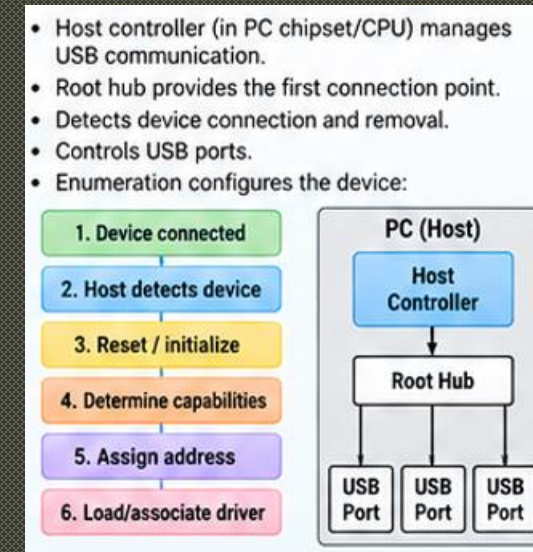
- Provides the first USB connection point from the host controller.
- Detects:
 - Device connection
 - Device removal
- Controls individual USB ports.
- Helps begin enumeration when a device is connected.

Enumeration

When a USB device is connected, the host:

- Detects the device.
- Resets/initializes the connection.
- Determines device capabilities.
- Assigns an address.

Loads or associates the appropriate driver.



3.7 USB PHYSICAL LAYER

- The USB Physical Layer defines how electrical signals are transmitted through the cable and connectors.
- USB uses differential signaling for data communication.

USB 2.0 Signaling

- Uses one differential pair:
 - D+
 - D-
- The receiver determines the data from the voltage difference between D+ and D-.
- Differential signaling provides:
 - Better resistance to electrical noise
 - Reduced electromagnetic interference
 - More reliable data transmission

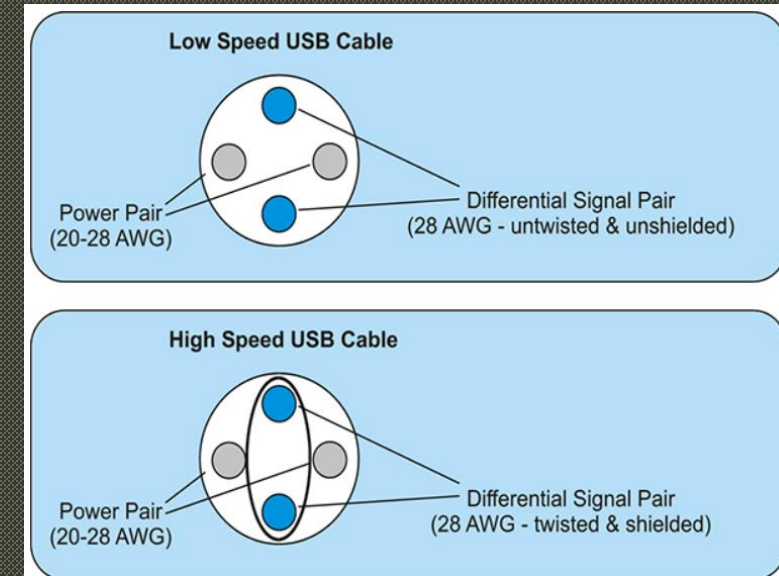
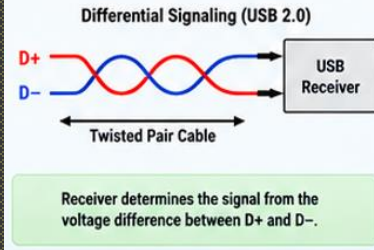
Higher-Speed USB

- SuperSpeed USB and USB4 use additional high-speed differential pairs.
- Separate transmit and receive paths are used for higher data rates.

Transmission Distance

- USB is designed mainly for short-distance peripheral communication.
- Reliable cable length depends on:
 - USB speed
 - Cable quality
 - Signal integrity
 - Timing requirements
- Higher-speed USB generally requires more tightly controlled cable characteristics.

- Uses differential signaling.
- USB 2.0 uses D+ and D- pair.
- Receiver looks at the voltage difference between D+ and D-.
- Good noise immunity.
- Designed for short-distance connections (typically up to 5 m for USB 2.0).



<https://www.ect.ac.uk/resources/data-communications-networking-and-protocols-for-industry/>

3.7 USB CONNECTORS

- USB connectors define the physical connection, not automatically the data rate.

USB Type-C

- Small and reversible.
- Used on most modern equipment.
- Depending on the device and cable, it may support:
 - USB 2.0
 - SuperSpeed USB
 - USB4
 - USB Power Delivery
 - Display and other alternate functions

USB Type-A

- Traditional rectangular host-side connector.
- Still common on PCs and industrial equipment.

USB Type-B

- Traditionally used on:
 - Printers
 - Instruments
 - DAQ equipment

Mini-USB / Micro-USB

- Used extensively in older portable equipment.
- Largely replaced by USB-C in newer designs.



<https://www.szapphone.com/blog/usb-connector-types-guide/>

3.7 USB POWER DISTRIBUTION

- USB can provide both:
 - Data
 - DC power
- Traditional USB normally supplies approximately 5 V.
- Available current depends on:
 - USB generation
 - Port type
 - Device configuration
 - Cable capability

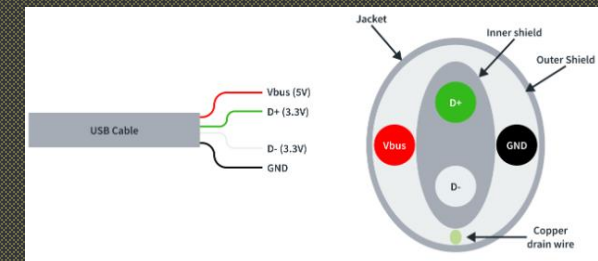
USB-C

- Can advertise higher current levels than older basic USB ports.
- Without USB Power Delivery, compatible Type-C connections can support increased current at 5 V.

USB Power Delivery (USB PD)

- Allows the source and device to negotiate:
 - Voltage
 - Current
 - Power level
- USB PD originally supported power levels up to 100 W.
- USB PD 3.1 Extended Power Range increased this to 240 W over properly rated USB-C cables.

5. USB Power Distribution		
• USB can provide both data and DC power. • Standard USB typically supplies 5 V. • Available current depends on the USB generation, port type, and cable. • USB-C can support higher current levels. • USB Power Delivery (PD) allows negotiated higher voltages and power levels (up to 240 W with PD 3.1).		
USB Version / Charging	Voltage	Typical Current / Power
USB 2.0	5 V	Up to 500 mA (2.5 W)
USB 3.0/3.1	5 V	Up to 900 mA (4.5 W)
USB-C (default)	5 V	Up to 3 A (15 W)
USB PD 3.0	5–20 V	Up to 100 W
USB PD 3.1	5–48 V	Up to 240 W



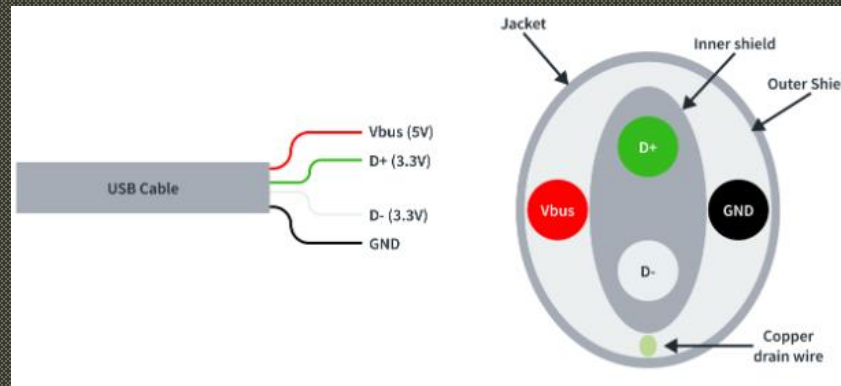
<https://www.circuitbread.com/tutorials/how-usb-works-introduction-part-1>

3.7 USB – C POWER NEGOTIATION AND DATA LINES

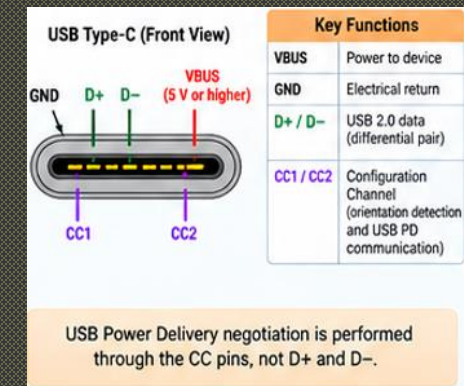
- USB-C normally begins with a safe default power condition.
- Higher-power operation must be negotiated.
- USB Power Delivery communication is performed using the USB-C Configuration Channel (CC).
- The negotiated power may increase depending on:
 - Source capability
 - Device requirement
 - Cable capability

Important Conductors

- VBUS
 - Supplies power.
- GND
 - Electrical return/reference.
- D+ and D-
 - Carry USB 2.0 data.
- CC pins
 - Used for Type-C connection/orientation detection and USB PD communication.



<https://www.circuitbread.com/tutorials/how-usb-works-introduction-part-1>



3.7 USB 2.0 ENCODING

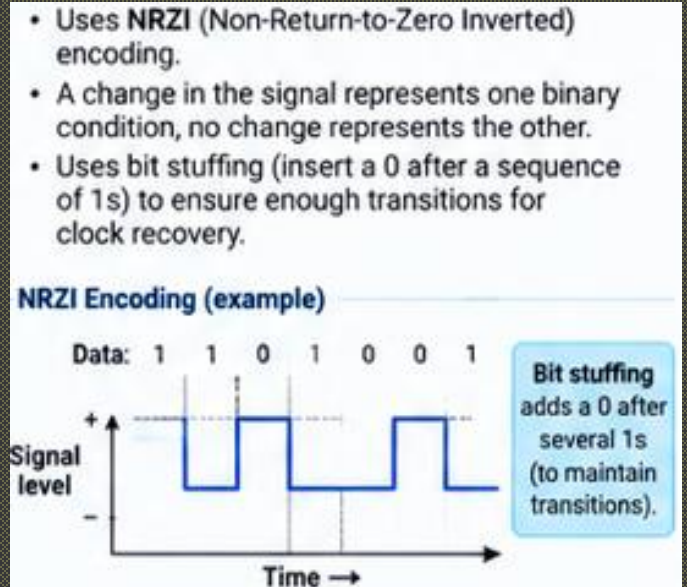
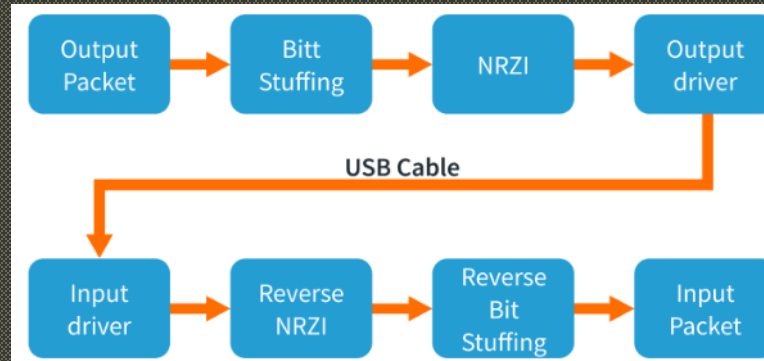
- USB 2.0 uses NRZI – Non-Return-to-Zero Inverted encoding.
- A change in the differential state represents one binary condition.
- No change represents the other condition.
- USB also uses bit stuffing.
- After a specified sequence of consecutive 1 bits, an additional 0 is inserted.
- Bit stuffing guarantees enough transitions for the receiver to maintain synchronization.
- The receiver removes the stuffed bits when recovering the original data.

Main Purpose

NRZI → represents the data

Bit stuffing → ensures enough transitions for clock recovery

USB 2.0 specifications use NRZI and bit stuffing in the physical bit stream.



3.7 USB 2.0 ENCODING

1. Transmitter Side

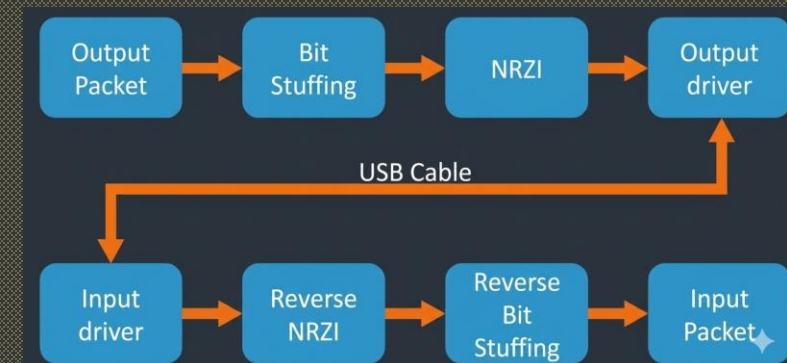
- Output Packet: Creates the initial data bundle containing the payload, headers, and error check codes.
- Bit Stuffing: Inserts a dummy 0 bit after every six consecutive 1 bits to prevent the receiver from losing timing.
- NRZI: Encodes data by changing voltage for every 0 bit and holding the voltage steady for every 1 bit.
- Output Driver: Converts these logical bits into physical voltage signals to send out over the wires.

2. Physical Layer

- USB Cable: The physical wire hardware that carries the electrical signals from sender to receiver.

3. Receiver Side

- Input Driver: Detects incoming wire voltages and translates them back into a raw, digital bitstream.
- Reverse NRZI: Decodes the voltage transitions back into regular 1 and 0 data bits.
- Reverse Bit Stuffing: Automatically removes the extra stuffed 0 bits to restore original data spacing.
- Input Packet: Fully reconstructs the clean data packet and passes it up to the software layers.



3.7 USB – HIGHER SPEED ENCODING

- As USB speeds increased, more efficient encoding methods were introduced.

SuperSpeed USB – 5 Gb/s

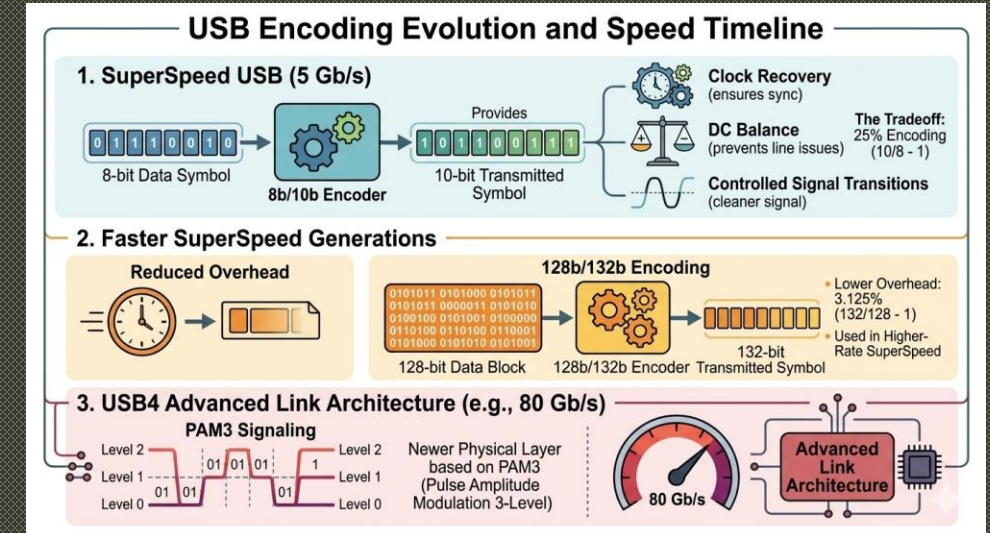
- Uses 8b/10b encoding.
- Converts 8 data bits into a 10-bit transmitted symbol.
- Provides:
 - Clock recovery
 - DC balance
 - Controlled signal transitions
- The tradeoff is encoding overhead.

Faster SuperSpeed Generations

- More efficient encoding reduces overhead.
- 128b/132b encoding is used in higher-rate SuperSpeed generations.

USB4

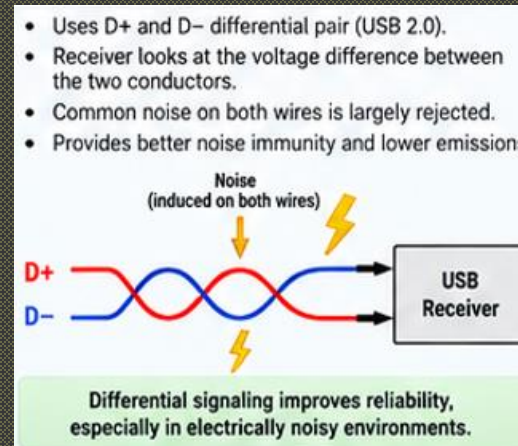
- USB4 introduces a more advanced link architecture.
- USB4 80 Gb/s uses a newer physical layer based on PAM3 signaling.



PAM3 (Pulse Amplitude Modulation with 3 Levels) is a multi-level signal encoding method (-1, 0, 1) used in high-speed data transmission systems

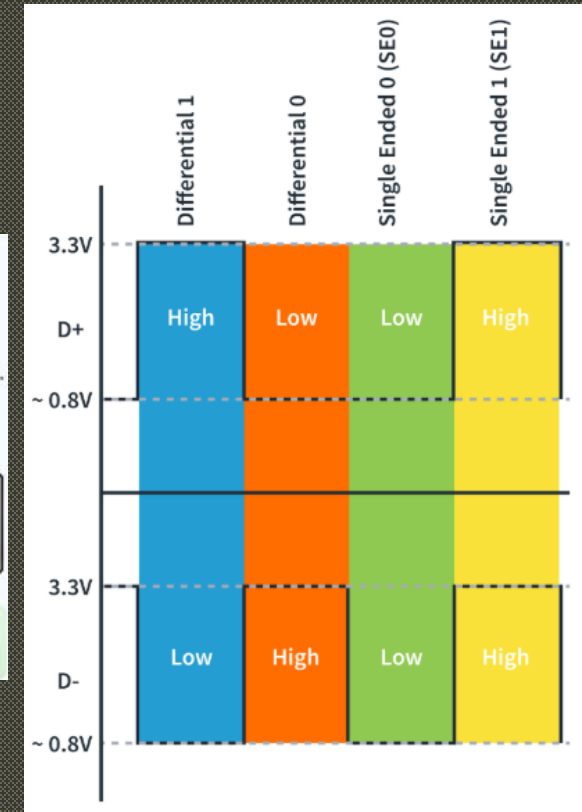
3.7 USB DIFFERENTIAL DATA TRANSMISSION

- USB 2.0 uses the D+ and D- differential pair.
- Differential signaling means the receiver looks at the relationship between the two conductors.
- External noise that couples similarly onto both wires can be largely rejected.
- Benefits include:
 - Better noise immunity
 - Reduced electromagnetic emissions
 - Reliable high-speed communication over short cables



Like RS-422/RS-485, USB benefits from differential signaling.
However:

- USB is optimized for short-distance peripheral communication.
- RS-422/RS-485 are designed for much longer industrial communication links.



3.7 USB BUS STATES AND PACKET STRUCTURE

USB 2.0 Bus States

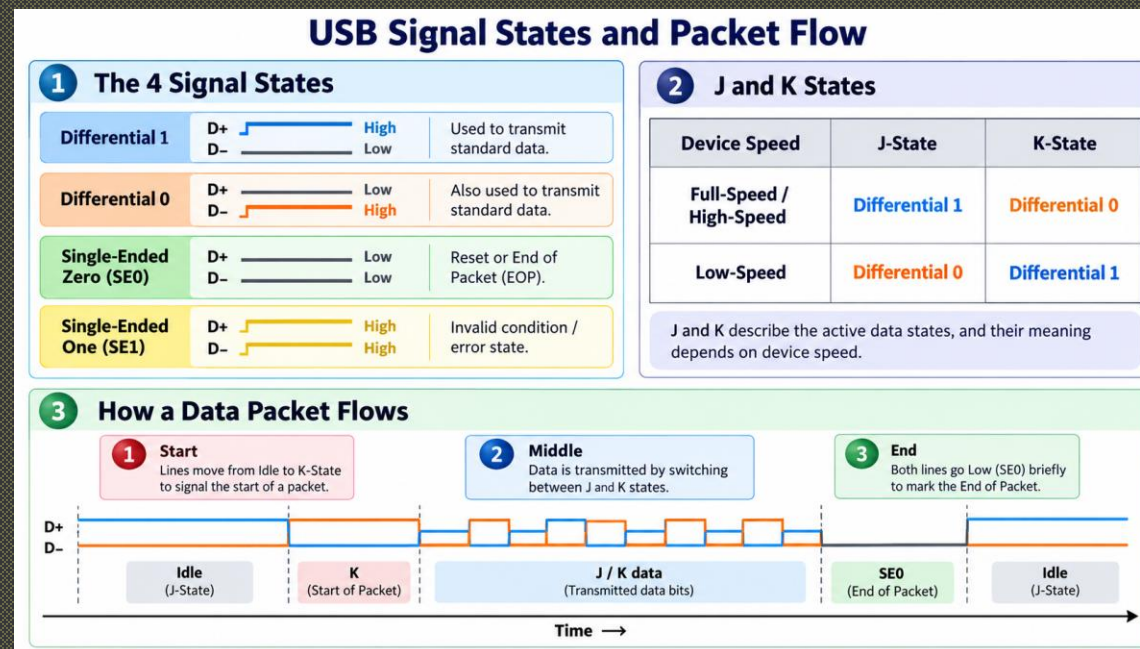
- USB represents information through differential states on D+ and D-.
 - Common terms include:
 - J state
 - K state
 - SE0 – Single-Ended Zero
 - The exact D+/D- polarity associated with J and K depends on the USB speed.
- A change from J to K or K to J represents a data 0.
 - No change represents a data 1.
 - If there are too many 1s in a row, USB inserts an extra 0 so that a J/K change occurs and the receiver stays synchronized.

Packet Concept

USB communication is organized into packets.

A packet may contain:

- SYNC
 - Helps synchronize the receiver.
- PID – Packet Identifier
 - Identifies the packet type.
- Address / Endpoint information
 - Used in applicable packet types.
- Payload
 - Actual data when required.
- CRC
 - Provides error detection.
- EOP – End of Packet
 - Marks the end of transmission.



3.7 USB DATA TRANSMISSION

The 4 Signal States

- Differential 1 (Blue): D+ is High and D- is Low - Used to transmit standard data.
- Differential 0 (Orange): D+ is Low and D- is High - Also used to transmit standard data.
- Single-Ended Zero / SE0 (Green): Both wires are driven Low at the same time. This tells the system to Reset or marks the End of Packet (EOP).
- Single-Ended One / SE1 (Yellow): Both wires are driven High at the same time. This is an invalid condition that indicates a system Error.

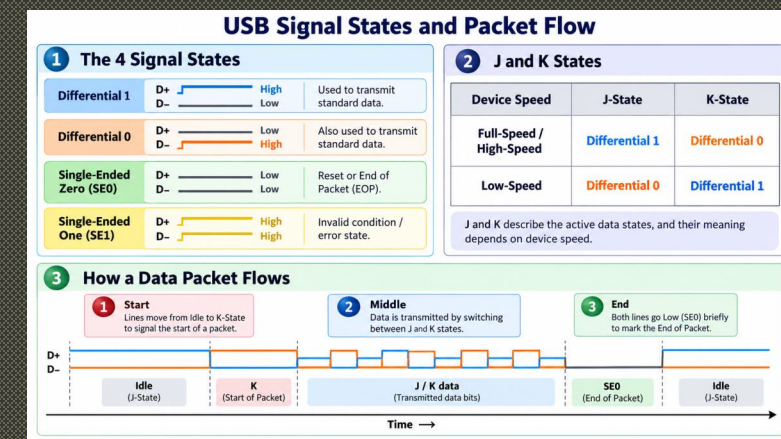
J and K States

USB uses the terms J-State and K-State to describe data states, but their actual meanings flip depending on how fast the device is running:

- Full-Speed / High-Speed Devices: J-State means Differential 1 and K-State means Differential 0
- Low-Speed Devices: J-State means Differential 0 and K-State means Differential 1

How a Data Packet Flows

- Start: The lines shift from resting (Idle) into a K-State to signal a new packet is beginning.
- Middle/Body: The system bounces between J and K states to send the actual 1s and 0s of data.
- End: The transmitter drops both lines to zero (SE0 state) for a moment to mark the end of the packet.



3.7 THE UNIVERSAL SERIAL BUS (USB)

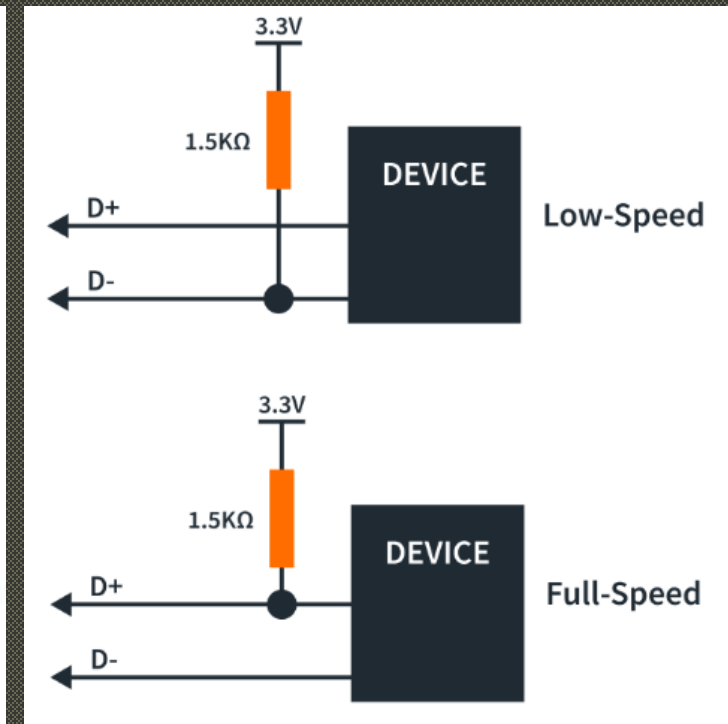
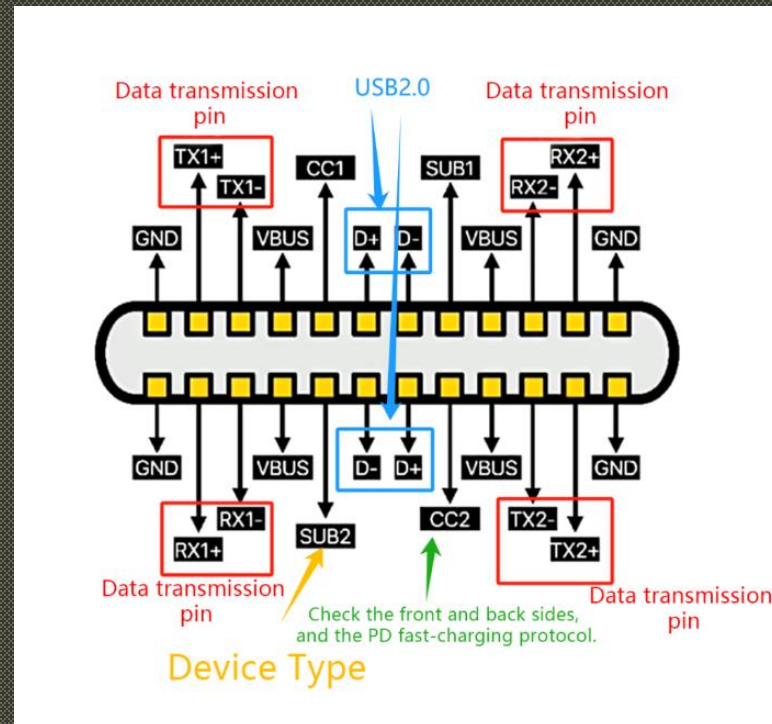
Speed Detection

- When a device is plugged in, the host/computer checks a simple signal:
 - If there is a resistor on the D- line, the device is a Low-Speed device.
 - If there is a resistor on the D+ line, the device is a Full-Speed device.
- For SuperSpeed (USB 3.0+) and USB4, the devices use a separate set of wires inside the cable specifically designed for super-fast data. The computer detects these new wires and knows it can use the "super-highway."

A USB-C connector has up to 24 pins that handle power, high-speed data, legacy data, configuration, and sideband use.

Key pins include (top and bottom are mirrored):

- VBUS and GND for power,
- D+/D- and TX/RX pairs for data,
- CC pins for orientation and power negotiation, and
- SUB pins for non-USB alternate modes like video.



3.7 USB SYSTEM ARCHITECTURE

Starting Point – Raw Data

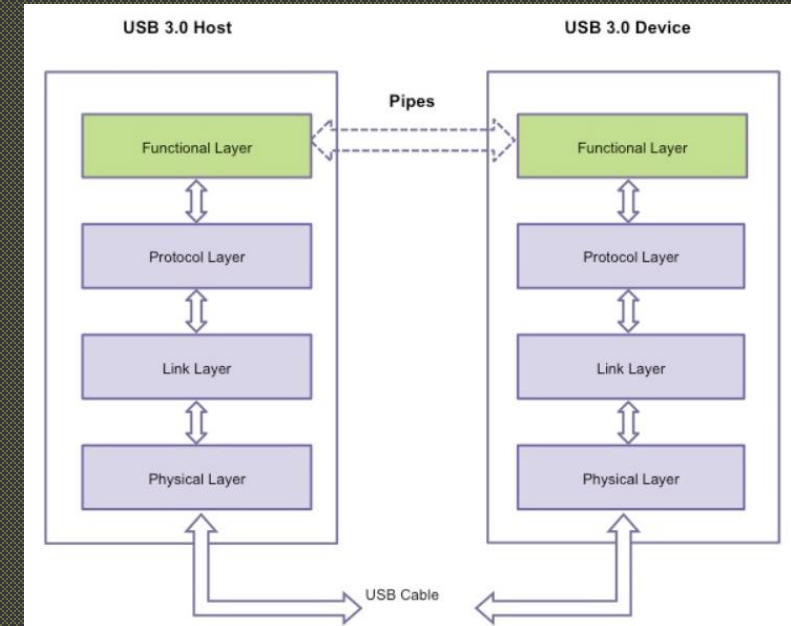
- An application asks the operating system to send data to a USB device.
- Example: A file explorer sends part of a file to a USB flash drive.
- The data begins as a block of raw bytes.

Step 1 – Functional Layer

- Determines what the data means.
- The USB class driver interprets the application request.
- Example: A Mass Storage Class Driver converts a file-copy request into a storage command.
- The data is directed to a logical channel such as a Bulk Out Pipe.

Step 2 – Protocol Layer

- Determines where the data is going and how it is identified.
- Creates a USB packet.
- Adds information such as:
 - PID – Packet Identifier
 - Device address
 - Endpoint information
- The payload is now organized as a USB protocol packet.



<https://www.techdesignforums.com/practice/technique/usb-3-0-functional-layer/>

USB Transfer Types			
Transfer Type	Purpose	Characteristics	Examples
Control	Device setup and configuration	Reliable, used during enumeration	Get/set descriptors
Bulk	Large amounts of data	Reliable, with error detection/retry (no guarantee/ timing)	Storage devices, printers
Interrupt	Small, time-sensitive data	Polled by host at intervals	Keyboard, mouse, sensors
Isochronous	Continuous real-time data	Reserved bandwidth, no retries	Audio, video streams

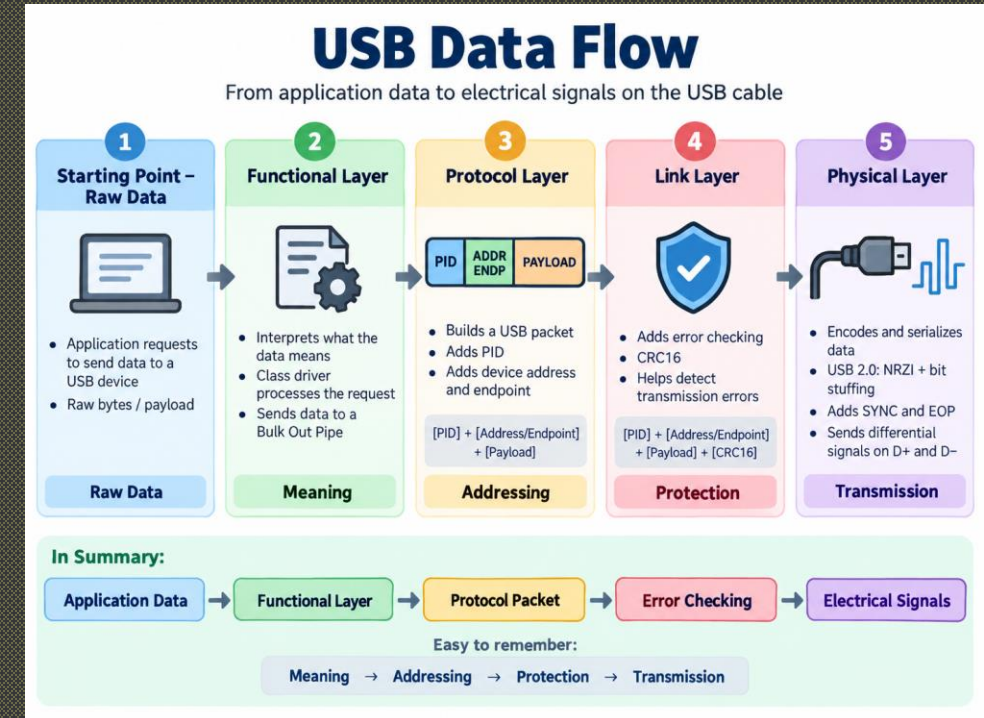
3.7 USB SYSTEM ARCHITECTURE

Step 3 – Link Layer

- Handles:
 - Error checking
 - Flow control
 - Transmission tracking
- Calculates and adds a CRC16 error-checking value.
- Helps the host detect whether data was received correctly.

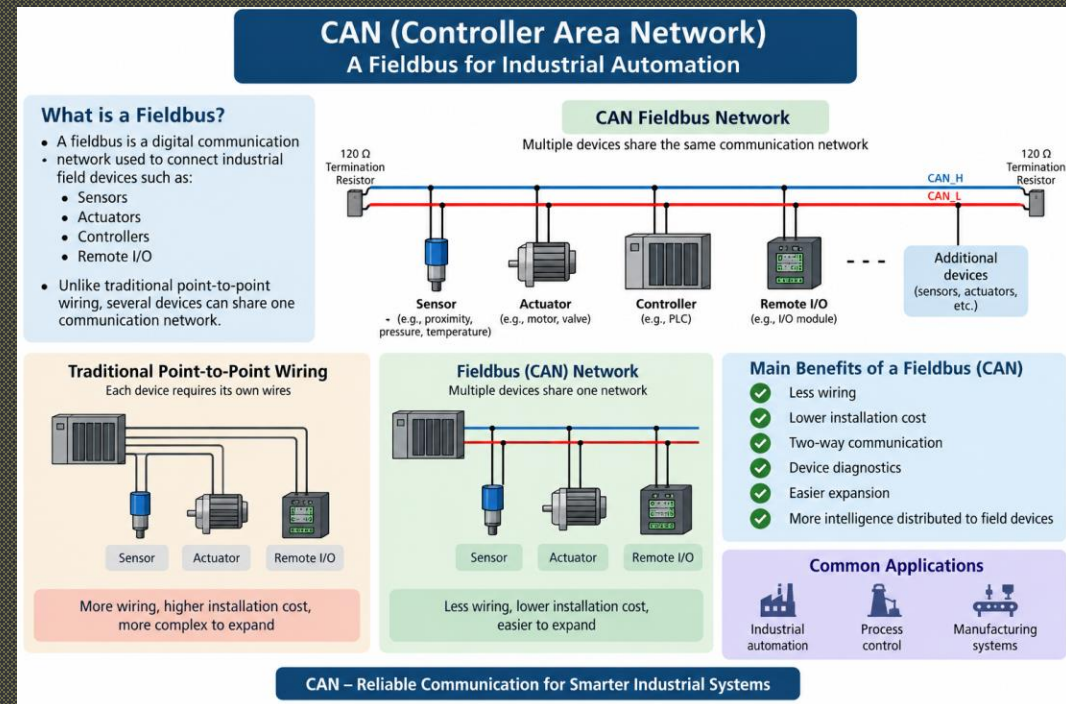
Step 4 – Physical Layer

- Converts the digital packet into actual electrical signals by performing:
 - Encoding
 - Synchronization
 - Serialization
- USB 2.0 uses:
 - NRZI encoding
 - Bit stuffing
- SuperSpeed USB can use encoding such as: 8b/10b
- Adds synchronization information at the beginning and an End of Packet (EOP) condition at the end.
- Sends the data electrically over:
 - D+ and D– for USB 2.0



3.8 Fieldbus: Basic Concept

- A fieldbus is a digital communication network used to connect industrial field devices such as:
 - Sensors
 - Actuators
 - Controllers
 - Remote I/O
- Unlike traditional point-to-point wiring, several devices can share one communication network.
- Main benefits:
 - Less wiring
 - Lower installation cost
 - Two-way communication
 - Device diagnostics
 - Easier expansion
 - More intelligence distributed to field devices
- Fieldbus systems are commonly used in:
 - Industrial automation
 - Process control
 - Manufacturing systems



3.8 FIELDBUS: EVOLUTION AND TOPOLOGIES

Before Fieldbus

- Industrial systems commonly used:
 - Pneumatic signals
 - Analog signals such as 4–20 mA
 - Centralized controllers
- Large systems required extensive wiring.

Fieldbus Development

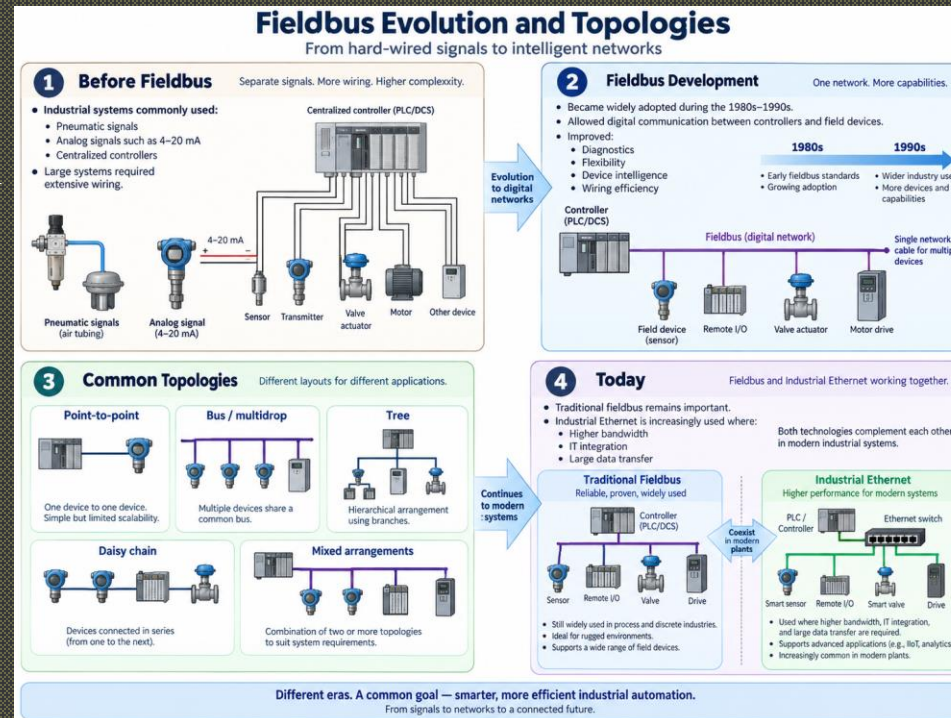
- Became widely adopted during the 1980s–1990s.
- Allowed digital communication between controllers and field devices.
- Improved:
 - Diagnostics
 - Flexibility
 - Device intelligence
 - Wiring efficiency

Common Topologies

- Point-to-point
- Bus / multidrop
- Tree
- Daisy chain
- Mixed arrangements

Today

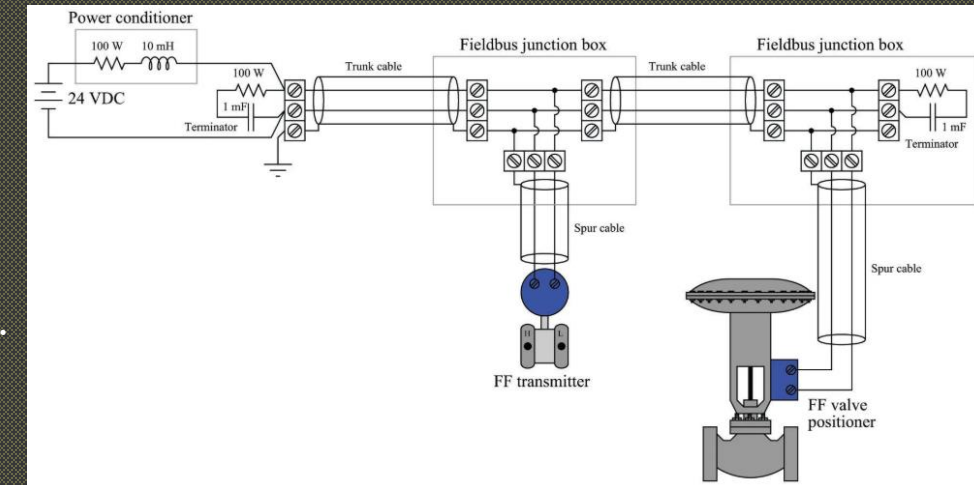
- Traditional fieldbus remains important.
- Industrial Ethernet is increasingly used where:
 - Higher bandwidth
 - IT integration
 - Large data transfer are required.



3.8 WHY FIELDBUS NETWORKS NEED TERMINATION

- Long communication cables behave like transmission lines.
- Electrical signals can reflect when they reach the end of the cable.
- Reflections may cause:
 - Distorted signals
 - Communication errors
 - Loss of communication
- Terminators are installed at the ends of the network to reduce reflections.
- For the fieldbus example shown:
 - A resistor helps match the cable impedance.
 - A capacitor allows the communication signal while blocking DC power.
- Normally:
 - One terminator is placed at each physical end.

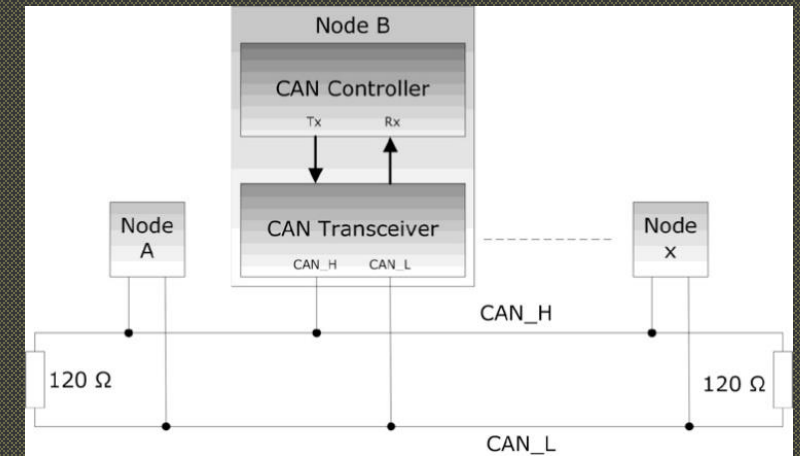
Two correctly placed terminators help keep the communication signal clean.



<https://control.com/textbook/foundation-fieldbus-instrumentation/h1-ff-physical-layer/>

3.8 INTRODUCTION TO CONTROL AREA NETWORK (CAN)

- CAN = Controller Area Network
- Developed by Bosch for automotive applications.
- Standardized through the ISO 11898 family.
- Designed so several controllers can communicate over a shared bus.
- CAN is a multi-master network:
 - Any node may attempt to transmit when the bus is free.
- Uses message identifiers to determine:
 - Message meaning
 - Message priority
- Important characteristics:
 - Reliable
 - Deterministic arbitration
 - Good error detection
 - Suitable for real-time control
 - Distributed control
- Originally developed for vehicles but later adopted in industrial automation.



<https://www.ti.com/video/series/precision-labs/ti-precision-labs-can-lin-sbc.html>

3.8 MAIN CAN FEATURES

- Multi-master
 - Any node can transmit when the bus is idle.
- Message priority
 - Message identifiers determine priority.
 - Lower numerical identifier → higher priority.
- Message-based communication
 - CAN messages are identified by their identifier rather than by a destination address.
- Broadcast operation
 - All nodes can see a transmitted message.
 - Each node decides whether the message is relevant.
- Error handling
 - CAN detects communication errors and can automatically recover.
- Flexible network size
 - Physical limits depend on:
 - Transceivers
 - Cable
 - Bit rate
 - Network loading

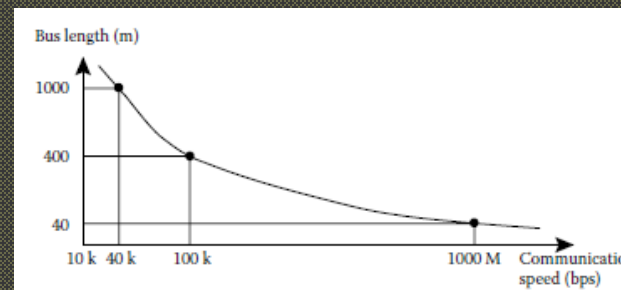
Types of CAN (ISO Standards)

- ISO 11898 (High-speed CAN)
 - Split into:
 - ISO 11898-1 → Data link layer.
 - ISO 11898-2 → Physical layer.
 - Used for speeds up to 1 Mbps (commonly in vehicles).
- ISO 11519-2 (Low-speed CAN)
 - Speeds up to 125 kbps.
 - Different physical layer compared to ISO 11898.

Speed vs. Bus Length

Inverse relationship:

- Higher speed → shorter maximum bus length.
- Longer bus → lower communication speed.



CAN communicates by message ID, not by directly addressing a particular receiving device.

3.8 CAN PHYSICAL LAYER: SPEED AND DISTANCE

High-Speed CAN

- Up to approximately 1 Mb/s in Classical CAN.
- Typical maximum cable length depends strongly on bit rate.
- Example:
 - High speed → short cable
 - Lower speed → much longer cable

Low-Speed / Fault-Tolerant CAN

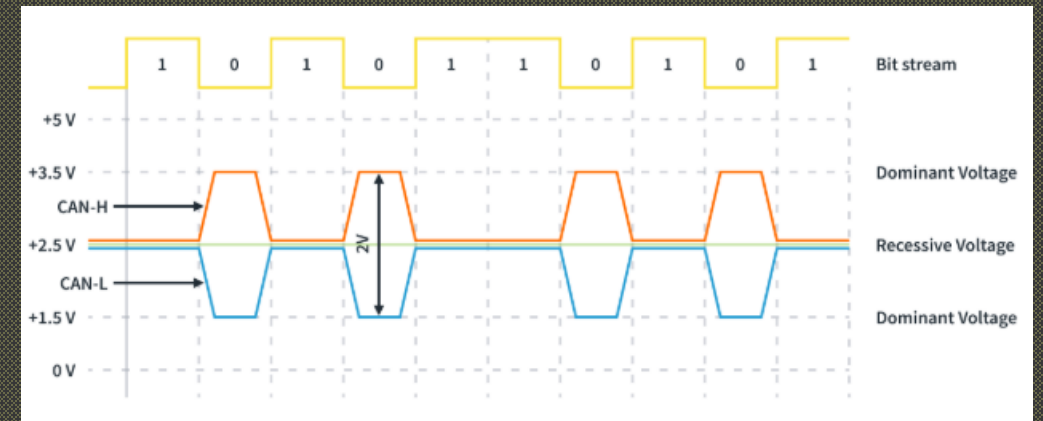
- Lower maximum bit rate.
- Designed for applications requiring improved fault tolerance.

Speed-Distance Relationship

- Communication delay through the cable limits network size.
- Therefore:

Higher bit rate → shorter cable

Lower bit rate → longer cable



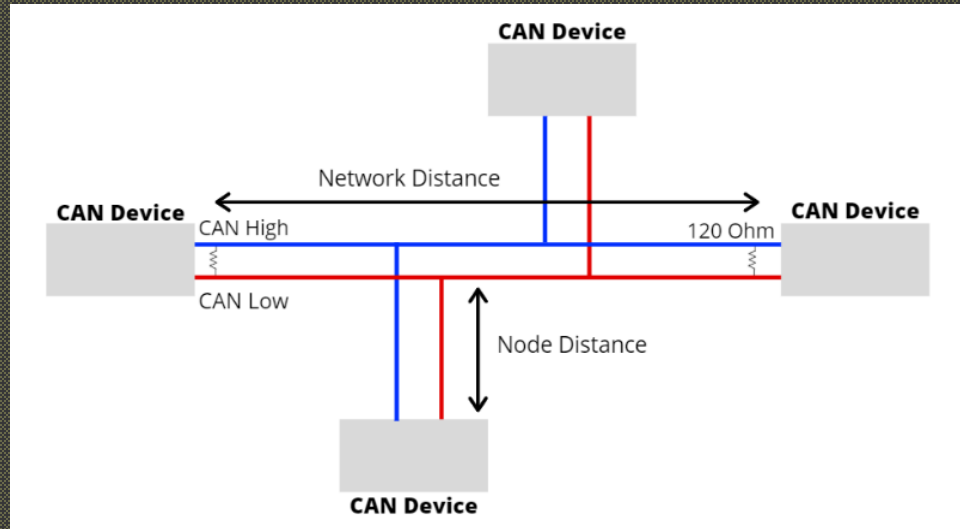
<https://www.circuitbread.com/tutorials/understanding-can-a-beginners-guide-to-the-controller-area-network-protocol>

3.8 CAN PHYSICAL LAYER: WIRING AND TERMINATION

- Normally uses a linear bus / trunk topology.
- Main conductors:
 - CAN_H
 - CAN_L
- Uses twisted-pair cable.
- Typically terminated with:
 - 120 Ω at one physical end
 - 120 Ω at the other physical end

Why termination?

- Matches the transmission-line impedance.
- Reduces signal reflections.
- Improves signal integrity.



Bus resistance check

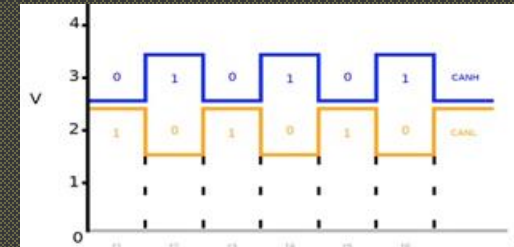
- Two 120 Ω terminators in parallel give approximately: 60 Ω
- With the network powered OFF, measuring about 60 Ω between CAN_H and CAN_L is a useful basic troubleshooting check.

3.8 DOMINANT AND RECESSIVE CAN STATES

CAN uses two logical bus states:

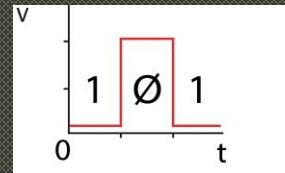
Recessive State

- Represents logic 1.
- CAN_H and CAN_L are relatively close together in voltage.
- Differential voltage is small.

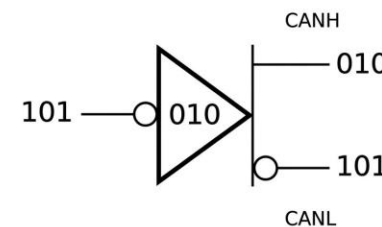


Dominant State

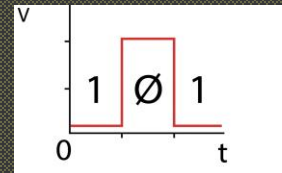
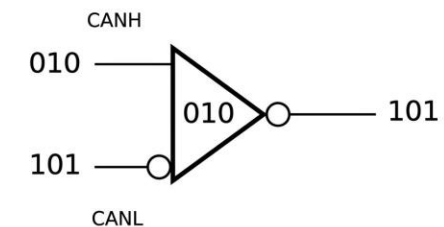
- Represents logic 0.
- CAN_H rises while CAN_L falls.
- Differential voltage becomes larger.



CAN Output Driver



CAN Input Receiver



- Dominant overrides recessive.
- If one device sends dominant while another sends recessive:
 - The bus becomes dominant.

This electrical behavior makes CAN arbitration possible.

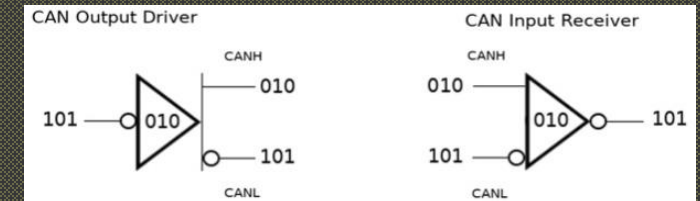
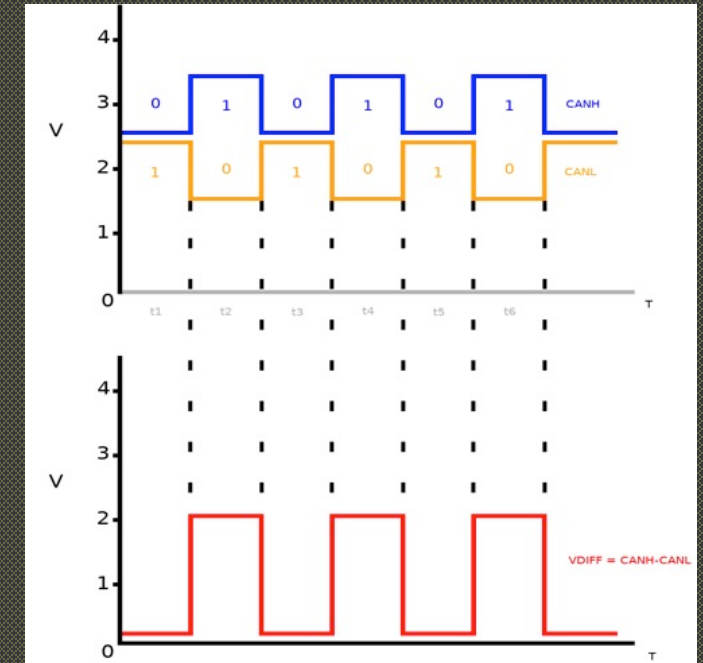
- 0 = Dominant
- 1 = Recessive
- Dominant wins

3.8 CAN_H AND CAN_L DURING TRANSMISSION

- CAN uses two wires:
 - CAN_H
 - CAN_L
- In the recessive state:
 - The two voltages are relatively close.
- In the dominant state:
 - CAN_H increases.
 - CAN_L decreases.
- The receiver examines the voltage difference between the two lines.

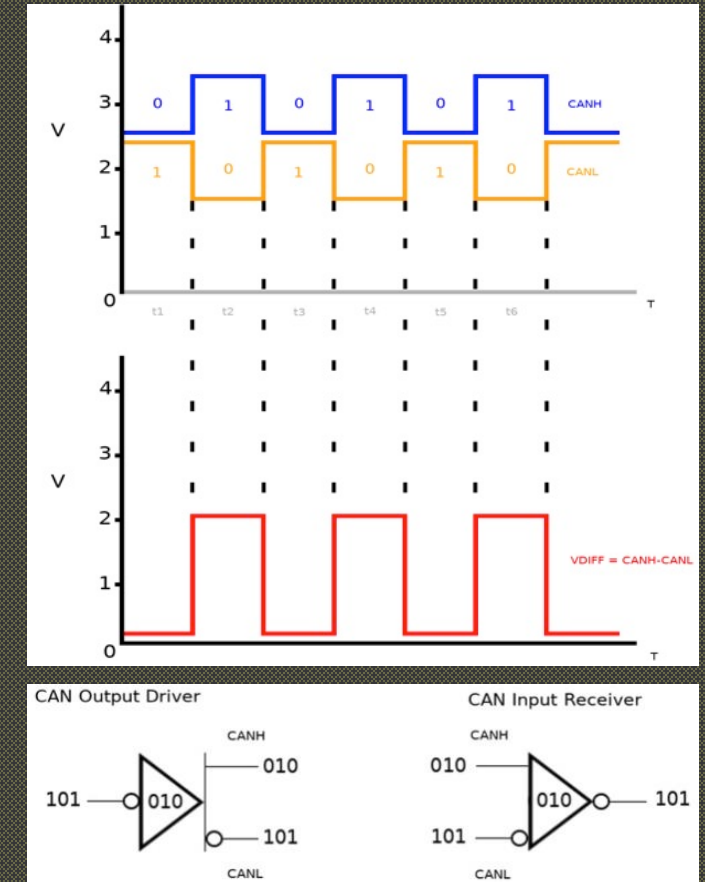
Typical High-Speed CAN concept

- Recessive:
 - $CAN_H \approx 2.5\text{ V}$
 - $CAN_L \approx 2.5\text{ V}$
 - Differential voltage $\approx 0\text{ V}$
- Dominant:
 - $CAN_H \approx 3.5\text{ V}$
 - $CAN_L \approx 1.5\text{ V}$
 - Differential voltage $\approx 2\text{ V}$



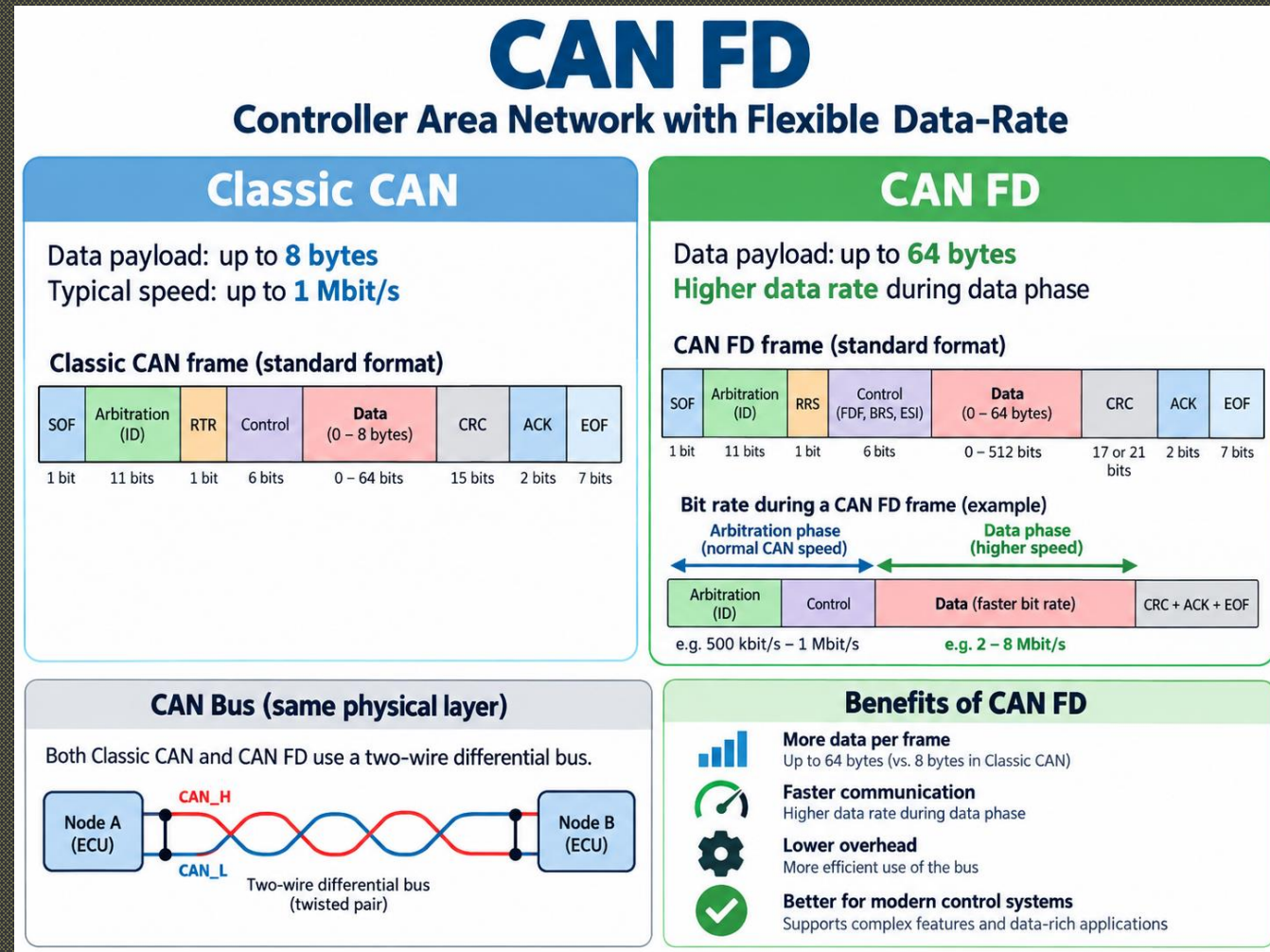
3.8 CAN RECESSIVE AND DOMINANT STATES

- At t_1 , the transmitted bit is logic 1.
 - CAN_H and CAN_L are at nearly the same voltage.
 - Therefore, the differential voltage $V_{DIFF} = CAN_H - CAN_L$ is close to 0 V.
 - This condition is called the recessive state.
- At t_2 , the transmitted bit is logic 0.
 - CAN_H voltage increases.
 - CAN_L voltage decreases.
 - This creates a larger voltage difference between CAN_H and CAN_L.
 - Therefore, V_{DIFF} becomes high.
 - This condition is called the dominant state.
- Why CAN logic appears inverted:
 - Logic 1 $\rightarrow$ low differential voltage $\rightarrow$ Recessive
 - Logic 0 $\rightarrow$ high differential voltage $\rightarrow$ Dominant



3.8 CAN FD – CONTROLLER AREA NETWORK WITH FLEXIBLE DATA-RATE

- CAN FD is an improved version of Classic CAN.
- Designed to transmit more data at higher speeds.
- Supports up to 64 bytes of data per frame.
 - Classic CAN supports only 8 bytes.
- Can use:
 - Normal CAN speed during arbitration.
 - Higher speed during the data portion of the frame.
- Uses the same basic CAN_H / CAN_L differential signaling.
- Still uses dominant and recessive bits.



3.8 CAN DATA FRAME STRUCTURE

1. Data Frame

- Used to transmit actual application data.
- Contains:
 - Start of Frame
 - Arbitration field
 - Control information
 - Data
 - CRC
 - ACK
 - End of Frame

2. Remote Frame

- Used in Classical CAN to request data associated with a particular identifier.
- Contains no application-data payload.
- Remote frames are part of Classical CAN and are not used in CAN FD.

3. Error Frame

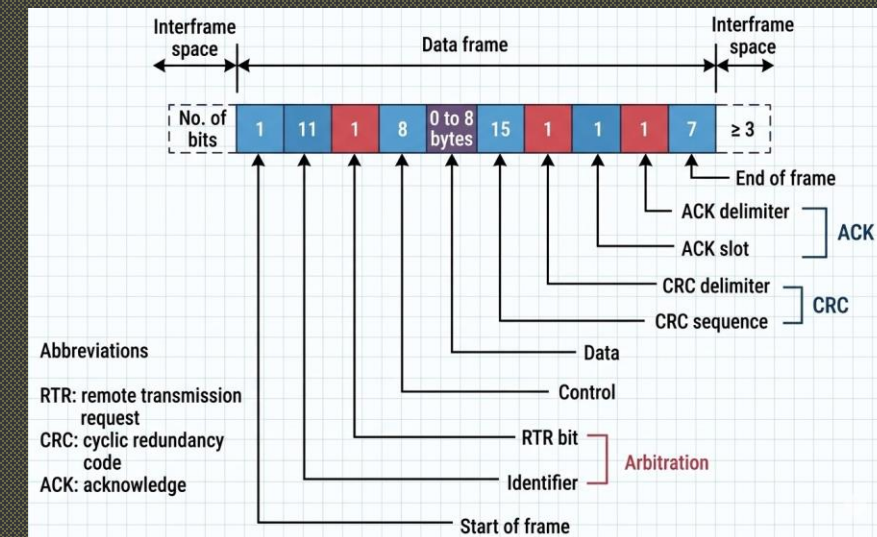
- Automatically generated when a node detects an error.
- Alerts other nodes that the current message is invalid.

4. Overload Frame

- Allows additional delay before the next frame.
- Rare in many modern implementations.

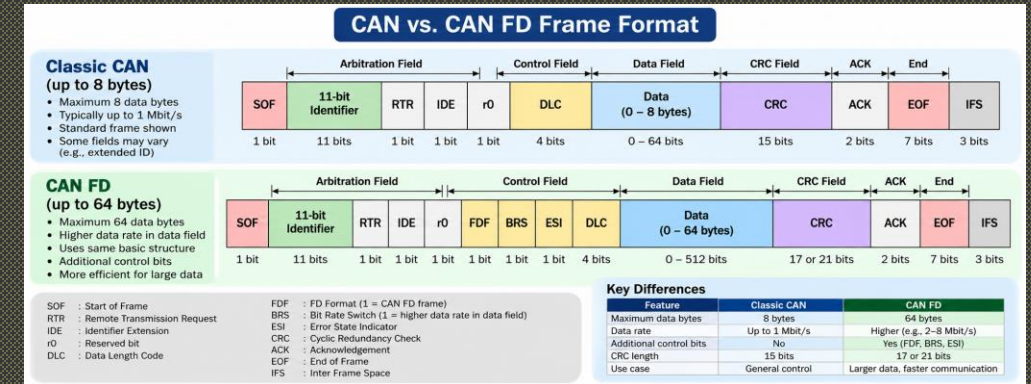
5. Interframe Space

- Small separation between frames.
- Gives the bus time before the next normal transmission begins.



3.8 CAN DATA FRAME STRUCTURE

- The CAN protocol supports two message frame formats, CAN base frame format and the CAN extended frame format (the difference is in the length of the identifier, 11 bits vs 29 bits).

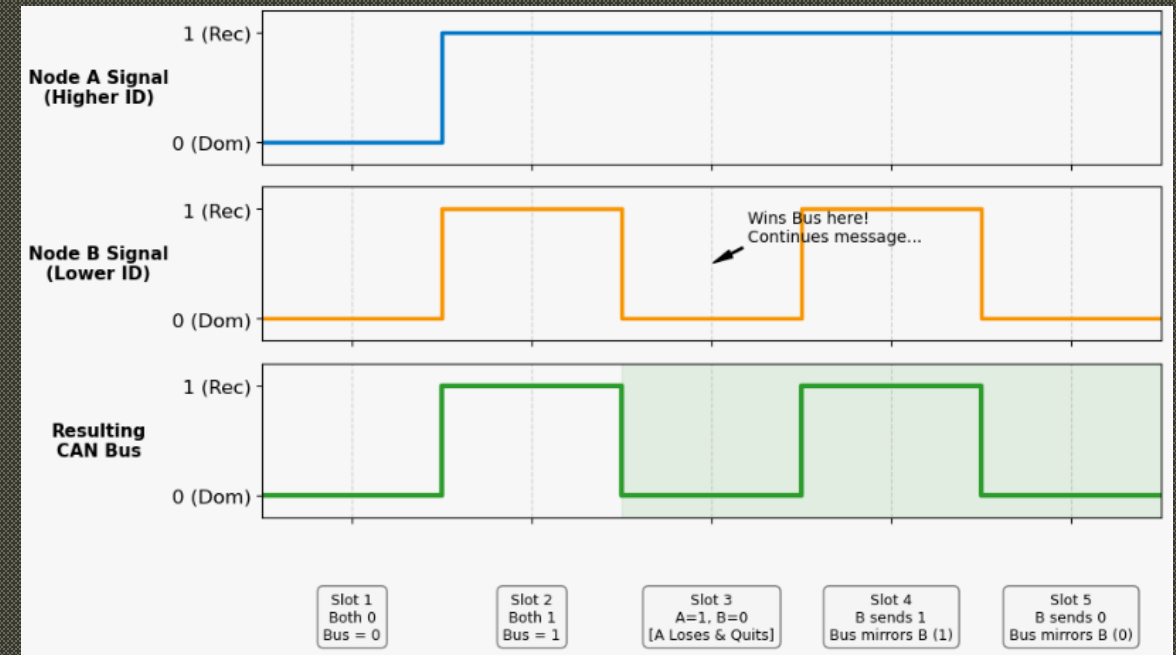


- SOF - start of frame, 1 bit (must be dominant)
- ID - arbitration field consisting of the identifier, 11 bits (indicates priority)
- RTR - remote transmission request to distinguish between the data frame and the remote frame, 1 bit (dominant in data frames; recessive in remote frames)
- IDE - identifier extension to distinguish between the CAN base frame and extended frame, 1 bit
- r0 – Reserved, 2 bits (must be dominant)
- DLC - data length code to indicate the number of data bytes in the data field, 4 bits
- Data – Data field holds up to 8 bytes
- CRC - cyclic redundant check sum, 15 bits
- ACK – comprises the ACK slot (transmitter sends recessive; receiver asserts dominant) and the ACK delimiter (must be recessive), 2 bits
- EOF – end of frame, 7 bits (must be recessive)
- IFS - intermission frame space (IFS) is the minimum time in equivalent number of bits separating consecutive messages

3.8 CAN ARBITRATION: BASIC PRINCIPLE

Arbitration occurs when multiple nodes begin transmitting at approximately the same time.

- Each device transmits its identifier.
- At the same time, each device monitors the bus.
- CAN uses:
 - 0 = dominant
 - 1 = recessive
- If a node transmits 1 but reads 0:
 - It loses arbitration.
 - It stops transmitting.
- The node with the highest-priority identifier continues.



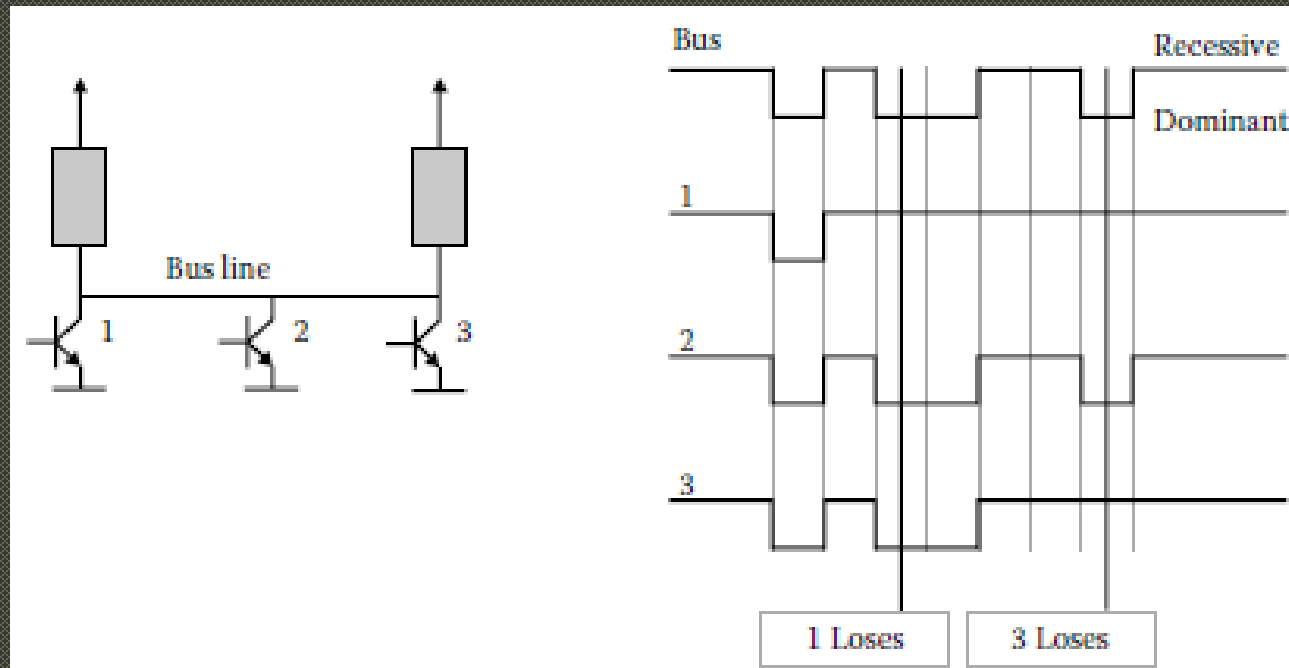
Lower numerical identifier → higher priority

CAN arbitration occurs bit by bit while the identifier is being transmitted.

- Recessive: $CAN_H \approx 2.5\text{ V}$, $CAN_L \approx 2.5\text{ V}$ (diff $\approx 0\text{ V}$)
- Dominant: $CAN_H \approx 3.5\text{ V}$, $CAN_L \approx 1.5\text{ V}$ (diff $\approx 2\text{ V}$)

3.8 CAN ARBITRATION EXAMPLE

- Example
 - Devices 1, 2, and 3 all try to transmit.
 - Device 1 loses first (its ID has a recessive bit where others send dominant).
 - Device 3 then loses to Device 2.
 - Device 2 wins arbitration and transmits its message.
 - Losers (1 and 3) wait for the next idle bus and retry automatically.



3.8 ADVANTAGES OF CAN ARBITRATION

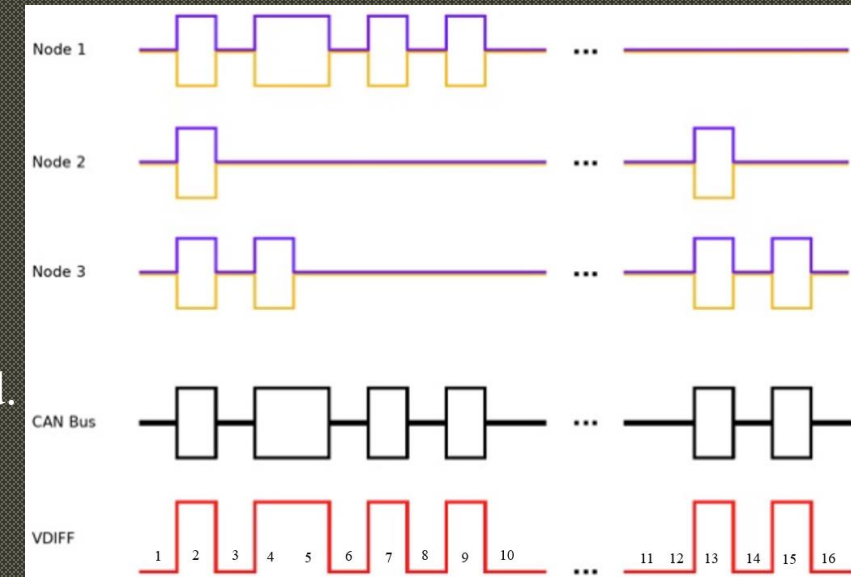
- Non-destructive
 - Winning message is not corrupted.
- No wasted collision frame
 - Unlike a traditional collision-and-retry system.
- Priority based
 - Critical messages can be transmitted first.
- Deterministic
 - High-priority messages have predictable access.
- Efficient bus utilization
 - Arbitration occurs while useful identifier bits are already being transmitted.

Industrial significance

Critical information such as:

- Brake commands
- Safety-related status
- Fault messages

can be assigned higher priority than less urgent information.



3.8 CAN COMMUNICATION SIGNALS

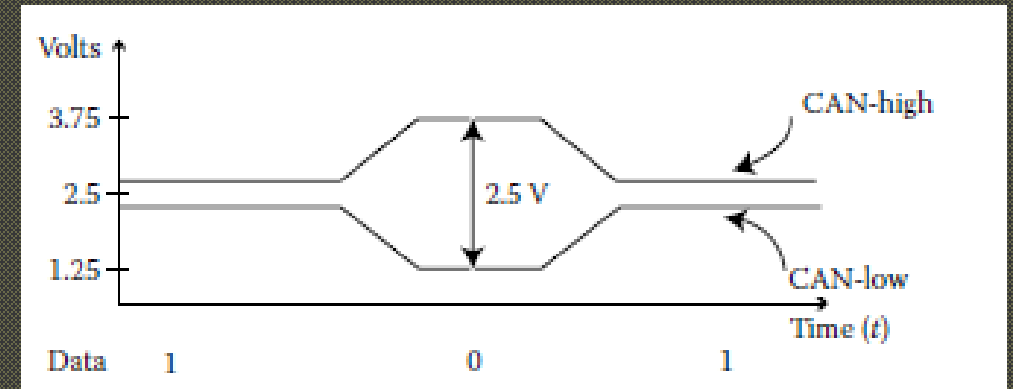
CAN communicates through the voltage difference between CAN_H and CAN_L.

Bus Idle

- CAN_H and CAN_L are close to approximately the same voltage.
- This corresponds to the recessive state.

Dominant Transmission

- CAN_H rises.
- CAN_L falls.
- Differential voltage increases.



Differential signaling

The receiver evaluates:

$$V_{CAN} = CAN_H - CAN_L$$

Why twisted pair?

- External interference tends to affect both conductors similarly.
- Differential reception rejects much of this common noise.

- CAN bus uses differential signaling on CAN_H and CAN_L.
- Idle = both at 2.5 V (recessive).
- Start of Frame = 3.75 V (CAN_H) & 1.25 V (CAN_L), creating a 2.5 V difference (dominant)
- Data = Combination of 0s and 1s (recessive and dominant)
- This design ensures high noise immunity and reliable communication in demanding environments.

3.8 CAN ERROR DETECTION

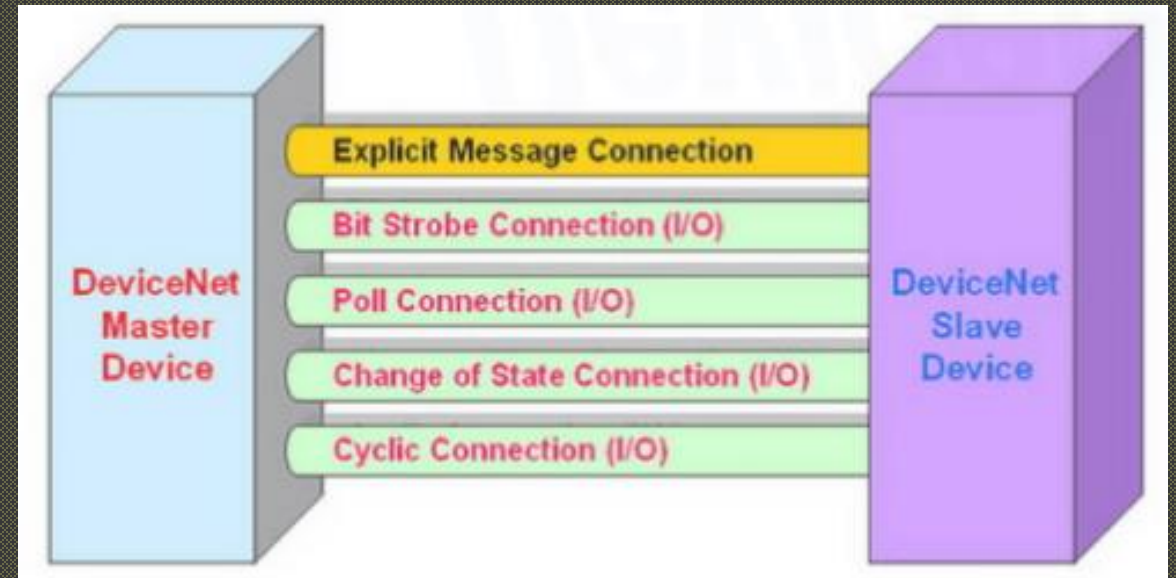
- Bit Error
 - Occurs when the transmitted bit does not match the actual bus level.
 - Example: Device sends dominant (0) but sees recessive (1).
 - Detected in data, remote, error, overload frames.
 - Checked by both transmit and receive units.
- Stuffing Error
 - CAN uses bit stuffing for synchronization: after 5 consecutive identical bits, an opposite bit is inserted.
 - Error occurs if 6 identical bits in a row are detected.
 - Detected in SOF to CRC sequence of data/remote frames.
- CRC Error
 - Detected when the calculated CRC and the received CRC do not match.
 - Detected by receiving units only.
- Form Error
 - Occurs if a field with a fixed format (e.g., CRC delimiter, ACK delimiter, EOF) is violated.
 - Detected in data, remote, error delimiter, overload delimiter frames.
- ACK Error
 - After transmitting, the sender expects an ACK (acknowledge bit) from receivers.
 - If the ACK slot stays recessive (1) → means no receiver acknowledged.
 - Detected by transmitting unit only.

3.8 DEVICENET INTRODUCTION

- DeviceNet is an industrial communication network used mainly in factory automation.
- Developed and managed through ODVA. *CAN provides the communication foundation; DeviceNet adds industrial device communication rules on top.*
- Built using the CAN physical/data-link technology.
- Uses CIP – Common Industrial Protocol at the upper layers.
- Connects devices such as:
 - PLCs
 - Sensors
 - Motor starters
 - Drives
 - Remote I/O
 - Operator interfaces

Communication can support:

- Producer-consumer communication
- Explicit messaging
- I/O messaging
- Peer-to-peer communication



- Explicit Messaging – a multipurpose communication path to perform request response functions such as device configuration.
- Strobing – the scanner broadcasts a request to all devices for a status update.
- Polling – the scanner/master asks each device for an update of its status.
- Change of state – devices only send messages when their status changes.
- Cyclic – devices automatically send messages on scheduled intervals.

3.8 DEVICENET PHYSICAL NETWORK

Topology

- Uses a trunk-and-drop arrangement.
- Main trunk carries the network.
- Short drop cables connect individual devices.

Device Addresses

- DeviceNet uses MAC IDs from:
 - 0–63
- Supports up to 64 node addresses.

Typical Data Rates and Trunk Distances

- 125 kb/s → approximately 500 m
- 250 kb/s → approximately 250 m
- 500 kb/s → approximately 100 m

Cable

- Standard DeviceNet cable provides:
 - CAN communication signals
 - Network power

Termination

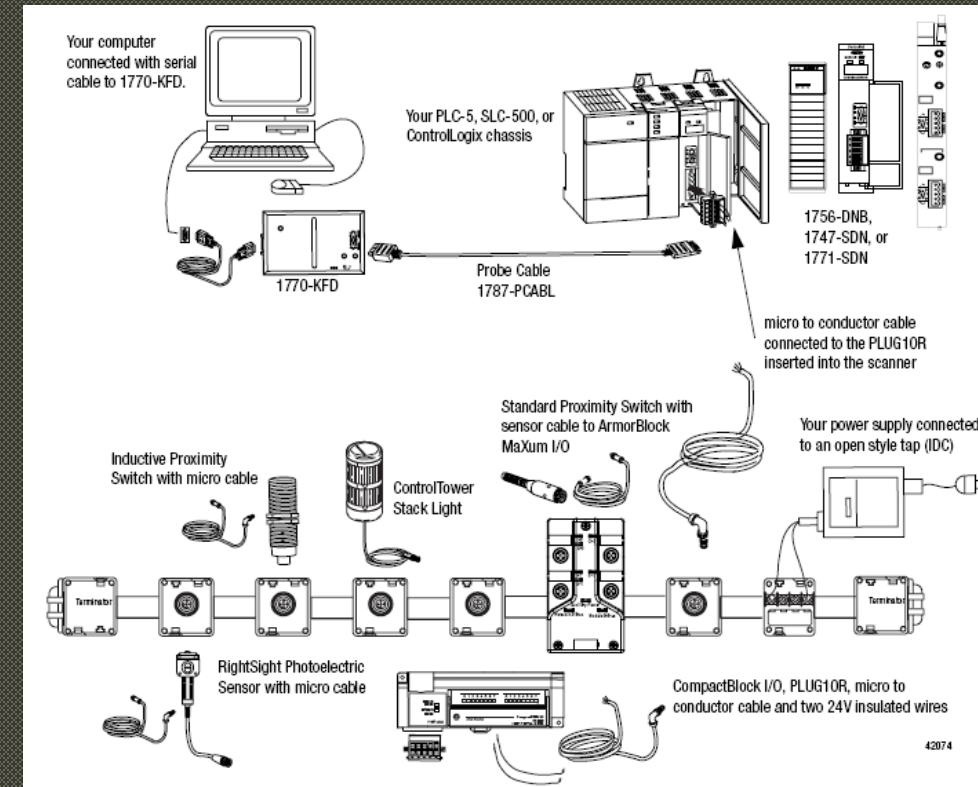
Suggested addition:

- The trunk should normally have 121 Ω termination at each physical end.

Higher-Level Protocol

- DeviceNet uses:

- CAN for lower-level communication
- CIP for application/device communication



3.8 CIP ON DEVICENET: HOW DEVICES EXCHANGE INFORMATION

What is CIP?

- CIP = Common Industrial Protocol
- CIP provides the application-level rules used by DeviceNet devices.
- It defines:
 - How device data is organized
 - How devices identify their functions
 - How controllers read/write parameters
 - How real-time I/O data is exchanged
- CIP is also used by other ODVA networks, including:
 - EtherNet/IP
 - ControlNet

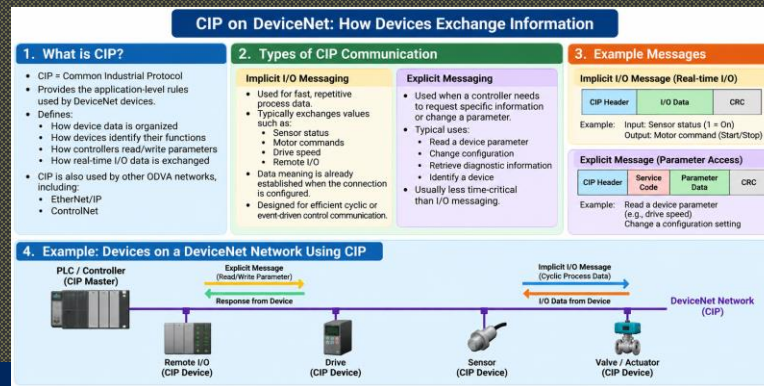
Two Important Types of Communication

Implicit / I/O Messaging

- Used for fast, repetitive process data.
- Typically exchanges values such as:
 - Sensor status
 - Motor commands
 - Drive speed
 - Remote I/O
- Data meaning is already established when the connection is configured.
- Designed for efficient cyclic or event-driven control communication.

Explicit Messaging

- Used when a controller needs to request specific information or change a parameter.
- Typical uses:
 - Read a device parameter
 - Change configuration
 - Retrieve diagnostic information
 - Identify a device
- Usually less time-critical than I/O messaging.



3.8 CIP ON DEVICENET: HOW DEVICES EXCHANGE INFORMATION

CIP Object Model

CIP organizes information into objects.

A simple hierarchy is:

Class → Instance → Attribute

Example:

- Class: Motor object
- Instance: Motor 1
- Attribute: Speed

A CIP service can then request an action such as:
Read Motor 1 Speed

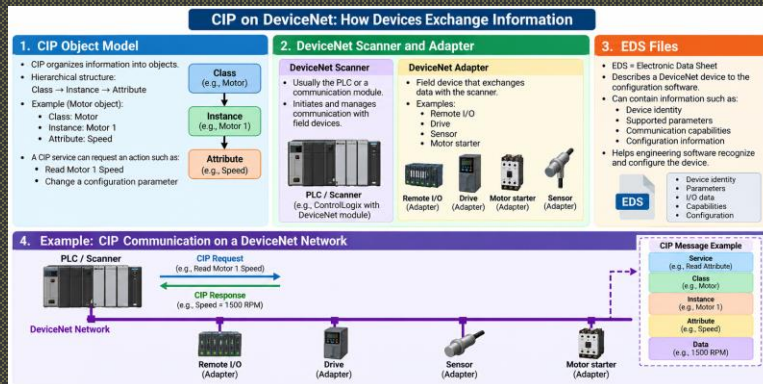
Or Change a configuration parameter

DeviceNet Scanner and Adapter

- Scanner
 - Usually the PLC or communication module.
 - Initiates/manages communication with field devices.
- Adapter
 - Field device that exchanges data with the scanner.
 - Examples:
 - Remote I/O
 - Drive
 - Sensor
 - Motor starter

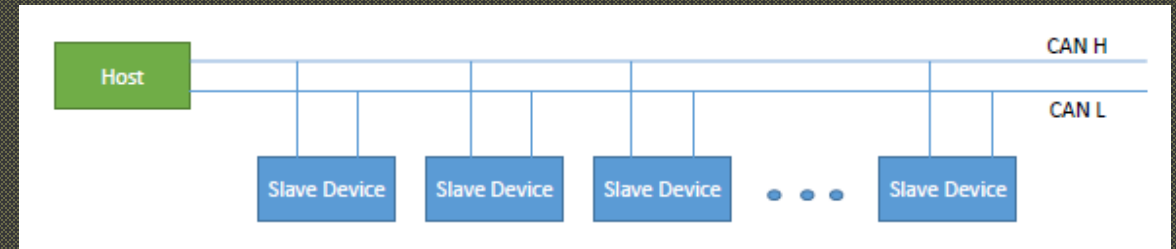
EDS Files

- EDS = Electronic Data Sheet
- Describes a DeviceNet device to the configuration software.
- Can contain information such as:
 - Device identity
 - Supported parameters
 - Communication capabilities
 - Configuration information
- Helps engineering software recognize and configure the device.



3.9 CAN VS. CANopen: THE BASIC DIFFERENCE

- CAN provides the basic communication mechanism between devices.
- It defines:
 - Physical signaling
 - CAN frames
 - Message identifiers
 - Arbitration
 - Error detection
- CAN is message-based:
 - Frames contain a CAN identifier.
 - The identifier represents the message and its priority.
 - It is not simply a device address.
- Lower CAN identifier value generally means higher arbitration priority.
- Classical CAN supports data frames containing up to 8 bytes.
- CAN itself does not define what the application data means.



Example

A CAN frame might contain:

- CAN ID = 0x200
- Data = 55 AA

CAN successfully delivers the message, but CAN itself does not tell us whether 55 AA means:

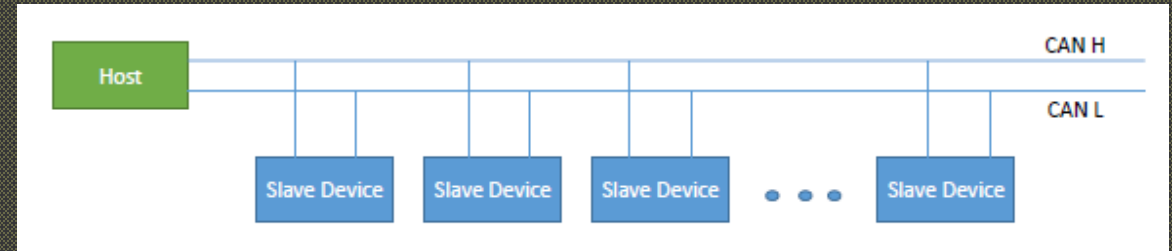
- Motor speed
- Temperature
- Pressure

CAN = Telephone line → just ensures voices travel clearly.

CANopen = Conversation rules → defines what language is spoken and what the words mean.

3.9 WHAT CANopen ADDS TO CAN

- CANopen is a higher-level communication protocol built on CAN.
- Standardized mainly through CiA – CAN in Automation specifications.
- CANopen defines:
 - What CAN messages mean
 - How device data is organized
 - How devices are configured
 - How devices start, stop, and report errors
- Major CANopen communication objects include:
 - PDO – Process Data Object
 - SDO – Service Data Object
 - NMT – Network Management
 - SYNC – Synchronization
 - EMCY – Emergency
 - TIME – Time synchronization
 - Heartbeat / Boot-up
- CANopen also defines standardized device profiles.



Example

A motor controller can define:

- Speed
- Position
- Status
- Control commands

using standardized CANopen objects.

CAN = how the message travels

CANopen = what the message means and how devices use it

3.10 CANopen APPLICATIONS AND INTEROPERABILITY

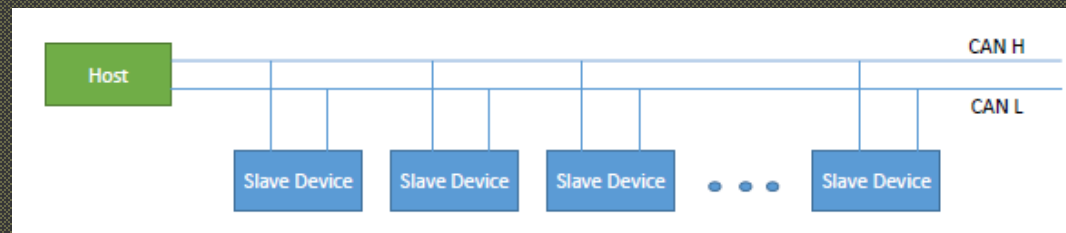
- CANopen was originally developed for embedded and motion-control systems.
- Today it is used in:
 - Industrial machinery
 - Drives and motion systems
 - Medical equipment
 - Off-road vehicles
 - Rail systems
 - Marine systems
 - Building automation
 - Robotics and mobile machines
- Standardized device profiles improve interoperability between manufacturers.
- Device profiles define how similar types of equipment represent their functions.

Example

Two CANopen-compliant motor drives from different manufacturers may expose similar standardized control and status objects.

Device Description Files

- CANopen devices can use EDS – Electronic Data Sheet files.
- An EDS describes:
 - Device identity
 - Supported objects
 - Communication parameters
 - Configuration information



3.10

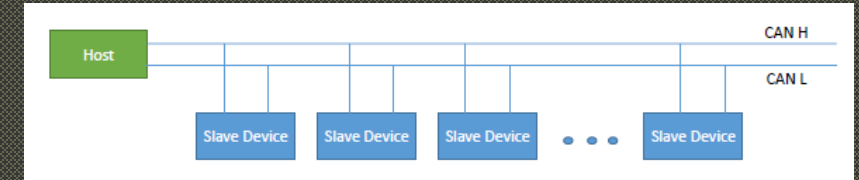
CANopen OBJECT DICTIONARY

The Object Dictionary is one of the most important CANopen concepts.

- It acts like a structured parameter and data table inside every CANopen device.
- Each item is identified by:
 - A 16-bit Index
 - Sometimes an 8-bit Sub-index
- Objects can contain:
 - Configuration parameters
 - Status information
 - Process values
 - Communication settings

Important Index Areas

- 1000h–1FFFh
 - Communication-profile objects
- 2000h–5FFFh
 - Manufacturer-specific objects
- 6000h–9FFFh
 - Standardized device-profile objects



Example

- 6040h → Control word (Start/Stop Motor)
- 6064h → Actual Position

A controller can read or write these objects using CANopen communication services.

Object Dictionary = the device's organized table of parameters, settings, commands, and process data.

3.10

CANopen - SDO: SERVICE DATA OBJECT

- SDO is mainly used to access entries in a device's Object Dictionary.
- Used for:
 - Configuration
 - Parameter changes
 - Diagnostics
 - Reading device information

Communication Model

- Uses client/server communication.
- SDO Client:
 - Requests information or sends configuration data.
- SDO Server:
 - Provides access to its Object Dictionary.

Example

A controller reads a motor's maximum-speed setting:

Controller → Request → Drive

Drive → Value → Controller

SDO Transfer Types

Expedited Transfer

- Used for small amounts of data.
- Up to 4 bytes in Classical CANopen SDO.
- Completed quickly.

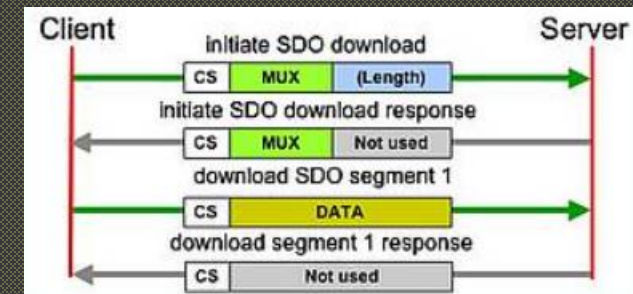
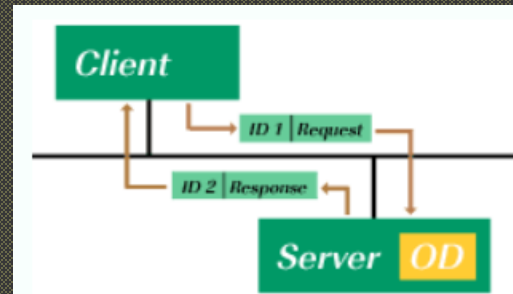
Segmented Transfer

- Used for larger data.
- Data is divided into several segments.

Block Transfer

- Used for larger transfers requiring greater efficiency.
- Transfers multiple segments before acknowledgement.

SDO = configuration and parameter access



3.10 CANopen - SDO CHANNELS AND PARAMETERS

- Every CANopen device normally provides an SDO server.
- Some devices can also act as SDO clients.

SDO Server

- Allows another device to access its Object Dictionary.

SDO Client

- Initiates access to another device's Object Dictionary.

SDO Channel Parameters

Stored in the Object Dictionary:

- 1200h–127Fh
 - SDO server communication parameters
- 1280h–12FFh
 - SDO client communication parameters

These entries define items such as:

- CAN identifiers used for communication
- Node-ID of the communication partner

Example

A PLC acting as an SDO client changes a parameter in a servo drive acting as an SDO server.

Index	Sub-Index	Description
12xx _h	00 _h	Number of entries
	01 _h	COB-ID client-to-server
	02 _h	COB-ID server-to-client
	03 _h	Node-ID of server/client

SDO communication is normally request → response.

3.10

CANopen - PDO: PROCESS DATA OBJECT

- PDO is used for fast exchange of real-time process data.
- Designed for values that must be exchanged repeatedly with minimal overhead.
- A Classical CANopen PDO can carry up to 8 bytes of process data.

Typical PDO Data

- Motor speed
- Motor position
- Torque
- Sensor reading
- Switch status
- Output commands

Example

A motor drive can periodically send:
Actual Speed = 1500 rpm
through a PDO.

PDO Parameter Sets

Each PDO has:

Communication Parameters

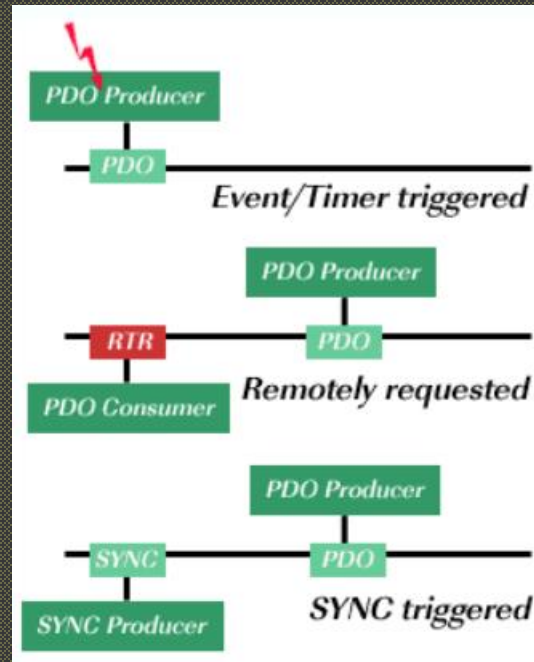
Define:

- CAN identifier
- Transmission method
- Triggering conditions

Mapping Parameters

Define:

- Which Object Dictionary entries are placed inside the PDO



PDO = fast real-time process data

SDO = configuration and parameter access

3.10

CANopen - PDO TRIGGERING AND MAPPING

How a PDO Can Be Transmitted Event-Driven

- Sent when a specified event occurs.

Example:

Temperature > 80°C → send PDO

Synchronous

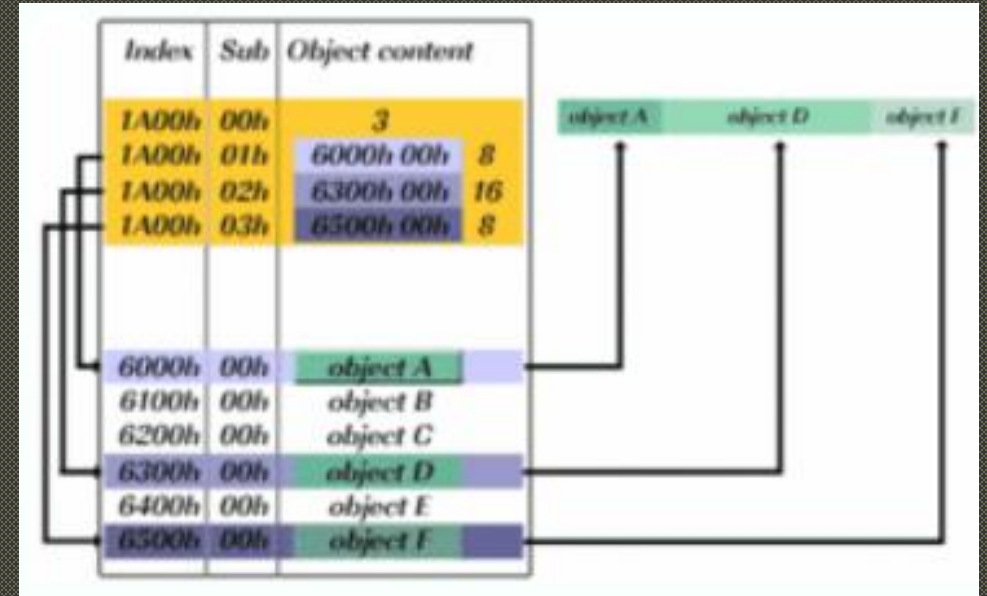
- Sent according to a SYNC message.

Example:

Every SYNC → transmit motor position

Cyclic / Timer-Based

- Sent periodically based on an event timer.



PDO Mapping

Mapping determines which Object Dictionary values are placed inside a PDO.

Example:

A PDO could contain:

Bytes	Data
0-1	Motor Status
2-3	Motor Speed
4-7	Motor Position

Mapping Types

- Static mapping
- Variable mapping
- Dynamic mapping

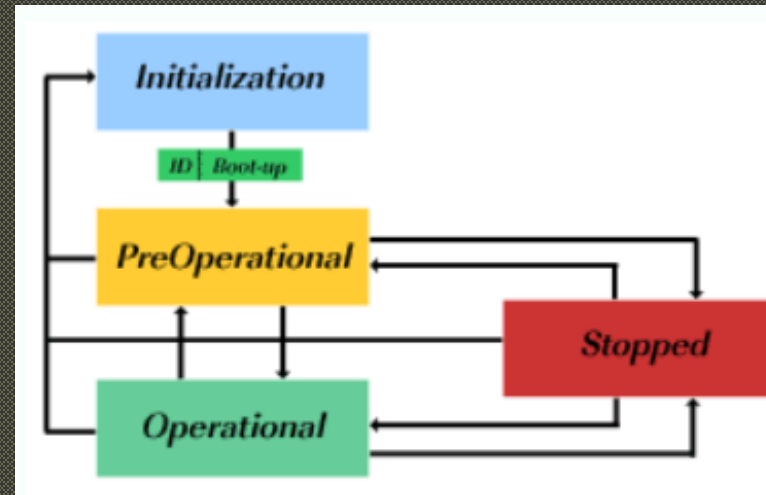
3.10

CANopen - NMT: NETWORK MANAGEMENT

- NMT = Network Management
- Controls the communication state of CANopen devices.
- Every CANopen device has an NMT state machine.
- An NMT Manager sends state-control commands.
- NMT controls whether a device is:
 - Starting
 - Being configured
 - Fully operational
 - Stopped

Main States

- Initialization
- Pre-Operational
- Operational
- Stopped



NMT controls the operating state of each CANopen node.

3.10

CANopen - CANOPEN NMT STATES

Initialization

- Device has just powered up or reset.
- Internal hardware and communication parameters are initialized.
- Device then normally enters Pre-Operational.

Pre-Operational

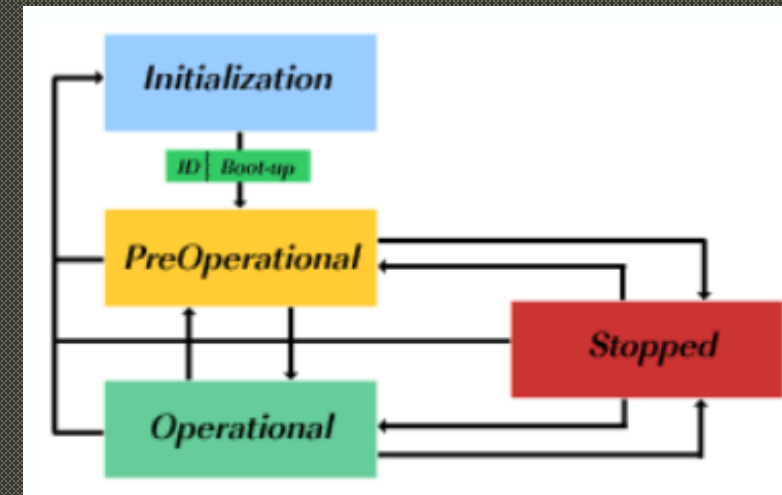
- Communication configuration is possible.
- SDO communication is available.
- PDO process communication is normally not active.
- Useful for:
 - Configuration
 - Parameter setting
 - Network setup

Operational

- Normal operation.
- Full application communication is enabled.
- PDO, SDO, SYNC, heartbeat, and other appropriate services can operate.

Stopped

- Most normal process communication stops.
- Device remains connected to the network and responds to NMT management.



Easy Flow

Initialization → Pre-Operational → Operational

A node can later be:

Stopped → Reset → Restarted

3.10

CANopen - NMT COMMANDS

- NMT commands are sent by the NMT Manager.
- NMT messages use:
CAN-ID = 0

This gives them very high bus priority.

NMT Message Contains

- Command byte
- Node-ID

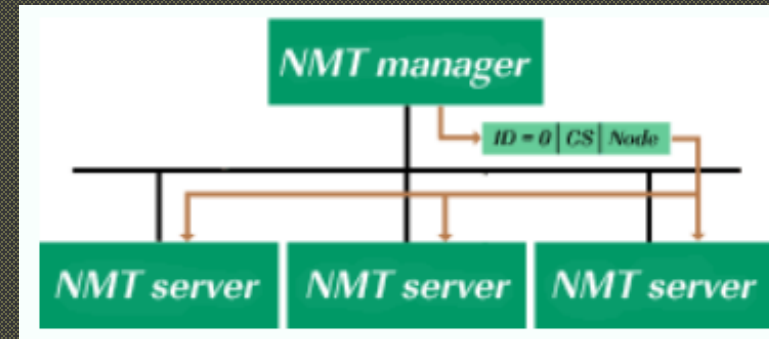
Node-ID

- Specific Node-ID → command one device.
- Node-ID 0 → broadcast command to all devices.

Common Commands

- Start Remote Node
- Stop Remote Node
- Enter Pre-Operational
- Reset Node
- Reset Communication

NMT lets the network manager control the operating state of individual nodes or the entire network.



Example

Command: Start Node 5
causes Node 5 to enter the Operational state.

3.10

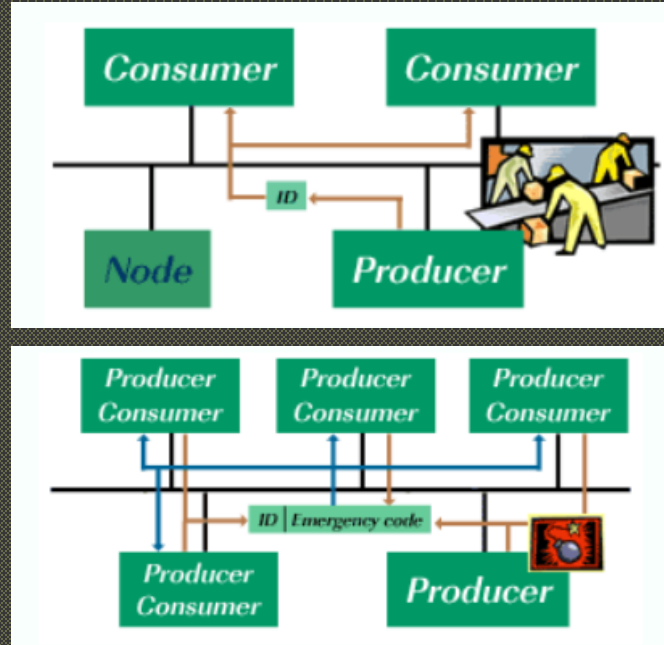
CANopen - SYNC AND EMCY

SYNC – Synchronization

- Used when several devices must perform actions at coordinated times.
- A SYNC Producer periodically sends a SYNC message.
- Other devices act as SYNC Consumers.
- Default CAN identifier is commonly: 0x080

Example

A controller sends SYNC every 10 ms.
Several servo drives then update their position at the same synchronized instant.



EMCY – Emergency Message

- Used when a device detects an important fault.
- Sent immediately when the fault occurs.
- Typical information includes:
 - Error code
 - Error register
 - Manufacturer-specific diagnostic information
- Default CAN identifier is typically: 0x080 + Node-ID

Example

Motor temperature exceeds its limit:
Drive → EMCY message → PLC detects fault → machine response

SYNC = coordinate devices
EMCY = report urgent faults

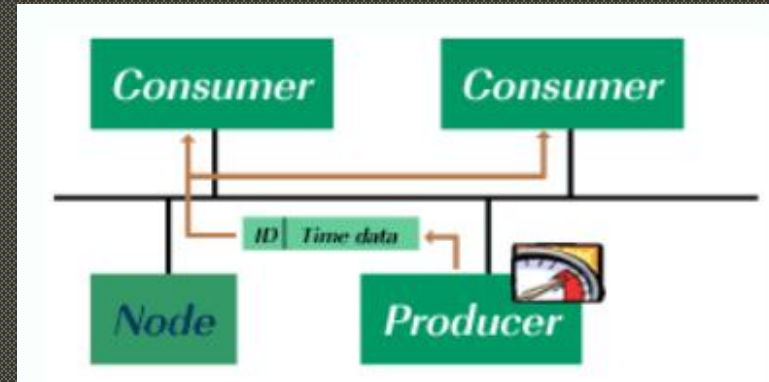
3.10 CANopen - TIME PROTOCOL

- CANopen TIME provides a common network time reference.
- Allows multiple devices to associate events with the same clock.
- Useful for:
 - Event logging
 - Data acquisition
 - Fault records
 - Sequence-of-events analysis
- TIME data includes:
 - Milliseconds after midnight
 - Number of days from the CANopen reference date
- Default CAN identifier is commonly:
0x100

Example

Several devices detect faults at nearly the same moment.

Using a common TIME reference allows the maintenance system to determine the actual event sequence.



- SYNC → keeps devices synchronized.
- EMCY → reports errors.
- TIME → gives a shared clock for all devices.

TIME = shared clock information for CANopen devices.

3.10

CANopen - HEARTBEAT AND BOOT-UP

Heartbeat Protocol

- Used to monitor whether CANopen devices are still operating.
- A Heartbeat Producer periodically transmits its current NMT state.
- A Heartbeat Consumer monitors these messages.
- If heartbeat messages stop arriving:
 - The consumer can detect that the node may have failed or disconnected.
- Heartbeat COB-ID is commonly:
 $0x700 + \text{Node-ID}$

Example

Motor drive sends heartbeat every second.
If the PLC stops receiving the heartbeat, it can generate a communication alarm.

Boot-Up Message

- Sent once when a node finishes initialization.
- Indicates that the device has entered Pre-Operational state.
- Uses: $\text{CAN-ID} = 0x700 + \text{Node-ID}$
with boot-up data indicating startup completion.

Example

A new temperature sensor powers ON:
Sensor → Boot-Up → Controller knows the node has joined the network



- Heartbeat = modern way to check devices automatically.
- Boot-up = signals when a device starts.
- Node/Life-guarding = old polling method, mostly replaced.

- **Boot-Up** = “I have started.”
- **Heartbeat** = “I am still alive.”

3.10

CANopen - HEARTBEAT AND BOOT-UP

CANopen communication objects and services relationship:

- CAN → Moves frames reliably over the network.
- CANopen Object Dictionary → Organizes the device's data and parameters.
- PDO → Fast real-time process data.
- SDO → Read/write configuration and parameters.
- NMT → Controls device operating states.
- SYNC → Synchronizes devices.
- EMCY → Reports urgent faults.
- TIME → Provides common time information.
- Heartbeat → Confirms that nodes are alive.
- Boot-Up → Announces that a node has started.

CAN provides the communication mechanism;

CANopen gives the CAN messages standardized meaning and defines how industrial devices interact.

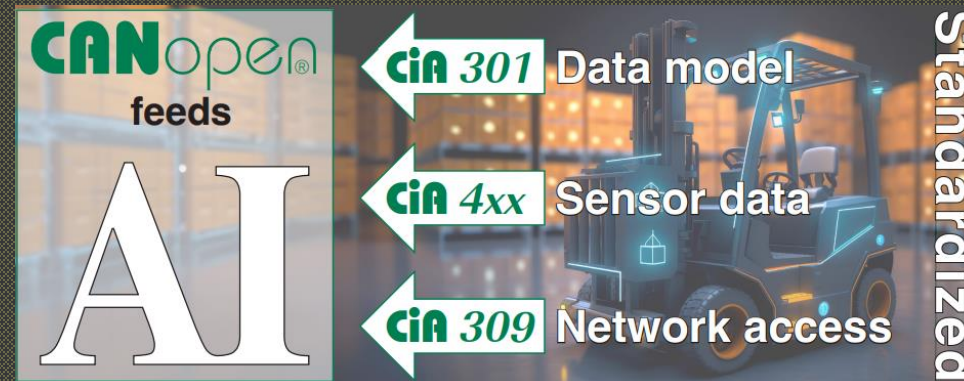
3.10

CANOPEN IN MODERN AUTOMATION, AGVS AND AMRS

- CANopen is widely used in:
 - AGVs – Automated Guided Vehicles
 - AMRs – Autonomous Mobile Robots
 - Servo drives
 - Steering systems
 - Battery-management interfaces
 - Mobile machines
- Standardized CANopen profiles can improve interoperability between equipment from different manufacturers.

Important Profiles

- CiA 301
 - Core CANopen communication profile.
- CiA 402
 - Drives and motion-control profile.
- Other CiA application/device profiles define behavior for specialized equipment.



3.10 CANopen

Integration with Higher-Level Systems

A mobile robot may use CANopen internally for:

- Motor drives
- Encoders
- Actuators
- Sensors

A gateway or industrial computer can then send this information to:

- Ethernet networks
- Robot controllers
- Edge computers
- AI or optimization software



https://www.can-cia.org/fileadmin/cia/documents/publications/cnlm/march_2024/cnlm_24-1_p07_canopen_feeds_ai_holger_zeltwanger_cia.pdf

CANopen does not perform AI.

Instead: CANopen provides standardized device data that higher-level control, analytics, or AI systems can use.

Example Architecture

Motors / Sensors → CANopen → Robot Controller / Gateway → Ethernet → AI or Fleet Management

References

- Practical Data Communications for Instrumentation and Control, John Park, Steve Mackay, Edwin Wright
- Data Communications & Networking With TCP/IP Protocol Suite, Behrouz A. Forouzan
- Fieldbus and Networking in Process Automation, Sunit Kumar Sen
- <http://www.can-cia.de>

INDUSTRIAL NETWORKING

MENG 4010

Thank you!

Discussions?